

《系统架构设计师》

(软考高级)

2026 版三色笔记

简洁高效|重点突出|覆盖全面

第 1 章 绪论

一、本章考试定位

第一章**不是**单纯“概念介绍”，而是后续学习**架构风格、质量属性、架构评估、系统设计、论文写作**的入口。**综合知识题**常考定义、分类、视图、发展阶段、架构师职责；**案例题**常把本章内容隐藏在“**架构选型、质量属性权衡、技术决策**说明”中；**论文题**则常要求考生体现架构师的**系统思维、质量属性**意识和**权衡**能力。

本章备考主线可以概括为：

系统越来越复杂 → 需要整体结构 → 架构承担抽象与**约束** → **架构师负责技术决策与质量属性** → **不同场景选择不同架构风格** → **用视图和文档表达架构**。

二、系统架构的核心概念

1. 架构的教材定义

教材引用 **IEEE 1471-2000** 的定义：架构是体现在**组件**中的系统基本组织，包括**组件**之间的关系、**组件**与环境之间的关系，以及指导系统设计和演化的原则。教材随后用通俗方式说明：**系统架构**是系统整体的高层次结构表示，是系统的骨架和根基，支撑并连接**组件、连接件、约束**规范和演化原则。

考试记忆式：

系统架构=组件+连接件+关系+约束+设计与演化原则

其中：

概念	应掌握口径
系统	为完成特定功能或一组功能而组织起来的 组件 集合
环境/上下文	影响系统开发、运行、政策和 约束 的外部条件
任务	利益相关者通过系统达成目标的用途或操作
架构	对系统整体结构的高层次抽象，不等于代码结构或 详细设计
架构设计	在 需求分析 之后、系统 详细设计 之前，对系统进行 关键 抽象和 技术决策

2. 架构设计的作用

架构设计主要解决五类问题：

问题类型	架构设计的作用
复杂需求问题	将需求转化为可设计、可实现的结构
非功能属性 问题	把性能、 可靠性、安全性、可维护性 等提前纳入设计
生命周期长的问题	通过扩展性、演化能力支撑系统长期运行
组件 集成问题	解决 组件 之间的协作、接口和依赖关系
业务流程再造问题	用架构支撑业务流程重组和系统整合

易错提醒：**架构设计不是“把功能模块画出来”这么简单**，它更关注系统整体结构、**关键约束**和**质量属性**。若题干出现“高并发、可扩展、可维护、可演化、**可靠性、安全性**”等**关键词**，通常已经进入**架构设计**考点。

三、软件架构的发展历程

1. 软件危机与架构产生

软件架构的发展与**软件危机**密切相关。20 世纪 60 年代中期出现大规模**软件危机**，表现为软件生产效率低、质量差。1968 年和 1969 年 **NATO** 会议提出**软件工程**概念。随着软件规模继续扩大，单靠算法和数据结构已难以解决系统复杂性问题，系统整体结构和规格说明的重要性不断提高，**软件架构**由此产生。教材还提到 1995 年 **Standish Group** 对大量软件项目调查，反映**软件工程**项目在进度、需求满足和预算控制方面面临困难。

考试理解：

第一次软件危机→软件工程; 第二轮复杂性上升→软件重用+组件化+软件架构

2. 软件架构四个发展阶段

阶段	时间	核心特征	常考点
基础研究阶段	1968—1994	软件架构 术语出现， 模块化 思想形成	高内聚、低耦合 、接口简单、 信息隐蔽

概念体系和核心技术形成阶段	1999—2000	架构定义、SAAM、多视角表示、组件化技术成熟	Perry/Wolf 公式、IEEE 1471-2000
理论体系完善与发展阶段	1996 年至今	架构描述、分析、设计、测试、演化、复用成为研究重点	ADL、SAAM、ATAM、CBAM、架构风格
普及应用阶段	2000 年至今	软件产品线、ADML、企业级架构实践推广	架构贯穿需求、设计、实现、维护

3. 模块化设计原则

模块化是架构思想的重要基础。教材提出模块分解应遵循以下规则：高内聚、低耦合、模块大小适度、调用链深度不过多、接口简单精炼、尽可能复用已有模块。

考试口诀：

好模块=高内聚+低耦合+适粒度+浅调用+简接口+可复用

易错提醒：

模块化不等于组件化。模块化更偏逻辑划分，组件化强调可组装、可插拔和运行依赖平台/容器。组件化也不等于应用集成，应用集成是把不同平台或方案的软件整合成统一综合应用；组件化比模块化更独立，但比应用集成结合得更紧密。

四、架构描述、分析、设计、测试与演化

1. 架构描述与表示

教材列举了多种架构描述语言，如 C2SADL、Wright、ACME、UniCon、Rapide、Darwin、MetaH、Aesop、Weaves、SADL、xADL 等。考试通常不要求深入掌握语法，但会考“架构描述语言用于描述组件、连接件、约束和系统结构”。

2. 架构分析

架构分析包括：

类型	含义
结构分析	分析组件、连接件、依赖关系和约束
功能分析	判断架构是否支撑功能需求
非功能分析	预测性能、可靠性、安全性、可维护性等质量属性

常见方法：

缩写	名称	记忆重点
SAAM	软件架构分析方法	场景驱动、早期经典方法
ATAM	架构权衡分析法	重点在质量属性之间的权衡
CBAM	成本效益分析法	将经济成本与收益纳入架构决策
SBAR	基于场景的架构再工程	面向已有系统重构与再工程
ALPSM	架构层次的软件可维护性预测	关注可维护性
SAEM	软件架构评估模型	架构评估

易错提醒：ATAM 的关键词是“权衡”。如果题干出现“性能提高会牺牲可维护性”“安全增强会带来复杂度和成本”等，优先联想到 ATAM。

3. 架构设计

架构设计是生成满足用户需求的软件架构的过程。常见方法包括：

方法	出发点
工件驱动	从工件描述中提取架构描述
用例驱动	从用例导出架构抽象
模式驱动	从架构模式导出架构结构
领域驱动	从领域模型导出架构抽象
属性驱动设计 ADD	从质量属性需求出发进行架构设计

4. 架构测试、发现、演化与复用

架构测试关注架构层主要问题，测试策略可对应单元、子系统、集成和验收等阶段。架构发现属于逆向工程，用于从已有系统中提取架构；架构演化来自需求、技术、环境和部署变化；架构复用属于设计复用，比代码复用更抽象，架构模式就是架构复用的重要成果。

五、常用软件架构分类

教材重点列出五类典型架构：**分层架构**、**事件驱动架构**、**微核架构**、**微服务架构**和**云架构**，同时指出 **C/S**、**B/S**、**管道-过滤器**、**PAC** 等也被广泛使用。

架构类型	核心结构	适用场景	质量属性侧重	易错点
分层架构	表现层、业务层、持久层、数据库层	常规企业应用、管理系统、Web 应用	可维护性、职责清晰、分工明确	请求通常逐层处理， 不能 任意跳层
事件驱动架构	事件队列、分发器、事件通道、事件处理器	异步处理、状态变化通知、松耦合系统	解耦、扩展性、响应能力	简单项目中队列、 分发器 、通道可合并
微核架构	小内核 + 插件	插件式平台、IDE、浏览器扩展、规则引擎	可扩展、可插拔	插件 之间应减少依赖
微服务架构	独立部署 单元，远程通信	大型互联网系统、复杂业务域、独立演进系统	独立部署 、弹性扩展、服务自治	微服务 不是 “把模块拆小”，还要求 独立部署 和解耦
云架构	处理单元 + 虚拟中间件	高并发、高扩展场景	扩展性、并发处理能力	数据在内存中复制时仍需考虑持久化

1. 分层架构

最常见的架构形式。通常四层：

表现层→业务层→持久层→数据库层

表现层负责界面与交互，**业务层**实现业务逻辑，**持久层**提供数据访问，**数据库层**保存数据。部分项目会在**业务层**和**持久层**之间增加**服务层**，提供通用接口。

2. 事件驱动架构

事件是状态变化时软件发出的通知。**事件驱动架构**通过事件进行通信，典型组成：

事件队列→分发器→事件通道→事件处理器

适合解耦、异步、消息通知、流程驱动场景。考试中如果题干强调“状态变化触发”“异步通知”“多个处理器响应不同事件”，通常选**事件驱动架构**。

3. 微核架构

又称**插件**架构。**核心**思想是：**内核只保留最小运行功能，业务能力通过插件扩展**。适用于需要动态扩展、**插件化**加载、可插拔能力的系统。

4. 微服务架构

微服务是 **SOA** 的升级。每个服务是**独立部署**单元，服务之间通过 **REST**、**SOAP** 等远程协议通信。实现模式包括：

模式	特点
RESTful API 模式	服务通过 API 提供，云服务常见
RESTful 应用模式	服务通过传统网络或应用协议提供，常见于企业内部
集中消息模式	使用 Message Broker 实现消息队列、负载均衡、统一日志和异常处理，但要防止 单点失败

5. 云架构

云架构主要解决扩展性和并发问题。教材强调其将数据复制到内存，形成可复制的数据单元，并把业务处理能力封装成**处理单元**。访问量上升则启动更多**处理单元**，访问量下降则关闭**处理单元**。**云架构**主要由**处理单元**和**虚拟中间件**组成，**虚拟中间件**又包括**消息中间件**、**数据中间件**、**处理中间件**和**部署中间件**。

六、系统架构常用建模方法

教材将架构建模方法分为四类：**结构模型**、**框架模型**、**动态模型**和**过程模型**。并介绍 **Philippe Kruchten** 的“4+1”视角模型。

1. 四类建模方法

建模方法	关注点	记忆关键词
结构模型	构件、 连接件 、配置、 约束 、风格、性质	最直观、最普遍
框架模型	整体结构，针对特定问题建立结构	不强调细节

动态模型	系统大颗粒行为，如重配置、演化、通信建立/拆除	行为变化
过程模型	构造系统的步骤和过程	构造过程

2. “4+1” 视角模型

考试**必须掌握**：

视角	关注内容	常见理解
逻辑视角 Logical	功能分解、对象模型、业务逻辑	系统 “做什么”
过程视角 Process	并发、进程、线程、同步、性能	系统 “如何运行”
物理视角 Physical	部署节点、网络拓扑、硬件环境	系统 “部署在哪里”
开发视角 Development	模块、包、 组件 、代码组织	系统 “如何开发”
场景视角 Scenarios	用例、 关键 业务场景、验证其他视角	用场景串联和验证

口诀：

逻辑看功能+过程看运行+物理看部署+开发看代码+场景做验证

易错提醒：

“4+1” **不是**简单的五张图，而是用**场景视角**验证和贯穿其他四个视角。案例题中让你说明架构视图时，**不要**只写模块图，要补充运行、部署、开发组织和**关键**场景。

七、架构风格与应用场景

架构风格	典型适用场景
管道-过滤器	系统可分为若干独立处理步骤，如编译器、数据处理流水线
主程序/子程序	结构较清晰、调用关系明确的过程式系统
面向对象	组件 内部设计、对象封装、复用与扩展
虚拟机	解释器、规则引擎、专家系统
C/S、B/S	数据和处理通过网络分布
平台/插件	需要 插件 扩展能力的应用
MVC	用户交互程序、Web 应用表现与业务分离
SOA	企业应用集成、服务复用、跨系统协同
C2	GUI 软件、灵活可扩展的交互系统

考试提醒：现代大型系统通常**不是**单一风格，而是**混合多种架构风格**。案例题若问“选择一种最适合的架构”，要结合题干主要矛盾；若问“总体**架构设计**”，可以采用组合式表达，如“前端采用 **MVC**，后端采用微服务，服务间采用事件驱动，部署采用**云架构**”。

八、系统架构设计师

1. 架构设计师定义

架构设计师是负责**系统架构**的人、团队或组织，是系统或产品线的设计责任人。其**核心**任务是理解、管理、确认并评估非功能性系统需求，给出开发规范，搭建系统实现的**核心**构架，并对架构、**关键**构件和接口进行总体设计，澄清**关键**技术细节。

考试化表达：

架构设计师=系统技术领导+架构责任人+非功能需求把关人+关键技术决策者

2. 架构设计师的职责

职责	考试理解
技术领导	对技术方向、 关键 决策、架构一致性负责
需求到设计的桥梁	将用户需求转化为可实施架构
非功能需求 管理	性能、 可靠性 、 安全性 、 可维护性 、 复用性 、 可测试性
技术决策	选择平台、框架、语言、工具和 关键 方案
团队协同	指导、培养团队，确保架构思想被理解和执行
风险控制	识别 关键 风险，提前通过 架构设计 降低风险

教材强调：项目经理更关注资源、进度、成本，**架构设计师**更关注技术方向和架构决策。两者都是项目中的公共角色，但职责重点不同。

3. 架构设计师的任务

架构设计师在项目中的主要任务：

- 1. 领导与协调整个项目中的技术活动，包括分析、设计和实施。
- 2. 推动主要**技术决策**，并最终表达为**系统架构**。
- 3. 确定**系统架构**，并促使**架构设计**文档化，包括需求、设计、实施和部署等视图。
- 4. 从技术角度看，**核心**任务可归纳为：**抽象设计、非功能设计、关键技术设计**。

4. 架构设计师可由个人或团队承担

架构设计师可以是一个人，也可以是一个团队。复杂项目中，由于业务领域和技术范围广，**架构设计师**角色常由多人共同履行。如果由团队承担，通常需要首席**架构设计师**作为单点协调人，保证架构内聚和决策一致。

易错提醒：

“**架构设计师**”是角色，不一定只对应一个人。一个人也可能履行多个角色；一个复杂系统也可能由多个架构师共同完成架构工作。

九、架构设计师应具备的专业素质

教材列出**架构设计师**应具备的专业知识和能力，**核心是业务与技术平衡**。

素质	备考理解
业务领域知识	不理解业务会导致架构偏离真实需求
技术知识	不必掌握所有 API 细节，但要理解 关键 技术因素和平台能力
设计技能	架构是 关键 设计决策的具体化，设计能力来自长期经验
编程技能	架构师不一定每天编码，但 必须 理解代码和实现 约束
沟通能力	口头、书面、图表表达都重要，是项目成功基础
决策能力	在信息不完整时仍要做出合理决策，并能纠偏
组织策略意识	不只关心技术，也要理解组织权力、资源和支持机制
谈判能力	与利益相关者协商需求、风险、成本和折中方案

架构师的关键能力三角

判断力：识别系统复杂度和脆弱点。

执行力：用合适方案解决复杂度问题。

创新力：创造新的解决方案应对复杂问题。

从工程师成长为**架构设计师**，需要长期积累经验、拓宽视野、深度思考，而**不是**只学习某个框架或工具。

十、架构设计师知识结构

教材将**系统架构设计师**的综合知识结构概括为十个方面：**战略规划、业务流程建模、信息数据架构、技术架构**设计与实现、应用**系统架构**解决方案、基础 **IT** 与资源调配、**信息安全**支持与管理保障、**IT** 审计治理与**需求分析**、**可靠性**与生命周期**质量保障**、新技术新概念分析能力。

考试记忆表：

能力维度	对应考试方向
战略规划	信息系统规划、 企业架构 、系统目标
业务流程建模	需求分析 、业务流程、 UML 、用例
数据架构	数据库、数据模型、数据治理
技术架构	中间件、分布式、微服务、云原生
应用架构	业务系统设计、集成、 组件化
基础设施	网络、服务器、存储、资源调配
信息安全	安全架构、 可靠性 、风险控制
IT 审计与治理	合规、治理、需求获取
质量保障	生命周期、 可靠性 、 可维护性
新技术分析	AI 、云计算、大数据、物联网、区块链等趋势理解

十一、优秀架构设计师的六种角色特质

教材引用 Pat Kua 的观点，优秀**架构设计师**应具备六种角色特质：

角色特质	含义
技术领导者	通过愿景、信任和影响力带领团队
开发者	理解代码、实现约束和技术选型后果
系统综合者	关注部署、测试、性能、安全、运维等整体质量
企业家思维	从成本、收益、风险角度评估技术选择
战略与战术权衡者	在长期一致性与短期敏捷性之间平衡
沟通者	用不同群体能理解的语言表达架构愿景

易错提醒：

优秀架构师不是“最会写代码的人”，也不是“只画图的人”，而是能在业务目标、技术方案、组织资源、成本风险之间进行权衡的人。

十二、从工程师到系统架构设计师的成长路径

阶段	典型时间	典型特征	关键积累
工程师	1~3 年	在指导下完成开发	编程语言、数据结构、开发环境、操作系统、数据库、开发流程
高级工程师	3~5 年	独立完成开发	方案设计经验、知道 Why
技术专家	4~8 年	某领域专家	拓展技术宽度，理解原理、优缺点和场景
初级架构师	5~8 年	独立完成一个系统架构设计	形成架构设计方法论
中级架构师	8~10 年以上	完成复杂系统架构设计	高性能、高可用、可扩展、海量存储等复杂能力
高级架构师	10 年以上	创造新的架构模式	创造性、技术趋势引领

考试重点不在死记时间，而在区分：

高级工程师偏局部方案设计；技术专家偏领域深度；架构师偏系统级抽象、方法论和复杂系统设计。

十三、公式化记忆与答题模板

本章没有传统数学计算公式，但有高频“结构公式”。

1. 架构定义公式

架构=组件+组件关系+环境关系+设计/演化原则

2. Perry/Wolf 架构表达

Software Architecture={elements,forms,rationale}

即：

软件架构=元素+形式+理由/依据

3. 模块化原则公式

好模块=高内聚+低耦合+适粒度+浅调用+简接口+可复用

4. 架构师职责公式

架构师职责=抽象设计+非功能设计+关键技术设计

5. “4+1”视图公式

软件架构=逻辑视图+过程视图+物理视图+开发视图+场景视图

十四、真题对应与应用方式

1. 综合知识题常考

考点	常见问法
IEEE 1471 架构定义	架构包含组件、关系、环境和设计演化原则
模块化原则	高内聚、低耦合、接口简单、信息隐蔽
组件化与模块化区别	组件更强调可组装、可插拔和运行独立性
架构分析方法	SAAM、ATAM、CBAM 等
五类典型架构	分层、事件驱动、微核、微服务、云架构

四类建模方法	结构、框架、动态、过程
4+1 视图	逻辑、过程、物理、开发、场景
架构师职责	技术领导、 关键决策 、 非功能需求 、文档化

2. 案例分析题常考

案例题不会直接问“什么是架构”，而会给出业务背景，让你选择或说明架构方案。例如：

题干关键词	优先联想
分层清晰、职责分离、维护方便	分层架构
异步通知、状态变化、松耦合	事件驱动架构
插件 扩展、 核心 稳定、功能可插拔	微核架构
独立部署 、服务自治、分布式团队	微服务架构
高并发、弹性扩展、按负载启动/关闭 处理单元	云架构
多 质量属性 冲突	ATAM、质量属性权衡
需求到设计可追踪	视图、场景、架构文档

3. 论文题可用表达

论文中可使用如下结构：

项目背景 → 业务痛点 → 质量属性目标 → 架构风格选择 → 关键视图设计
→ 关键技术决策 → 风险与权衡 → 实施效果

例如论述**微服务架构**时，**不要**只写“拆分服务”，要写：

服务边界划分、独立部署、接口协议、服务治理、数据一致性、故障隔离、监控运维、性能与复杂度**权衡**。

第 2 章 计算机系统基础知识

一、本章学习主线

本章不能按“计算机基础常识”轻视。系统架构设计师考试常把它作为后续架构设计能力的底层支撑来考：

硬件资源 → 操作系统资源管理 → 数据库与文件组织 → 中间件与构件 → 嵌入式/网络/语言/多媒体 → 系统工程 → 性能评价。

综合知识题常考概念辨析、分类归属、公式计算；案例题常把性能瓶颈、**数据库**设计、网络选型、嵌入式**实时性**、**安全攸关软件**、Web 服务器性能指标放进工程场景中考；论文题则常用于支撑“高性能、高可用、分布式、云原生、数据架构、嵌入式/物联网系统设计”的基础论证。

二、2.1 计算机系统概述

1. 核心概念

计算机系统是由**硬件、软件及网络**组成，用于接收、存储、处理数据并输出结果的系统。狭义上通常指硬件子系统和软件子系统组成的计算机；多个计算机之间实现数据交换的设备和连接称为**计算机网络**。

对象	考试化理解	常见混淆
硬件	有形实体，如 处理器 、 存储器 、输入/输出设备	不只是 CPU
软件	数据、指令及文档，控制硬件完成任务	不等同于单个程序
系统软件	支撑应用运行、管理资源、提供开发和运行平台	操作系统 、 编译器 、DBMS、网络软件
应用软件	面向特定应用目的开发的软件	办公、财务、图像处理、管理系统等

2. 考点抓手

计算机系统分类没有绝对边界，可从硬件结构、规模、性能、应用领域、服务对象等角度分类。教材中的图 2-1 用于理解“硬件—软件—**系统软件**—**应用软件**”层次，图 2-2 用于理解嵌入式计算机、通用计算机、网络服务器、超级计算机、云计算、移动终端等分类关系。

真题对应

选择题常考“**计算机系统**由什么组成”“**系统软件与应用软件**区别”“硬件与软件边界是否绝对”。

易错提醒：可编程逻辑、**FPGA** 等在设计阶段像软件，运行中表现为硬件逻辑门，说明硬件与软件边界在工程实现中会模糊。

三、2.2 计算机硬件

1. 冯·诺依曼结构与现代硬件组成

冯·诺依曼结构把计算机硬件划分为**运算器**、**控制器**、**存储器**、**输入设备**、**输出设备**。现代计算机中，**运算器**和**控制器**通常集成为**处理器**，输入/输出设备常通过**总线**、**接口**、**外部设备**组织。

考试记忆：

传统五大部件：运控存入出。

现代展开：**处理器**、**存储器**、**总线**、**接口**、**外部设备**。

2. 处理器 CPU

CPU 是计算机运算和控制核心。发展趋势包括：位宽从 4 位到 64 位；能力从运算控制扩展到多级缓存、**总线**和**接口**；内核从单核到多核、异构多核和众核。

分类	代表	特点
CISC 复杂指令集	x86 体系	指令复杂，历史包袱较重
RISC 精简指令集	ARM、Power 等	指令相对规整，现代指令集重要趋势

专用**处理器**常考：

GPU：大量核心，适合并行计算，常用于图形、AI、深度学习。

DSP：面向实时数字信号处理，适合高速采集、滤波、傅里叶变换、矩阵运算。

FPGA：现场可编程逻辑门阵列，适合专用硬件加速和可重构场景。

3. 存储器层次

存储器按与 CPU 距离和速度形成层次结构：

片上缓存 → **片外缓存** → **主存** → **外存**。

层次	常见介质	特点	考点
片上缓存	SRAM	最快、最小、靠近核心	L1/L2
片外缓存	SRAM	比片上略慢、容量较大	L2/L3/ 平台 Cache
主存	DRAM	需刷新，掉电丢失	内存
外存	磁盘、光盘、Flash、磁带	慢、容量大、可持久保存	不等于永久保存

易错提醒：

DRAM 需要刷新；**SRAM** 常用于 **Cache**；Flash 可掉电保存但有寿命限制；**外存**能断电保持数据，但不代表永久保存。

4. 总线、接口、外设

总线是部件间按协议进行数据交换的形式。

分类	含义	示例
内总线/片上总线	芯片内部互连	On-Chip Bus
系统总线	CPU、 主存 、I/O 接口 之间连接	局部 总线 也可视为广义 系统总线
外部总线/通信总线	计算机与外设或系统间互连	USB、SATA、以太网、RS-232 等

总线性能指标：**带宽**、**QoS**、**时延**、**抖动**。

接口是不同功能层之间的通信规则，如 HDMI、USB、RJ45、SATA、RS-232 等。外部设备通过**接口**连接主机并通过指令、数据实现功能。

真题对应

常见问法：

给出 GPU/DSP/FPGA 场景判断**处理器**类型；给出 SRAM/DRAM/Flash 判断存储层次；给出 USB、SATA、PCIe、RS-485 判断**总线/接口**；问**总线**性能指标不只带宽，还包括**时延、抖动和 QoS**。

四、2.3 计算机软件

1. 软件系统分类

计算机软件包括程序、数据及相关文档。按作用和运行环境分为**系统软件**和**应用软件**。

类别	含义	典型内容
系统软件	管理资源、支撑应用运行	操作系统 、编译系统、 数据库 管理系统、网络软件
应用软件	面向某类应用或特定问题	财务、人事、图像处理、办公、游戏、管理系统

2. 操作系统

操作系统是**计算机系统资源**管理者，管理 CPU、**存储器**、文件和 I/O 设备，并为用户和应用程序提供**接口**。

操作系统四大特征

特征	关键词	易错点
并发性	宏观同时、微观交替	单 CPU 不是并行，是轮流执行
共享性	多进程/线程共享资源	分为 同时共享 和 互斥共享
虚拟性	物理实体逻辑化	虚拟内存、虚拟设备
不确定性	执行顺序和时间不可预知	并发导致结果可能不唯一

操作系统类型

类型	关键特征	常考场景
批处理系统	作业成批提交，自动顺序处理	单道/多道批处理
分时系统	时间片轮转，多终端交互	多路性、独立性、 交互性 、及时性
实时系统	在限定时间内响应	控制系统、订票系统、武器控制
网络操作系统	网络资源共享和安全管理	文件、打印、目录服务
分布式操作系统	多计算机协同，对用户呈现统一系统	透明性、 可靠性 、高性能
微机操作系统	Windows、Linux、Mac OS	桌面/服务器基础
嵌入式操作系统	资源受限、实时、可裁剪	VxWorks、μC/OS-II、eCos 等

易错提醒：

实时系统重点是**时间约束**和**可靠性**，不是**交互性**；**分时系统**强调**交互及时响应**；**分布式操作系统**比**网络操作系统**更强调统一管理和透明性。

3. 数据库

数据库是长期存储在计算机中、有组织、统一管理、可共享的相关数据集合。**数据库**不仅存数据，还描述数据之间的联系。

数据库类型

类型	特征	高频考点
关系型数据库	二维表 、关系模型、SQL	主码 、 外码 、 范式 、 完整性
键值数据库	Key-Value	高并发缓存、简单查询
列存储数据库	按列组织	分析型场景、数据仓库
文档数据库	XML/JSON/BSON	半结构化、结构灵活
搜索引擎数据库	倒排索引、全文检索	搜索场景

关系数据库设计步骤

考试必须按顺序记：

需求分析 → 概念结构设计 → 逻辑结构设计 → 物理结构设计 → 应用程序设计 → 运行维护。

阶段	产物/重点
需求分析	数据需求、业务处理需求
概念结构设计	E-R 模型，局部 E-R、全局 E-R、优化
逻辑结构设计	E-R 图转关系模型，确定完整性约束和用户视图
物理结构设计	存储结构、存取路径、数据存放位置
应用程序设计	架构、安全策略、开发计划
运行维护	备份恢复、安全完整性控制、性能监督、重组重构

分布式数据库满足：分布性、逻辑相关性、场地透明性、场地自治性。特点包括集中控制、数据独立、冗余可控、场地自治、存取有效。四层结构：全局外层、全局概念层、局部概念层、局部内层。

4. 文件系统

文件是具有符号名、逻辑完整的一组相关信息项。文件系统是操作系统中统一管理文件的软件和数据集合。

角度	分类
性质用途	系统文件、库文件、用户文件
保存期限	临时文件、档案文件、永久文件
保护方式	只读、读写、可执行、不保护
UNIX 视角	普通文件、目录文件、设备文件

文件逻辑结构：

记录式文件：由记录组成，可定长或不定长。

流式文件：字节流，不划分记录，常按顺序访问。

文件物理结构：

结构	特点	优缺点
连续结构	物理块连续	访问快，但易产生外部碎片，不利扩展
链接结构	物理块不连续，指针连接	扩展方便，随机访问差
索引结构	索引表记录逻辑块到物理块映射	支持随机访问，但索引占空间
多重索引	多级索引扩展大文件	适合大文件管理

5. 网络协议、中间件、构件

网络协议规定通信的数据格式、时序、控制信息和应答信号。常见协议体系包括 LAN、WAN、无线网、移动网协议，互联网核心是 TCP/IP 协议簇。

中间件位于操作系统、网络、数据库之上，应用软件之下，作用是屏蔽底层差异，提供标准接口和协议，实现跨平台、分布式、复用。

中间件类型	作用
消息中间件	异步通信、可靠传输
交易/事务中间件	OLTP、事务协调、负载均衡
数据访问中间件	数据访问、格式转换
Web 服务器中间件	扩展 Web 会话与应用能力
安全中间件	认证、加密、访问控制
架构中间件	跨平台构件集成，如 CORBA、JavaBeans、COM+
专用平台中间件	面向电子商务等领域
网络中间件	网管、接入、网络测试等

构件是自包容、可复用、通过接口访问的软件单元。构件组装模型适合“搭积木式开发”，优点是复用、扩展、并行开发；缺点是构件设计难、第三方质量不可控、学习成本和性能取舍。

主流构件规范：

规范	关键词
CORBA	ORB 软总线、公共对象服务、公共设施
J2EE/EJB	Java 企业级分布式构件、Servlet/JSP/EJB
DNA/COM/DCOM/COM+	Microsoft 平台分布式构件、事务、安全、对象池

真题对应

常考“中间件的位置、类型和作用”“构件与模块区别”“CORBA/J2EE/COM+归属”。

易错提醒：中间件不是应用软件本身，也不等同于操作系统；构件强调接口、自包容、可复用，不是任意代码片段。

五、2.4 嵌入式系统及软件

嵌入式系统是为特定应用构建，把信息处理过程和物理过程紧密结合的专用计算机系统。特点是专用、资源受限、实时性强、可靠性要求高、软硬件可裁剪。

1. 嵌入式系统组成

组成	作用
嵌入式处理器	低功耗、小体积、适应恶劣环境
支撑硬件	存储器、定时器、总线、I/O、专用硬件
嵌入式操作系统	任务调度、资源管理、实时支持
支撑软件	驱动、BSP、HAL、调试工具
应用软件	完成特定控制和业务功能

2. 嵌入式系统特点

高频记忆：

专用性、实时性、可靠性、可裁剪、资源受限、功耗低、体积小、开发环境与运行环境分离、安全性要求高。

分类：

按实时性分为实时系统和非实时系统；实时系统又分强实时和弱实时。

按安全要求分为安全攸关系统和非安全攸关系统。

易错提醒：

强实时系统错过时间约束可能导致灾难性后果；弱实时系统错过时间约束主要导致服务质量下降。

3. 嵌入式软件与安全攸关软件

嵌入式软件通常在宿主机开发、目标机运行。目标机与被控对象直接关联，因此架构设计会直接影响系统功能、实时性和可靠性。安全攸关软件是可能导致不可接受风险的软件，常见于航空航天、轨道交通、核工业、医疗等领域。DO-178B/DO-178C 类标准强调目标、过程、数据三类证据；软件生命周期包括计划过程、开发过程和综合过程。

真题对应

选择题常考嵌入式系统特点、HAL/BSP 作用、强实时/弱实时区别、安全攸关系统定义。

案例题可能给出物联网、工业控制、车载系统，要求从实时性、可靠性、安全性、功耗、接口、调试方式分析架构风险。

六、2.5 计算机网络

计算机网络是把地理上分散、具有独立功能的计算机系统和通信设备连接起来，通过网络软件和协议实现资源共享与信息传递的系统。

1. 网络基本概念

概念	备考抓手
拓扑结构	总线、星型、环型、树型、网状
网络分类	LAN、MAN、WAN、互联网、无线网
网络设备	网卡、集线器、交换机、路由器、网关、防火墙
协议体系	OSI 七层、TCP/IP 四层/五层

2. 通信技术

通信技术围绕信道、编码、调制、复用、交换、差错控制展开。

技术	关键词
基带传输	数字信号直接传输
频带传输	调制后传输
多路复用	频分、时分、波分、码分
交换技术	电路交换、报文交换、分组交换
差错控制	检错、纠错、重传

3. 网络技术与组网

常考内容包括局域网、以太网、无线局域网、广域网、IPv4/IPv6、VLAN、VPN、存储网络、接入网、网络管理、网络安全。

技术	考试重点
以太网	CSMA/CD、交换式以太网、速率演进
VLAN	逻辑隔离广播域，提高管理性和安全性
IPv6	地址空间更大，支持扩展和自动配置
VPN	利用公共网络构建安全专用通道
SAN/NAS	存储网络架构区别
WLAN	无线接入、安全认证、覆盖规划

真题对应

选择题常考 OSI 层次、TCP/IP 协议归属、交换机/路由器/网关功能、VLAN 作用、IPv6 特点。

案例题常考“网络瓶颈定位、接入方案、安全隔离、负载均衡、存储网络选型”。

易错提醒：

交换机主要工作在二层，路由器主要工作在三层；VLAN 不是物理隔离，而是逻辑隔离；VPN 不是专线，而是在公共网络上构建安全通道。

七、2.6 计算机语言

计算机语言是人与计算机之间传递信息的媒介，主要由表达式、流程控制和集合/数据结构组成。教材重点包括机器语言、汇编语言、高级语言、建模语言和形式化语言。

1. 机器语言与汇编语言

类型	特点	易错点
机器语言	二进制指令，计算机直接执行	执行快但难写、难维护、不可移植
汇编语言	用助记符表示机器指令	仍依赖硬件，需汇编程序翻译
高级语言	接近自然语言/数学表达	可移植性好，但需编译或解释

机器指令通常包含操作码和地址码。常见指令格式按地址数分为三地址、二地址、单地址、零地址和可变地址指令。

2. 高级语言

语言	记忆点
----	-----

语言	记忆点
C	高效、可操作硬件、系统级开发
C++	在 C 基础上支持面向对象
Java	跨平台，“一次编写，处处运行”
Python	简洁、库丰富，适合 Web、科学计算、大数据等

3. UML 与形式化语言

UML 是面向对象技术中的标准建模语言，主要由基本构造块、规则、公共机制组成。UML 事物包括结构事物、行为事物、分组事物、注释事物；关系包括依赖、关联、泛化、实现。

形式化方法用数学语言描述系统规格，优点是精确、无二义性，适用于高可靠、高安全系统；缺点是学习成本高、工程推广难。Z 语言是典型形式化规格说明语言。

真题对应

常考“机器语言/汇编/高级语言区别”“编译与解释区别”“UML 关系类型”“形式化方法适用场景”。

易错提醒：UML 是建模语言，不是开发方法；形式化语言适合安全关键系统，但不是所有项目都必须使用。

八、2.7 多媒体

多媒体是将文本、图形、图像、声音、动画、视频等多种媒体综合处理，并形成可交互系统的技术。教材中图 2-35 说明多媒体系统由硬件和软件组成，包括多媒体计算机、板卡、外设、驱动、操作系统、创作工具、应用软件等。

1. 多媒体四大特征

特征	含义
多维化	媒体类型多样
集成性	设备、信息和表现形式集成
交互性	用户可主动控制信息获取
实时性	音视频等媒体具有时间约束

2. 多媒体关键技术

技术	高频考点
视音频技术	视频数字化、视频编码、音频数字化、语音识别/合成
通信技术	传输信道、数据传输、同步、多路复用
数据压缩	有损/无损、即时/非即时、数据/文件压缩
编码标准	JPEG、JPEG2000、MPEG 系列、H.26L 等
VR/AR	桌面式、分布式、沉浸式、增强式

易错提醒：

无损压缩解压后与原数据一致；有损压缩会丢失人眼/人耳不敏感信息，丢失信息不可恢复。JPEG 偏静态图像，MPEG 偏动态图像视频。

九、2.8 系统工程

系统工程是用系统思想和科学方法解决复杂问题的组织管理技术，强调整体性、综合性、协调性、科学性和实践性。本章对应大纲中的“系统工程及应用相关技术”。

1. 系统工程核心思想

系统工程不是只研究单个部件，而是从整体出发，分析系统目标、结构、要素、信息流、控制机制、环境约束，使整体达到较优或最优。

关键词：

整体最优、跨学科、系统思维、需求—设计—评价—修改—再设计循环。

2. 霍尔三维结构

霍尔三维结构是硬系统方法论代表，由三维构成：

维度	内容
时间维	规划、拟订方案、研制、生产、安装、运行、更新
逻辑维	明确问题、确定目标、系统综合、系统分析、优化、决策、实施
知识维	工程、管理、法律、社会科学等专业知识

3. 切克兰德方法

切克兰德方法属于软系统方法论，适合社会经济、组织管理等难以完全量化的问题。核心不是最优化，而是比较与探寻。

步骤：

认识问题 → 根底定义 → 建立概念模型 → 比较及探寻 → 选择 → 设计与实施 → 评估与反馈。

4. 并行工程、生命周期与 MBSE

并行工程强调在产品开发初期组织多职能团队并行协作，目标是提高质量、降低成本、缩短周期和上市时间。

系统工程生命周期一般包括：

探索性研究 → 概念 → 开发 → 生产 → 使用 → 保障 → 退役。

生命周期方法包括：计划驱动、渐进迭代、精益开发、敏捷开发。

MBSE 是基于模型的系统工程，强调用形式化、图形化、关联化模型支持需求、分析、设计、验证和确认。三大支柱：建模语言、建模工具、建模思路。SysML 是系统工程标准建模语言之一。

真题对应

常考霍尔三维结构三维内容、切克兰德方法步骤、硬系统与软系统区别、MBSE 三大支柱。

易错提醒：霍尔方法偏硬系统、工程优化；切克兰德方法偏软系统、社会组织问题；MBSE 不是单纯“画图”，而是用模型贯穿需求、分析、设计、验证确认。

十、2.9 系统性能

系统性能包括硬件性能、软件性能、部件性能和综合性能。本节是计算题和案例诊断高频区。

1. 性能指标

对象	高频指标
计算机	主频、运算速度、运算精度、内存容量、存取周期、吞吐率、响应时间、利用率、RASIS
操作系统	上下文切换、响应时间、吞吐量、资源利用率、可靠性、可移植性
数据库	数据库大小、表规模、索引数量、并发事务能力、连接数、负载均衡
Web 服务器	最大并发连接数、响应延迟、吞吐量
网络设备	吞吐量、线速转发、时延、抖动、丢包率、QoS、冗余能力

RASIS：

Reliability 可靠性、Availability 可用性、Serviceability 可维护性、Integrity 完整性、Security 安全性。

2. 常用性能公式

1) CPU 执行时间

CPU 执行时间：

$$T_{CPU} = \frac{IC \times CPI}{f}$$

符号说明：IC=指令条数，f=时钟频率。

2) MIPS

MIPS：

$$MIPS = \frac{IC}{T \times 10^6}$$

符号说明：IC=指令条数，T=执行时间。

若已知主频和 CPI：

已知主频和 CPI 时：

$$MIPS = \frac{f}{CPI \times 10^6}$$

符号说明：f=时钟频率。

3) 平均 CPI

平均 CPI：

$$CPI_{avg} = \sum(CPI_i \times P_i)$$

符号说明：P_i=第 i 类指令比例。

4) 阿姆达尔定律

设可改进部分比例为 F，该部分加速比为 S，则总加速比为：

阿姆达尔定律总加速比：

$$Speedup = \frac{1}{(1 - F) + \frac{F}{S}}$$

符号说明：F=可改进部分比例，S=该部分加速比。

极限加速比：

极限加速比：

$$Speedup_{max} = \frac{1}{1 - F}$$

3. 性能调整步骤

性能调整前先看三件事：

识别约束 → 指定负载 → 设置性能目标。

调整循环：

收集 → 分析 → 配置 → 测试。

数据库性能调整关注 CPU/内存、数据库设计、数据库管理、进程/线程状态、磁盘空间、日志大小等。应用系统性能调整关注可用性、响应时间、并发用户数、资源占用。

4. 性能评估

评价程序准确程度由高到低：

真实程序 → 核心程序 → 小型基准程序 → 合成基准程序。

基准测试程序包括 Dhystone、Linpack、Whetstone、SPEC、TPC 等。Web 服务器性能评测常用基准性能测试、压力测试、可靠性测试。

真题对应

计算题常给主频、CPI、指令数求执行时间/MIPS；给增强比例和局部加速比求 Amdahl 总加速比。

案例题常让分析“响应慢、吞吐不足、并发连接高、数据库锁等待、CPU 或 I/O 瓶颈”等问题。

易错提醒：阿姆达尔定律中只优化局部，整体加速受未优化部分限制；增强比例 F 不能大于 1；MIPS 越高不一定代表不同体系结构下真实性能一定更好。

第 3 章 信息系统基础知识

一、本章考试定位

本章属于系统架构设计师的基础高频章节，常在综合知识选择题中考概念辨析、阶段顺序、系统类型、ERP/MIS/DSS/ES/OAS 区别；在案例分析中考信息系统建设、系统集成、数据共享、信息孤岛治理、业务流程重构、电子政务或企业信息化架构；在论文中可作为“背景知识+架构论证”的基础素材。

核心主线可以记成：

信息系统基础 → TPS → MIS → DSS → ES/OAS → ERP → 电子政务与企业信息化架构

也就是从事务处理走向管理控制，再走向辅助决策、智能化、集成化和平台化。

二、信息系统概述

1. 信息系统的定义

信息系统是由计算机硬件、网络通信设备、软件、信息资源、信息用户、规章制度组成，以处理信息流为目的的人机一体化系统。它不是单纯的软件，也不是单纯的计算机系统，而是“技术 + 人 + 数据 + 组织规则”的综合体。

必背 5 大功能：

功能	备考理解
输入	获取原始数据，是信息处理的起点
存储	保存数据、资料、信息资源
处理	对数据进行加工，可结合 OLAP、数据挖掘等
输出	向用户提供报表、查询结果、分析信息
控制	对信息处理过程、设备、流程进行控制和管理

易错提醒：

计算机只是信息系统的一部分。信息系统强调人机交互，用户提出问题、使用结果、参与决策，不能把信息系统等同于计算机软件系统。

记忆公式：

信息系统 = 技术平台 + 信息资源 + 用户活动 + 管理制度

2. 信息化的含义

信息化是指在国家宏观信息政策指导下，通过信息技术开发、信息产业发展、信息人才配置，最大限度开发和利用信息资源，推动社会各领域发展的过程。

考试中要抓住两点：

第一，信息化的核心不是“买设备”，而是信息资源开发利用。

第二，信息化不仅是技术变革，也会带来组织、流程、管理方式和生产关系变革。

3. 诺兰模型：信息系统发展的 6 阶段

诺兰模型是本章高频记忆点。它描述组织从手工信息系统向计算机化信息系统发展的规律。

阶段	核心特征	高频考点
初始阶段	少数人使用计算机，多用于报表、统计	常发生在财务部门
传播阶段	应用快速增加，投入增加	出现数据冗余、不一致、难共享
控制阶段	开始控制 IT 投资和系统建设	成立领导小组，采用数据库技术，是从计算机管理转向数据管理的关键
集成阶段	建立集中数据库和统一信息系统	由分散应用走向体系化
数据管理阶段	信息成为重要资源	统一数据库平台、数据管理体系
成熟阶段	IT 与管理深度结合	支持各层管理和决策，整合内外资源

易错提醒：

前 3 阶段偏“计算机时代”，后 3 阶段偏“信息时代”；转折点是信息资源规划。不要把“控制阶段”误认为“已经集成”，控制阶段仍存在部门壁垒和信息孤岛。

4. 信息系统分类

传统信息系统主要包括 5 类：TPS、MIS、DSS、ES、OAS。教材还提到现代企业常见系统如 ERP、WMS、MES、PDM。

类型	中文名称	主要服务对象	核心作用
TPS/DPS	业务/数据处理系统	作业层	处理日常事务数据

类型	中文名称	主要服务对象	核心作用
MIS	管理信息系统	中基层至 管理层	提供管理信息、计划与 控制
DSS	决策支持系统	中高层 决策者	支持 半结构化、非结构化 决策
ES	专家系统	专业领域 用户	模拟专家推理
OAS	办公自动化系统	办公人员、管理人员	办公事务自动化、协同化
ERP	企业资源规划	企业整体	集成 物流、资金流、信息流
WMS	仓储管理系统	仓储/ 物流 部门	库位、收发料、先进先出、盘点
MES/SFC	制造执行系统	车间生产层	生产过程管控、设备集成、防呆
PDM	产品数据管理	研发/工程部门	管理 BOM、物料、工程变更、产品数据

真题对应：

常考“给出场景判断系统类型”。比如银行转账、库存出入库是 **TPS**；企业全局管理**报表**是 **MIS**；高层经营方案比较是 **DSS**；诊断、规则推理是 **ES**；**电子邮件**、会议、文档流转是 **OAS**；**供应链**、生产、财务一体化是 **ERP**。

5. 信息系统生命周期

信息系统生命周期分为 4 个阶段：

产生阶段 → **开发阶段** → **运行阶段** → **消亡阶段**

其中**开发阶段**最关键，又包括：

总体规划 → **系统分析** → **系统设计** → **系统实施** → **系统验收**

阶段	重点任务
产生阶段	提出建设想法，开展需求分析，形成需求分析报告
总体规划	明确目标、总体 架构 、组织流程、实施计划、技术规范
系统分析	建立逻辑模型，分析组织结构、 业务流程、数据流程
系统设计	形成实施方案，包括 架构、数据库 、模块、安全 控制 等
系统实施	将设计转化为可运行系统，用户参与非常关键
系统验收	试运行并检验性能、友好性、 可用性
运行阶段	正式交付用户，系统维护成为核心工作
消亡阶段	更新、改造、扩展、重建或报废

维护类型必背：

维护类型	含义
排错性维护	修复运行中发现的错误
适应性维护	适应环境、平台、法规、业务变化
完善性维护	增强功能、性能和易用性
预防性维护	提前改进，防止未来故障

易错提醒：

运行初期通常排错性、**适应性维护**较多；后期完善性、**预防性维护**增多。**消亡阶段**也要纳入规划，不能认为系统上线后“一劳永逸”。

6. 信息系统建设原则

原则	考点化理解
高层管理人员介入	信息系统 服务企业战略， CIO 是典型角色
用户参与开发	核心用户应全过程、深度参与

原则	考点化理解
自顶向下规划	解决信息不一致、信息孤岛问题
工程化原则	用系统工程、软件工程方法保证质量
创新性、整体性、发展性、经济性	分别对应先进性、完整性、超前性、实用性

易错提醒：

“用户参与”不是只让领导签字，也不是只参加验收，而是核心用户从规划、分析、设计到运行全过程参与。

7. 信息系统开发方法

方法	适用场景	优点	易错点
结构化方法	业务成熟、流程稳定，如银行、电信、零售	阶段清晰、文档规范、便于控制	不适合需求频繁变化场景
原型法	用户需求不明确、需要快速反馈	快速展示、迭代修改、用户参与强	原型不等于最终系统，不能忽视总体架构
面向对象方法	复杂系统、可复用需求高	封装、继承、多态，模型接近现实	不是简单画类图，强调对象与类的抽象
面向服务方法	跨系统集成、服务复用、异构互操作	解耦、复用、共享、快速响应变化	服务不是接口的简单堆砌，需治理和标准

案例答题模板：

当题目出现“系统间难以共享、接口混乱、业务变化快、异构系统集成难”时，可答：采用面向服务/服务化架构思想，通过服务封装、统一接口、服务治理、标准化数据交换提升系统复用性和互操作性。

三、业务处理系统 TPS

1. TPS 概念

TPS 又称电子数据处理系统 EDP，是计算机在管理中最早、最基础的信息系统形式，主要处理财务、销售、库存、合同、工资、订货、发货等日常事务。

TPS 服务于组织的最低层、作业层，处理的问题高度结构化，流程明确。

关键词：原始数据、传票、账簿、报表、日常事务、作业层、结构化处理。

2. TPS 功能

TPS 的数据处理周期包括 5 个阶段：

数据输入 → 数据处理 → 数据库维护 → 文件/报表生成 → 查询处理

阶段	考点
数据输入	信息处理瓶颈，要求准确、快速
数据处理	批处理、联机事务处理
数据库维护	更新组织数据库，支撑 MIS/DSS/ES
文件报表生成	行动文件、信息文件、周转文件
查询处理	按权限进行条件检索

3. 批处理与 OLTP 对比

维度	批处理	OLTP
处理时机	数据积累后定期处理	事务发生即处理
适用场景	工资、银行日终处理、周期报表	银行转账、订票、证券交易
优点	经济，适合大量数据	实时性强，查询及时
缺点	主文件可能过时，不能满足实时查询	成本高，需要授权、容错和安全控制

易错提醒：

批处理不是落后技术，它适合“无须即时更新”的场景；OLTP 不是 DSS，OLTP 强调事务处理的实时性，而 DSS 强调决策支持。

4. TPS 特点

TPS 虽然是最低级的信息系统，但非常关键，因为它是其他信息系统的数据库。TPS 一旦故障，企业日常运作会受影响，如订票系统、银行存取款系统、物料出入库系统。

真题对应：

“某系统支持柜台交易、库存出入库、订单录入、工资核算，并要求数据及时入账”，优先判断为 TPS/OLTP。

四、管理信息系统 MIS

1. MIS 概念

MIS 是在 TPS 基础上发展起来的，以企业整体管理为目标，引入管理方法，对企业信息进行处理，并用于预测、计划、控制、辅助管理。

MIS 由四大部件组成：

信息源、信息处理器、信息用户、信息管理者

2. MIS 结构

MIS 可从两个角度理解：

第一，按决策反馈机制分为开环系统和闭环系统。

第二，按管理层次和职能构成金字塔结构。

管理层次常见为：

战略规划层 → 管理控制层 → 运行控制层 → 业务处理层

职能条线常见为：

销售与市场、生产、财务与会计、人事、后勤、信息处理、高层管理

易错提醒：

下层系统处理量大、结构化程度高；上层系统处理量小，但综合性、外部性、决策性更强。

3. MIS 功能

MIS 的本质是把企业功能和业务过程组织起来，实现跨部门信息联系。典型功能包括：

功能领域	典型内容
销售市场	销售分析、市场计划、客户和竞争者信息
生产	生产计划、质量控制、进度与成本控制
后勤	采购、收货、库存、配送
人事	招聘、培训、工资、考核
财务会计	预算、成本、会计报表、资金计划
信息处理	数据处理、程序维护、系统规划
高层管理	查询、综合分析、决策支持

真题对应：

若题目强调“面向企业整体、整合各业务部门、提供管理报表、支持计划与控制”，一般选 MIS，不选 TPS。

五、决策支持系统 DSS

1. DSS 概念

DSS 是辅助决策者利用数据、模型、方法和知识解决半结构化、非结构化决策问题的交互式系统。

DSS 不是替人决策，而是支持人决策。

2. DSS 核心特征

特征	解释
面向决策者	系统围绕决策需要组织数据和模型

特征	解释
支持半结构化/非结构化问题	比 MIS 更偏高层、复杂和不确定问题
辅助而非替代	决策者仍是主体
动态性	决策过程可随环境、理解、模型变化调整
交互性	通过人机对话融合经验判断与客观数据

3. DSS 功能

DSS 通常具备：

1. 整理并提供内部数据。
2. 收集并提供外部信息，如市场、价格、竞争者信息。
3. 提供反馈信息，如计划完成、销售情况、用户反馈。
4. 管理决策模型，如库存控制、生产调度、投入产出模型。
5. 管理数学、统计、运筹方法，如回归分析、线性规划。
6. 管理数据、模型和方法，支持增删改查。
7. 利用模型和方法进行汇总、分析、预测。
8. 提供人机对话、图形输出。
9. 支持分布式使用和资源共享。

4. DSS 组成与数据仓库

DSS 常见组成包括：

数据库、模型库、方法库、知识库、会话部件/对话子系统

数据仓库用于解决作业层数据库不适合决策分析的问题。其核心是把来自多个系统的数据进行抽取、清洗、确认、整合、装载，形成支持分析查询的数据环境。

数据挖掘常见结果类型：

类型	含义
关联	发现事件之间的联系
定序	识别事件发生顺序模式
分类	按已知类别组织数据
聚类	发现自然分组并形成区分规则

易错提醒：

DSS 关注“提高决策有效性”，不只是提高处理效率；TPS/MIS 解决结构化和信息管理问题，DSS 解决更复杂的决策支持问题。

六、专家系统 ES

1. ES 概念

专家系统是人工智能的重要分支，是一种利用专家知识和推理过程解决特定领域复杂问题的智能计算机程序系统。

记忆公式：

$$\begin{aligned} \text{专家系统} &= \text{知识} + \text{推理} \\ \text{传统程序} &= \text{数据结构} + \text{算法} \end{aligned}$$

2. ES 与普通系统区别

对比项	专家系统	一般计算机系统
功能	解决问题、解释结果、判断与决策	解决问题
处理能力	处理数字与符号	主要处理数字
问题类型	半结构化、非结构化、不确定知识	结构化、确定性知识
适用范围	特定领域	通用程序或确定流程

3. ES 特点

特点	备考理解
超越时间限制	可长期、持续运行
操作成本较低	可复制专家能力，降低长期使用成本
易传递复制	专家知识可沉淀和传播
处理一致	输出稳定，不受人情绪影响
善于处理复杂重复问题	适合诊断、监控、规划等
适用特定领域	通用性差，依赖知识库质量

4. ES 组成

组成	作用
知识库	存放领域知识，包括事实性知识和启发性知识
综合数据库	动态知识库，保存初始状态、中间结果、求解记录等
推理机	核心控制部件，根据知识进行搜索和推理
知识获取模块	编辑、求精、自学习，维护知识库
解释程序	解释推理过程和结论依据
人机接口	用户、专家、知识工程师与系统交互

5. 推理方式

推理方式	又称	适用场景
正向推理	数据驱动	由事实条件推出结论，适合目标解空间大
反向推理	目标驱动	先假设目标，再验证条件，适合解空间较小
双向推理	混合推理	正反向结合，提高效率

真题对应：

医疗诊断、故障诊断、质量监控、专家规则判断、解释推理过程，一般对应专家系统。

七、办公自动化系统 OAS

1. OAS 概念

OAS 是以先进技术和办公设备为基础，帮助办公人员处理数据、文字、声音、图形、图像等信息的综合性人机信息处理系统。

OAS 四大技术支柱：

计算机技术、通信技术、系统科学、行为科学

其中行为科学强调人的办公行为和组织协同，系统科学强调整体设计。

2. OAS 功能

功能	内容
事务处理	文件、通知、报表、日程、会议、文档管理
信息管理	信息收集、加工、传递、存取、分析、反馈
辅助决策	利用信息和模型辅助领导层判断

事务办公系统又可分为：

类型	功能
单机系统	文字处理、日程安排、文档管理、电子报表、数据处理
多机系统	电子会议、电子邮件、语音处理、图像处理、联机检索

3. OAS 组成

组成	内容
计算机设备	主机、终端、外设、输入输出设备
办公设备	电话、传真、复印、轻印刷、电子会议设备
数据通信与网络设备	连接远程节点，支持及时传输
软件系统	系统软件、专用软件、支持软件

易错提醒：

OAS 不是简单的“办公软件集合”，它强调办公活动的信息化、流程化和协同化。

八、企业资源规划 ERP

1. ERP 概念

ERP 是在 MRP II 基础上发展起来的企业级管理思想和信息系统。它以信息技术为基础，集成企业内外资源，为企业提供决策、计划、控制、经营绩效评估的管理平台。

ERP 管理三大流：

物流、资金流、信息流

易错提醒：

ERP 不仅是软件，更是一种管理思想。它从企业内部管理扩展到供应链管理，强调跨部门、跨组织的资源优化。

2. ERP 结构模块

ERP 主要包括 11 个基本模块：

模块	作用
生产预测	支撑经营计划、生产计划大纲、主生产计划
销售管理	连接企业与市场
经营计划/生产计划大纲 PP	细化经营目标，指导 MPS
主生产计划 MPS	明确生产什么、多少、何时交货
物料需求计划 MRP	分解物料需求，生成制造/采购计划
能力需求计划 CRP	校验生产能力与 MRP 需求是否匹配
车间作业计划 PAC	下达车间生产订单，常与 JIT 相关
采购与库存管理	管理采购全过程和物料进出存
质量与设备管理	质量贯穿全过程，设备覆盖寿命周期
财务管理	成本、预算、资金、会计核算
扩展模块	CRM、DRP、SCM、电子商务等

3. ERP 功能

ERP 主要功能包括：

- 1. 支持企业多层面决策。
- 2. 为不同行业提供针对性 IT 解决方案。
- 3. 将企业内部供应链扩展到全行业、跨行业供应链。
- 4. 实现财务、生产、采购、销售、质量、人力、客户、供应链的协同。

真题对应：

若题干出现“打通采购、库存、生产、销售、财务，实现供应链协同和资源计划”，优先判断 ERP。

九、典型信息系统架构模型

1. 政府信息化与电子政务

电子政务概念

电子政务是利用信息技术和相关技术，对政府管理和服务职能进行集成，在网络上实现组织结构和 workflows 优化重组，突破时间、空间、部门分隔，实现政务、政务、商务、事务的一体化管理与运行。

电子政务包括：

- 1. 政府部门内部电子化和网络化办公。
- 2. 政府部门之间信息**共享**和实时通信。
- 3. 政府部门与居民之间双向信息交流。

电子政务五类互动

类型	内容
G2G	政府与政府，部门协同、信息 共享 、 公文流转
G2B	政府对企业，政策发布、许可、认证、监管
G2C	政府对居民，公共服务、证照、公共安全信息
B2G	企业对政府，纳税、统计 报表 、投标、采购供应
C2G	居民对政府，缴费、申报、意见反馈、报警求助

电子政务发展四阶段

信息发布 → 单向互动 → 双向互动 → 网上事务处理

易错提醒：

电子政务成熟的标志不是“有网站”，而是能在线完成真实政务**事务处理**。

2. 企业信息化与电子商务

企业信息化概念

企业信息化是企业利用现代信息技术，通过**信息资源**开发和利用，实现：

生产过程自动化、管理方式网络化、决策支持**智能化**、商务运营**电子化**

其根本目的是提高企业竞争力。

企业信息化目的

目标	说明
技术创新	通过 CAD 、互联网、生产技术与信息技术结合提升技术转化
管理创新	通过流程 整合 、财务资金管理、 供应链 与客户关系管理提高效率
制度创新	通过网络化管理、岗位责任和监管体系推动现代企业制度

企业信息化规划三层融合

企业信息化不是把手工流程简单搬到计算机上，而是技术与业务的深度融合：

层面	重点
企业战略层	分析业务策略、发展方向、 供应链 和管理模式
业务运作层	找出关键流程和关键业务需求
管理运作层	设计管理模式、组织 架构 和信息技术体系

企业信息化方法

方法	适用理解
业务流程重构 BPR	对组织结构和工作方法进行根本性再设计
核心业务应用	围绕企业核心业务推进信息化
信息系统建设	大多数 企业信息化 的重点

方法	适用理解
主题数据库	面向核心业务主题，缓解信息孤岛
资源管理方法	ERP、SCM 等
人力资本投资方法	适用于知识密集型企业

案例答题模板：

对于**企业信息化**建设题，可按“战略规划—流程梳理—数据治理—**系统集成**—分步实施—**安全合规**—持续优化”展开。

十、综合辨析表：TPS、MIS、DSS、ES、OAS、ERP

系统	层次	问题类型	核心资源	典型关键词
TPS	作业层	高度结构化	原始事务数据	录入、记账、库存、订单、转账
MIS	管理层	结构化/半结构化	管理信息、报表	计划、控制、部门管理、报表分析
DSS	中高层	半结构化/非结构化	数据、模型、方法	预测、方案比较、辅助决策
ES	专业领域	半结构化/非结构化	知识库、推理机	诊断、规则、解释、专家经验
OAS	办公层	办公事务	文档、邮件、会议、流程	公文流转、日程、电子会议
ERP	企业整体	集成管理	物流、资金流、信息流	采购、生产、销售、财务、供应链

第 4 章 信息安全技术基础知识

一、本章考试定位与知识主线

1. 题型定位

题型	高频内容	复习重点
综合知识	安全属性、算法参数、RSA 计算、密钥数量、ACL 与能力表、数字签名、攻击类型、安全等级	定义辨析、参数记忆、公式计算
案例分析	安全体系设计、身份认证、权限控制、密钥分配、网络攻击防护、漏洞扫描、风险评估	“风险—控制—验证—改进”闭环
论文支撑	信息系统安全架构、等级保护、数据安全、访问控制、密码应用、安全运维	技术、组织、管理一体化

2. 全章主线

识别资产 → 分析威胁与脆弱性 → 计算或判断**风险** → 选择安全控制 → 实施身份、权限、密码和**审计** → 监测攻击 → 评估**残余风险** → 持续改进

架构师答题不能只列**防火墙**、杀毒软件等产品，而应形成：

安全体系 = 技术体系 + 组织体系 + 管理体系

二、4.1 信息安全基础知识

1. 信息安全五个基本要素

要素	核心问题	典型破坏方式	常见控制
机密性	未授权者不能知晓	窃听、泄露、越权查询	加密、访问控制、数据脱敏
完整性	信息未经 未授权 修改	篡改、插入、删除、重放	摘要、签名、校验、审计
可用性	授权用户需要时能够使用	DoS 、资源耗尽、设备故障	冗余、容灾、限流、备份
可控性	信息流向和行为处于控制之中	权限失控、非法外传	授权、策略控制、数据防泄漏
可审查性	安全行为可以追踪和调查	删除日志、抵赖操作	日志、审计、时间戳、留痕

易错提醒

- 完整性不等于正确性测试。安全语境中的完整性强调信息未被非授权篡改，并可识别篡改。
- 可用性不等于系统永不故障，而是授权主体在需要时能够获得服务。
- CIA 三要素是机密性、完整性、可用性；教材在此基础上增加可控性和可审查性。

2. 信息安全范围

(1) 设备安全

- 稳定性：一定时间内不发生故障的概率。
- 可靠性：一定时间内正常完成规定任务的概率。
- 可用性：设备处于可正常使用状态的概率。

辨析：可靠性强调“完成任务”，可用性强调“当前能否使用”。

(2) 数据安全

核心是保证数据：

不被未授权知晓；不被非法修改或毁坏；在需要时能够正常使用。

(3) 内容安全

属于政治、法律、道德层面的安全要求。现实答题中还应体现违法有害信息治理、内容审核、处置记录和申诉机制。

(4) 行为安全

关注系统操作过程及结果：

不破坏数据秘密性；不破坏数据完整性；行为偏离预期时能够发现、控制和纠正。

3. 信息存储安全

信息存储安全可按四层理解：

身份层：谁能进入

身份鉴别可基于：

- 知识：口令、PIN；
- 持有物：智能卡、令牌；
- 生物特征：指纹、语音、人脸等。

教材重点列举物理特征和安全物品；案例题宜补充多因素认证。

权限层：进入后能做什么

两类基本方法：

- 隔离控制：物理隔离、时间隔离、逻辑隔离、密码隔离。
- 限制权限：用户分类、权限分配、目录与文件控制、临时文件加密及及时清除。

监控层：做了什么

重点监控：当前进程和登录用户；文件所有者、授权及修改时间；口令文件、启动文件和可执行文件变更；登录历史及特权账号操作。

恢复层：出事后怎么办

备份和恢复；病毒检测与隔离；补丁管理；应急响应；事件复盘。

高频口诀

认证解决“你是谁”；授权解决“你能做什么”；审计解决“你做过什么”；加密解决“别人看不懂”；备份容灾解决“出事后能否恢复”。

4. 网络安全威胁

威胁	主要破坏属性	识别关键词
非授权访问	机密性、可控性	冒充、越权、绕过控制
信息泄露或丢失	机密性、可用性	窃听、介质丢失、隐蔽通道
破坏数据完整性	完整性	删除、修改、插入、重放
拒绝服务	可用性	资源耗尽、服务瘫痪
网络传播病毒	完整性、可用性	感染、传播、破坏

教材中的安全目标可归纳为：

访问控制、认证、完整性、审计、保密。

三、4.2—4.3 信息安全作用与组成框架

1. 信息安全的作用

信息系统已经与关键业务、公共服务、个人权益和国家安全深度结合。安全问题不再局限于“数据被偷”，还可能造成：业务中断；经济损失；个人信息泄露；设备和工业控制系统受损；社会公共利益和国家安全受到影响。教材通过 APT 攻击案例说明：网络攻击可以由“窃取信息”发展为“破坏物理设备”。考试中看到长期潜伏、定向攻击、多阶段渗透等描述，应优先考虑 APT。

2. 信息安全系统三大体系

(1) 技术体系

层次	主要安全技术
物理与基础设施	机房、电源、环境、防电磁干扰与泄漏
网络	隔离、防火墙、VPN、传输加密、入侵检测、审计
操作系统	标识鉴别、 最小权限 、可信通路、文件与存储保护
数据库	身份认证 、访问控制、 完整性 、审计、推理控制
终端	终端准入、恶意代码防护、数据防泄漏、介质管理

(2) 组织机构体系

机构通常分为：

- 决策层：确定安全战略、资源和重大**风险**接受原则；
- 管理层：制定制度、组织实施、监督考核；
- 执行层：落实配置、监测、运维、处置等具体工作。

还应设置安全岗位，并覆盖人员的入职、在岗、转岗和离职管理。

(3) 管理体系

包括：

法律与合规管理；内部制度管理；安全教育培训。

“三分技术，七分管理”的考试含义不是轻视技术，而是强调安全不能仅靠设备堆叠。

3. 案例题标准表达

面对“为某系统构建安全体系”的问题，可按以下六层作答：

1. 安全治理与责任；
2. 资产识别和数据分类分级；
3. **身份认证与最小权限**；
4. 边界、网络、主机、应用和数据防护；
5. 日志监控、漏洞管理和事件响应；
6. 备份容灾、**风险评估**和持续改进。

四、4.4 信息加解密技术

1. 保密通信基本模型

设明文为 (P)，密文为 (C)，密钥为 (K)：

$$C = E_K(P)$$
$$P = D_K(C)$$

算法原则上可以公开，安全性主要依赖**密钥**，而不是依靠隐藏算法。

安全性辨析

- **无条件安全**：无论攻击者拥有多少计算资源，也无法从密文确定明文。
- **计算安全**：理论上可以破解，但破解所需时间或成本在现实条件下不可接受。

绝大多数现代密码体制追求的是**计算安全**。

2. 对称加密与非对称加密

项目	对称加密	非对称加密
密钥	加密、解密使用相同或可相互推导的密钥	使用 公钥 和 私钥
优点	速度快，适合大量数据	密钥分配方便，可支持 数字签名
缺点	密钥分配和扩展困难	运算较慢

项目	对称加密	非对称加密
典型用途	文件、报文、数据库字段加密	会话密钥交换、身份认证、数字签名
教材算法	DES、3DES、IDEA、AES	RSA

工程应用原则

非对称密码建立身份和分配会话密钥，对称密码加密大量业务数据。

这就是混合密码体制，也是案例题最常见的正确方案。

3. 对称加密算法

DES

必须记住：

- 明文分组：64 位；
- 有效密钥：56 位；
- 核心迭代：16 轮；
- 主要操作：替代与换位。

易错：DES 的 64 位中包含校验位，有效密钥长度为 56 位。

三重 DES

两密钥 EDE 模式：

$$C = E_{K_1} \left(D_{K_2} \left(E_{K_1}(P) \right) \right)$$

特点：

- 使用 (K_1、K_2)；
- 第一次和第三次使用 (K_1)；
- 有效密钥长度通常按 112 位理解；
- 运算速度约为单 DES 的 3 倍；
- 使用 EDE 可兼容单 DES 场景。

易错：两重 DES 容易受到中间相遇攻击，因此采用三重 DES，而不是简单做两次 DES。

IDEA

- 密钥：128 位；
- 分组：64 位；
- 迭代：8 轮；
- 可由软件或硬件实现。

AES

- 支持 128、192、256 位密钥；
- 安全性和效率优于 DES；
- 适合软件和硬件实现。

2026 工程口径提醒

DES、3DES、IDEA 主要用于教材原理识别和历史算法辨析，不宜在新建系统方案中写成首选算法。国内密码合规场景还应掌握：

- SM2：公钥密码与数字签名；
- SM3：密码杂凑；
- SM4：分组密码。

SM2、SM3、SM4 已有现行国家标准，但是否必须使用，应根据系统类别、适用法规、密码应用要求和项目规范判断，不能笼统写成“所有系统强制使用”。(国家标准开放平台)

4. RSA 算法

计算步骤

选择两个大素数 (p、q)：

$$n = pq$$
$$\varphi(n) = (p - 1)(q - 1)$$

选择 (e、d)，满足：

$$ed \equiv 1 \pmod{\varphi(n)}$$

公钥：

$$(e, n)$$

私钥：

$$(d, n)$$

加密：

$$C \equiv P^e \pmod{n}$$

解密：

$$P \equiv C^d \pmod{n}$$

教材例题

已知：

$$p = 3, q = 11$$

则：

$$n = 33, \quad \varphi(n) = 20$$

取：

$$e = 3, \quad d = 7$$

因为：

$$3 \times 7 = 21 \equiv 1 \pmod{20}$$

当 (P=2) 时：

$$C = 2^3 \pmod{33} = 8$$

解密：

$$P = 8^7 \pmod{33} = 2$$

RSA 易错点

1. 计算欧拉函数时是：

$$(p-1)(q-1)$$

不是 $(pq-1)$ 。

2. 公钥中的 (e) 用于加密或验签，私钥中的 (d) 用于解密或签名。
3. RSA 安全性建立在大整数因子分解困难性的基础上。
4. RSA 不适合直接加密大体量业务数据。
5. “私钥签名”不等于把所有正文简单进行私钥加密，现实中通常对消息摘要执行签名算法。

五、4.5 密钥管理技术

密码系统真正困难的部分往往不是算法，而是密钥的生成、分发、保存、轮换、吊销和销毁。

1. 密钥分类

- 数据加密密钥 DK：直接加密业务数据。
- 密钥加密密钥 KK 或 KEK：用于保护其他密钥。
- 主密钥：用户与 KDC 等可信机构长期共享。
- 会话密钥：一次会话或一定时间内使用，结束后销毁。

2. 无中心方式的密钥数量

若 (N) 个用户之间任意两人都需要一个独立共享密钥，则密钥数量为：

$$K_{\text{pair}} = \binom{N}{2} = \frac{N(N-1)}{2}$$

例如，100 个用户需要：

$$\frac{100 \times 99}{2} = 4950$$

把“有向通信”误写成 $(N(N-1))$ 是常见错误。题目若明确每对用户共享同一密钥，应除以 2。

3. KDC 密钥分配

采用密钥分配中心后：

- 每个用户只需与 KDC 共享一个主密钥；
- 主密钥数量约为 (N)；
- KDC 为通信双方生成临时会话密钥；
- 会话结束后销毁会话密钥。

优点

- 降低长期密钥数量；
- 便于集中管理、轮换和吊销；

- 可通过随机数和时间戳防止重放。

风险

- **KDC** 可能成为性能瓶颈；
- **KDC** 是重要攻击目标；
- **KDC** 故障可能影响整个认证与密钥分配系统。

大型网络可使用分层 **KDC**，将**风险**和管理范围限制在局部区域。

4. 公钥分配方式

方式	优点	主要风险
公开发布	简单	容易被伪造 公钥
公用目录表	便于查询和更新	目录管理者被攻击后影响较大
公钥管理机构	每次查询由可信机构认证	管理机构可能成为瓶颈
公钥证书	可离线交换和验证 公钥	依赖 CA 、证书有效期和吊销管理

5. 数字证书

证书本质上是 **CA** 对“主体身份—**公钥**”绑定关系所作的**数字签名**，可抽象表示为：

$$Cert_A = Sign_{SK_{CA}}(T, ID_A, PK_A)$$

其中：

- (T)：时间信息；
- (ID_A)：主体 A 的身份；
- (PK_A)：A 的**公钥**；
- (SK_{CA})： **CA** 的**私钥**。

接收方使用 **CA 公钥**验证证书。

易错提醒

- 证书不是**公钥**本身；
- 证书不是用来保存用户**私钥**；
- **CA** 一般不替用户生成或保管业务**私钥**；
- 验证**证书**还应检查有效期、用途、信任链和吊销状态。

6. 混合密钥分配

实际安全通信通常采用：

1. 验证对方**证书**；
2. 使用非对称密码协商或封装**会话密钥**；
3. 使用对称密钥加密正文；
4. 使用摘要、MAC 或**数字签名**保证**完整性**和真实性；
5. 会话结束后销毁临时**密钥**。

六、4.6 访问控制与数字签名

1. 访问控制模型

访问控制包括三个基本要素：

- **主体 S**：主动发起访问的实体，如用户、进程、应用。
- **客体 O**：被访问的资源，如文件、数据库、接口。
- **控制策略 KS**：主体能够对客体执行哪些操作的规则集合。

完整实现通常包括：

$$\text{访问控制} = \text{认证} + \text{授权策略实施} + \text{审计}$$

2. 访问控制矩阵

以主体为行、客体为列：

$$M_{ij} \subseteq \{R, W, O, ... \}$$

其中：

- (R)：读；
- (W)：写；
- (O)：管理或拥有。

矩阵直观，但大型系统中空元素很多，直接保存完整矩阵成本较高，因此通常按行、按列或仅保存非空元素。

3. 四种实现方式

实现方式	保存方式	查询优势	查询劣势
访问控制矩阵	完整矩阵	关系直观	矩阵庞大、空值多
ACL 访问控制表	按列、以客体为中心	易查“谁能访问该客体”	难查“该主体能访问什么”
能力表	按行、以主体为中心	易查“该主体拥有哪些权限”	难查“谁能访问该客体”
授权关系表	保存非空三元组或元组	可按主体或客体建立索引	依赖关系表和索引管理

高频判断

- 文件权限列表通常接近 **ACL**。
- 用户持有一组访问令牌或能力票据，通常接近 **能力表**。
- 数据库中保存“用户—对象—权限”记录，通常接近 **授权关系表**。

4. 数字签名目标

数字签名主要实现：

1. 身份真实性；
2. 消息**完整性**；
3. **不可否认性**。

一般过程：

$$\begin{aligned}h &= H(m) \\ s &= \text{Sign}_{SK_A}(h)\end{aligned}$$

接收方计算：

$$h' = H(m)$$

并使用发送方**公钥**验证：

$$\text{Verify}_{PK_A}(h', s)$$

为什么签摘要而不是签全文

- 摘要长度固定，计算效率高；
- 能检测文件变化；
- 签名和正文可分开存储；
- 适合大文件和档案验证。

5. 加密、摘要、签名和证书辨析

技术	主要解决问题	典型操作
加密	机密性	接收方 私钥 解密或共享密钥解密
Hash 摘要	完整性 检测	对消息计算定长摘要
数字签名	真实性、 完整性 、不可否认	私钥 签名、 公钥 验证
数字证书	证明 公钥 属于谁	CA 签发和验证
访问控制	防止未 授权 操作	认证、 授权 、策略执行

易错提醒

- **数字签名**通常不保证正文**机密性**。
- 仅计算 Hash 不能证明发送者身份，因为任何人都可以计算 Hash。
- 加密不能天然实现不可否认。
- “**公钥**加密、**私钥**解密”主要解决保密；“**私钥**签名、**公钥**验证”主要解决身份和不可否认。
- 签名验证成功只说明消息与签名匹配，还需确认**公钥证书**可信、有效且未被吊销。

七、4.7 信息安全抗攻击技术

1. 密钥选择

若密钥长度为 (n) 位，理论密钥空间为：

$$|\mathcal{K}| = 2^n$$

穷举攻击平均尝试次数约为：

密钥应满足：

- 足够长度；
- 随机或不可预测；
- 不使用姓名、生日、字典词等弱口令；
- 不重复使用长期密钥；
- 定期轮换；
- 安全存储并限制导出；
- 泄露后立即吊销和更换。

2. DoS 与 DDoS

DoS

通过资源耗尽、异常请求、协议缺陷或系统配置破坏，使合法用户无法获得服务。

DDoS

攻击者控制大量主机同时攻击目标，具有：

- 来源分散；
- 流量规模大；
- 追踪困难；
- 容易造成带宽、连接、CPU 或内存耗尽。

案例题防护答案

- 入口访问控制和反欺骗过滤；
- 限速、连接数控制、超时和队列保护；
- CDN、负载均衡和弹性扩容；
- 流量清洗与上游联动；
- 服务降级、熔断、隔离；
- 实时监测、告警和应急预案；
- 多地域部署和容灾切换。

3. 欺骗攻击

攻击	原理	防护方向
ARP 欺骗	伪造 IP 与 MAC 映射	DHCP Snooping、DAI、端口安全、网络分段
DNS 欺骗	返回伪造域名解析结果	可信递归服务、DNSSEC、缓存保护
IP 欺骗	伪造源 IP 地址	入口/出口过滤、反欺骗 ACL、会话验证
身份欺骗	冒用账号或凭据	多因素认证、异常登录检测、设备绑定

静态 ARP 绑定只能在小规模、固定环境中提供部分保护，不宜作为大型动态网络的唯一方案。

4. 端口扫描

常见方式：

- TCP 全连接扫描；
- SYN 半连接扫描；
- FIN 扫描；
- 借助第三方或“僵尸主机”的间接扫描。

判断线索

- 完成完整三次握手：全连接扫描；
- 发送 SYN，根据 SYN/ACK 或 RST 判断但不完成连接：半连接扫描；
- 发送异常 FIN 包，根据响应差异判断端口：FIN 扫描。

防护

- 关闭不必要服务和端口；
- 边界 ACL 及主机防火墙；
- IDS/IPS 和行为检测；
- 限制扫描速率；
- 服务暴露面最小化；
- 运维接口仅向管理区开放。

5. TCP/IP 协议栈强化

教材中的具体注册表、旧版 Windows 或传统协议配置主要用于理解历史防护思路。2026 案例题不应机械照抄旧操作系统参数，而应概括为：

- 及时修补协议栈和内核漏洞；
- 关闭无用协议、服务和监听端口；
- 设置连接队列、超时、重传和速率限制；
- 防范 SYN Flood、ICMP 滥用和分片攻击；
- 对管理协议启用认证和加密；
- 持续监控异常流量与连接状态。

6. 漏洞扫描

项目	网络型扫描	主机型扫描
部署	从远程网络发起	在目标主机安装代理或本地检查
优点	部署快、覆盖广、不需逐台安装	检查深入、结果详细、网络负载较低
缺点	受防火墙影响、深度有限、可能增加流量	部署维护成本高、需较高权限
适用	外部暴露面、批量资产发现	配置、补丁、文件、账号和本地漏洞检查

易错提醒

- 漏洞扫描不等于渗透测试。
- 扫描发现“可能存在漏洞”，还需人工验证和风险确认。
- 修复漏洞后应复测，形成发现—确认—修复—复测—关闭的闭环。
- 扫描工具本身也应经过授权，未经许可的扫描可能构成安全和合规风险。

八、4.8 信息安全保障体系与风险评估

1. 安全保护五个等级

按教材及 GB 17859—1999，应掌握以下名称和能力递增关系：

等级	名称	识别重点
第一级	用户自主保护级	用户自主决定和保护资源
第二级	系统审计保护级	增加身份鉴别和审计
第三级	安全标记保护级	引入安全标记及强制访问控制
第四级	结构化保护级	安全机制结构化、系统化
第五级	访问验证保护级	对访问实施严格验证和高强度保护

易错提醒

- 不要交换第二级“系统审计”和第三级“安全标记”。
- 安全等级越高，控制要求越严格，但不意味着所有系统一律选择最高等级。
- 现实项目应先定级，再依据相应标准建设、测评和整改，不能只写教材五个名称。

2. 风险评估基本概念

核心要素

- 资产 A：对组织有价值的信息、系统、设备、服务、人员和声誉。
- 威胁 T：可能造成损害的潜在原因。
- 脆弱性 V：可能被威胁利用的弱点。
- 安全事件：威胁成功利用脆弱性后发生的事件。
- 风险 R：事件发生可能性及其影响的组合。
- 安全措施 C：降低可能性或影响的控制。
- 残余风险：采取措施后仍然存在的风险。

关系模型

$$L = f(T,V)$$

其中（L）为安全事件发生的可能性。

$$R = f(A,L)$$

风险由资产价值、事件可能性和影响共同决定。

除非题目明确给出量化模型，不要擅自把风险固定写成简单乘法。可以使用“风险取决于资产价值、威胁、脆弱性、发生可能性和影响程度”的定性表达。

3. 风险评估准备

实施前应确定：

1. 评估范围；
2. 评估目标；
3. 评估组织及人员；
4. 评估依据和方法；
5. 获得管理层批准；
6. 确定资产、业务边界和数据范围。

4. 风险评估流程

资产识别 → 威胁识别 → 脆弱性识别 → 可能性分析 → 影响分析 → 风险定级 → 措施选择 → 残余风险确认

处理风险的四种基本策略

- 规避：停止或改变高风险活动；
- 降低：增加控制，降低可能性或影响；
- 转移：保险、外包、合同责任分担；
- 接受：经授权接受可容忍的残余风险。

5. 自评估与第三方评估

- 自评估：组织自行开展，熟悉业务、成本较低，但独立性相对不足。
- 第三方评估：由相对独立的专业机构实施，客观性较强，但需要组织配合并控制数据泄露风险。

第 5 章 软件工程基础知识

一、本章考试定位

第 5 章属于系统开发基础知识中的高频章节，综合知识常以选择题考查，案例分析中常与需求变更、需求追踪、软件测试、架构视图、项目管理、质量管理、风险管理结合出题。备考重点不是背概念原文，而是能根据项目场景判断：用什么过程模型、如何控制需求变更、DFD/ER/类图如何建模、测试处于哪个阶段、质量与配置管理如何落地。

本章主线公式：

软件工程 = 过程模型 + 需求工程 + 分析设计方法 + 测试验证 + 质量/配置/风险管理

二、软件工程与过程模型

1. 软件危机与软件工程

软件危机本质上是软件规模扩大后，传统“个人写程序”方式无法支撑复杂系统开发，典型表现为：进度不可预测、成本难控制、功能不满足期望、质量难保证、维护困难、文档不足。软件工程就是用系统化、工程化、可度量的方法开发、运行、维护软件。

记忆公式：软件工程过程 = P-D-C-A

阶段	含义	考试理解
P: Plan	软件规格说明	规定功能与运行限制
D: Do	软件开发	开发满足规格的软件
C: Check	软件确认	确认满足用户需求
A: Action	软件演进	运行后持续改进

真题对应：常考“软件危机表现”“软件工程定义关键词”“软件过程活动”。

易错提醒：软件工程不是单纯“写代码”，而是覆盖开发、运行、维护、管理和文档的工程活动。

2. 瀑布模型

瀑布模型按阶段顺序推进：需求分析 → 系统设计 → 程序设计 → 编码实现 → 单元测试 → 集成测试 → 系统测试 → 运行维护。其特点是阶段间因果关系强，前一阶段输出是后一阶段输入，每阶段结束有里程碑审查。

适用场景	不适用场景
需求明确、变更少、合同约定强、文档要求高	需求不清、用户难描述、业务变化快、需要快速反馈

易错提醒：瀑布模型最大问题不是“没有测试”，而是严格串行、反馈晚、需求变更代价高。

3. 原型化模型

原型模型适合**用户需求**不明确时，通过快速**构造**原型帮助用户确认需求。原型可通过界面模拟、真正开发原型、寻找类似系统比较三种方式形成。

类型	含义	考点
抛弃型原型	用于需求确认，确认后废弃	之后可回到瀑布式完整开发
演化型原型	在原型上不断补充完善	最终演进为正式产品

真题对应： 给出“用户说不清需求、界面交互不确定、报表查询不明确”等场景，优先选**原型法**。

易错提醒： 大型复杂系统如果没有成熟工具或现成原型，快速原型可能反而失控。

4. 螺旋模型

螺旋模型是**生命周期模型** + **原型模型** + **风险分析**的结合。每一轮迭代包含四个部分：

目标设定 → **风险分析** → **开发和有效性验证** → **评审**

它适用于**大型、复杂、高风险**的软件项目，可以多轮逼近目标系统。

高频判断：

场景关键词	对应模型
需求稳定、阶段清晰	瀑布模型
需求不清、需要用户反馈	原型模型
风险高、规模大、需多轮评估	螺旋模型
快速交付、拥抱变化	敏捷 方法

易错提醒： **螺旋模型**的核心不是“迭代”，而是每轮都强调**风险分析**。

5. 敏捷模型

敏捷方法产生于需求快速变化、交付周期缩短的背景下。其**核心特征**是：

核心思想	解释
适应型 而非 预设置	欢迎变化，用反馈机制控制不可预测过程
以人为本 而非以过程为本	强调开发人员创造性、面对面沟通和技术决策权
迭代增量式开发	小版本、小步快跑、持续反馈

主要敏捷方法：

方法	关键词	考试记忆
XP 极限编程	交流、朴素、反馈、勇气	轻量、严谨、小周期、快速反馈
水晶方法	Alistair Cockburn，以人为中心	根据项目与环境选择不同 Crystal 成员
Scrum	项目管理 、 Product Backlog 、 Sprint	需求按商业价值排序，每轮交付潜在可用增量
FDD	特征驱动	人、过程、技术；围绕特征设计和构建

真题对应： “产品 Backlog、**Sprint**、迭代结束交付产品增量”一般指 **Scrum**。

易错提醒： **敏捷**不是“没有过程”，而是轻过程 + 强反馈 + 持续交付。

6. RUP 统一过程

RUP 是一种重量级过程，特点是：用例驱动、以体系结构为中心、迭代和增量。

RUP 记忆公式： 9 **workflow** + 4 阶段 + 4 核心概念 + 3 特点

分类	内容
9 个核心 workflow	业务建模、需求、分析与设计、实现、测试、部署、配置与变更管理、 项目管理 、环境
4 个阶段	初始 、 细化 、 构造 、 移交
4 个核心概念	角色 Role、 活动 Activity、 制品 Artifact、 workflow Workflow
3 个特点	用例驱动、体系结构为中心、迭代增量

RUP “4+1” 视图模型：

视图	主要关注者	关注点
用例视图	分析人员、测试人员	系统行为
逻辑视图	最终用户	系统功能
实现视图	程序员	配置、装配、代码组织

视图	主要关注者	关注点
进程视图	系统集成人员	性能、吞吐率、可伸缩性
部署视图	系统工程师	安装、发布、拓扑结构

易错提醒： RUP 的“迭代”不是重复做同一件事，而是围绕不同用例逐步**细化**和实现。

7. CMM/CMMI 成熟度

CMMI 用于指导软件过程改进和开发能力评估。五级成熟度从低到高为：

等级	名称	关键词
Level 1	初始级	随意、混乱，依赖个人英雄主义
Level 2	已管理级	项目级过程被策划、执行、监督、控制
Level 3	已定义级	组织级标准过程制度化
Level 4	量化管理级	建立定量目标，过程性能可预测
Level 5	优化级	持续过程改进，创新与增量改进

真题对应：“标准流程制度化”选 3 级；“定量管理、过程性能可预测”选 4 级；“持续优化、创新改进”选 5 级。

三、需求工程

1. 需求层次

软件需求分为三个层次：

层次	含义	典型表达
业务需求	组织或客户的高层目标	为什么要做
用户需求	用户要完成的任务和期望	用户要做什么
功能需求	开发人员必须实现的软件功能	系统必须提供什么功能

补充：**非功能需求**包括性能、标准规范、外部接口、设计约束、实现约束、质量**属性**等。

易错提醒：“用户需求”不等于“功能需求”。**用户需求**偏任务和期望，**功能需求**偏系统实现能力。

2. 需求工程活动

需求工程覆盖体系结构设计之前的开发**活动**，主要包括：

活动	作用
需求获取	从用户、现场、业务流程中发现需求
需求分析	建立概念模型， 抽象 现实语义
需求规格说明	形成 用户需求文档 和软件 需求规格说明
需求确认与验证	检查完整性、正确性、一致性、可测试性、可行性
需求管理	追踪、 变更控制 、 版本控制 、状态控制

记忆公式：需求工程 = 获取 + 分析 + 规格 + 验证 + 管理

$$\text{需求工程} = \text{获取} + \text{分析} + \text{规格} + \text{验证} + \text{管理}$$

真题对应：问“需求规格说明通过**评审**后形成什么？”——形成**需求基线**。

3. 需求获取

需求陈述应说明**做什么**，而不是**怎样做**。

常见需求获取步骤：

1. 建立高层业务模型。
2. 定义项目范围和高层需求。
3. 识别用户**角色**和用户代表。
4. 获取具体需求。
5. 确定业务**工作流**和业务规则。
6. 整理并总结软件综合要求。

常见需求获取方法：

方法	适用场景	注意点
用户面谈	最常见、最有效	需准备问题，复查记录
专题讨论会	多方涉众快速达成共识	可解决需求冲突
问卷调查	收集统计倾向	不适合探索复杂模糊问题

方法	适用场景	注意点
现场观察	理解真实业务流程	适合流程复杂场景
原型化	界面、报表、交互不确定	通过迭代反馈澄清需求
头脑风暴	新业务、新产品、新 App	激发创意，但需整理收敛

4. 需求变更

需求变更是常态，但无控制的变更会导致项目失控。**变更控制**过程不是阻碍变更，而是确保合理变更被采纳，不合理变更被过滤，变更影响被评估。

需求变更流程：

问题识别 → 问题分析和变更描述 → 变更分析和成本计算 → 决策 → 变更实现

变更成本不仅包括改需求**文档**，还包括对设计、实现、测试、计划、资源、进度、质量目标的影响。

易错提醒：

计划驱动模型中，重大变更可能要回溯到**需求分析**阶段。

敏捷模型中，变更通常纳入后续迭代。

接受重大变更后，可能需要重新协商交期、人力、范围或质量目标。

5. 需求追踪

需求追踪的目标是维护**需求—设计—编码—测试**之间的一致性，常通过需求跟踪矩阵实现。

类型	检查方向
正向跟踪	每个需求是否在设计、代码、测试中有对应实现
逆向跟踪	设计、代码、测试是否都能回溯到需求来源
双向跟踪	正向 + 逆向

真题对应： 问“如何确认所有测试用例都有需求依据？”——**逆向跟踪**。

易错提醒： **需求追踪**信息必须随项目演进同步维护，否则后期很难重建。

四、系统分析与设计

1. 结构化方法 SASD

结构化方法也称面向功能或面向数据流的方法，包括：

阶段	方法
分析	SA： 结构化分析
设计	SD： 结构化设计
编程	SP： 结构化编程

1.1 结构化分析 SA

结构化分析使用**数据流图 DFD**、**数据字典**、结构化语言、判定表、判定树表达需求。

DFD 四元素：

元素	含义
数据流	数据在系统中的流向
处理 /加工	对数据进行转换
数据存储	数据保存位置
外部项	数据源或数据终点

DFD 建模步骤：

明确目标与范围 → 建立顶层 **DFD** → 构建第一层分解图 → 开发层次结构图 → 检查确认 **DFD**

DFD 检查规则高频考点：

父图数据流必须在子图中体现。

一个**处理**至少有一个输入和一个输出。

一个**数据存储**应有流入和流出数据流。

一个数据流至少一端连接**处理**。

模型信息应完整、正确、一致。

易错提醒： **DFD** 描述“数据如何流动和被加工”，**不是系统部署图**，也**不是程序流程图**。

1.2 数据字典

数据字典是对 DFD 中元素的文字定义集合，主要描述：

条目	说明
数据项	不可再分的数据单位
数据结构	数据项或数据结构的组合
数据流	数据结构在系统中的传输路径
数据存储	数据保存位置及存取特征
处理过程	输入、输出和处理说明

真题对应： DFD 解决“图形化功能模型”，数据字典解决“元素含义定义不清”的问题。

1.3 结构化设计 SD

结构化设计以 SRS、DFD、数据字典为基础，采用自顶向下、逐步求精、模块化思想。

概要设计：确定系统结构、模块划分、模块功能、接口和调用关系。

详细设计：为每个模块设计实现细节。

模块化设计核心原则：

原则	含义
信息隐藏	隐藏内部实现，只暴露必要接口
抽象	忽略非本质细节，控制复杂度
模块化	将系统分解为功能相对独立模块
高内聚低耦合	模块内部联系强，模块之间依赖弱

耦合强度从低到高：

非直接耦合 < 数据耦合 < 标记耦合 < 控制耦合 < 通信耦合 < 公共耦合 < 内容耦合

内聚强度从高到低：

功能内聚 > 顺序内聚 > 通信内聚 > 过程内聚 > 时间内聚 > 逻辑内聚 > 偶然内聚

易错提醒： 考试最容易反着问：耦合越低越好，内聚越高越好。

1.4 结构化编程 SP

结构化程序设计采用顺序、选择、循环三种基本控制结构，要求单入口、单出口。

记忆公式：程序 = 算法 + 数据结构

程序 = 算法 + 数据结构

32 字原则：自顶向下，逐步细化；清晰第一，效率第二；书写规范，缩进格式；基本结构，组合而成。

1.5 数据库概念结构设计

数据库设计包括：需求分析、概念结构设计、逻辑结构设计、物理结构设计、数据库实施、运行维护。

E-R 图元素：

元素	图形表示	含义
实体	矩形	客观可区分的事物
属性	椭圆	实体特征
联系	菱形	实体之间的关系
主属性	下画线	唯一标识实体的属性

联系类型：1:1、1:N、M:N。

易错提醒： 联系也可以有属性，例如“学生选修课程”的“成绩”属于联系属性，而不单独属于学生或课程。

五、面向对象方法

1. 面向对象核心思想

面向对象方法认为客观世界由对象组成，对象包括属性和操作，对象之间通过消息通信，具有封装、继承、多态等特性。

面向对象开发过程通常包括：需求分析、系统分析、系统设计、系统实现，但阶段边界不如结构化方法严格，常迭代推进。

2. OOA 面向对象分析

OOA 模型包括 5 个层次和 5 个活动：

5 个层次	主题层、对象类层、结构层、属性层、服务层
5 个活动	标识对象类、标识结构、定义主题、定义属性、定义服务

OOA 原则：抽象、封装、继承、分类、聚合、关联、消息通信、粒度控制、行为分析。

OOA 基本步骤：

- 1. 确定对象和类。
- 2. 确定结构。
- 3. 确定主题。
- 4. 确定属性。
- 5. 确定方法。

3. OOD 面向对象设计

OOD 是 OOA 的延续，核心是设计类及类之间关系。类可分为三类：

类别	作用	识别方法
实体类	保存需要持久化的信息	从业务名词、数据库表、领域对象中找
控制类	控制一个或多个用例流程	常由动宾短语转化而来
边界类	系统与外部交互接口	窗体、报表、接口、终端、传感器等

易错提醒：

- 实体类通常有属性，不一定有操作。
- 控制类通常有方法，不一定有属性。
- 边界类可有属性也可有方法，负责隔离外部变化。

4. OOP 面向对象编程

记忆公式：OOP = 对象 + 类 + 继承 + 多态 + 消息

$$OOP = 对象 + 类 + 继承 + 多态 + 消息$$

OOP 的三大基本特征：

特征	含义
封装	数据与操作绑定，隐藏内部细节
继承	子类复用父类属性和行为
多态	同一消息作用于不同对象产生不同行为

对象持久化：对象存在于内存中，若要长期保存，需要保存到数据库或持久化存储。常见 ORM 思想是将对象映射到关系数据库表。

六、软件测试

1. 软件测试目标

软件测试用于发现错误、验证系统是否满足需求、支持质量评价和风险评估。测试应贯穿软件开发全过程，而不是只在编码结束后进行。

2. 测试方法分类

分类维度	类型	含义
是否运行程序	静态测试	不运行程序，检查文档、代码、结构
是否运行程序	动态测试	运行程序，比较实际结果与预期结果
是否关注内部结构	黑盒测试	不看内部结构，按需求规格测试功能
是否关注内部结构	白盒测试	基于内部逻辑、路径、控制流设计用例
折中	灰盒测试	既关注输入输出，也关注部分内部逻辑
执行方式	人工测试/自动化测试	人执行或工具自动执行

白盒测试常见覆盖：语句覆盖、判定覆盖、分支覆盖、路径覆盖。

易错提醒：黑盒测试不是“不懂业务”，而是不关注程序内部结构；白盒测试不是只测功能，而是关注内部逻辑路径。

3. 测试阶段

阶段	主要对象	主要方法	重点
单元测试	模块	静态 + 白盒为主，必要时黑盒补充	发现编码错误和模块功能问题
集成测试	模块组装	黑盒 + 白盒	发现接口和组装问题
系统测试	完整系统	黑盒为主	验证系统是否满足需求
性能测试	性能指标	自动化工具	负载、压力、峰值、异常条件

系统测试常包括：功能测试、**性能测试**、健壮性测试、安装/卸载测试、界面测试、压力测试、可靠性测试、安全性测试。

真题对应：

- “模块接口错误” 常对应**集成测试**。
- “整个系统是否满足需求规格” 对应**系统测试**。
- “修改后原有功能是否受影响” 对应**回归测试**。
- “模拟正常、峰值、异常负载” 对应**性能测试**/负载测试/压力测试。

七、净室软件工程

净室软件工程强调在规约和设计阶段消除错误，通过数学和统计方法追求零缺陷或近零缺陷。其思想不是“先开发再大量调试”，而是**第一次就尽量正确**。

1. 理论基础

理论	含义
函数理论	程序可看成输入到输出的映射，要求完备性、一致性、正确性
抽样理论	不可能穷尽所有测试，通过统计抽样评估可靠性

2. 技术手段

技术	重点
统计过程控制下的增量开发	小增量、受控迭代
基于函数的规格与设计	黑盒、状态盒、明盒
正确性验证	以验证代替大量调试
统计测试与软件认证	基于使用模型抽样测试

易错提醒： 净室强调正确性验证，但教材也指出完全不做传统模块测试并不现实。

八、基于构件的软件工程 CBSE

CBSE 强调通过可复用**构件**组装系统，体现“购买而不是重新**构造**”的思想。开发重心从**实现**转向**集成**。

1. 构件特征

特征	含义
可组装性	通过公开接口与外部交互
可部署性	可作为独立 实体 部署运行
文档化	用户可据 文档 判断是否满足需求
独立性	可独立组装部署，依赖需显式声明
标准化	符合 构件 模型标准

常见**构件**模型：Web Services、EJB、.NET。

2. CBSE 过程

系统需求概览 → 识别候选**构件** → 根据**构件**修改需求 → 体系结构设计 → **构件**定制与适配 → 组装**构件**创建系统

与传统开发的区别：

- 早期更需要完整需求，以便识别**构件**。
- 可根据可用**构件**调整需求。
- 架构设计阶段非常关键，因为**构件**模型和平台会限制**构件**选择。
- 开发**活动**更多表现为集成、适配和胶水代码编写。

3. 构件组装方式

方式	含义
顺序组装	一个 构件 输出作为下一个 构件 输入
层次组装	一个 构件 直接调用另一个 构件 服务
叠加组装	多个 构件 合并成新 构件 ，对外提供新接口

接口不兼容类型：

类型	含义
参数不兼容	操作名相同，但参数类型或个数不同

类型	含义
操作不兼容	请求接口和提供接口操作名不同
操作不完备	一个接口只是另一个接口的子集

解决方式：编写**适配器构件**转换接口。

九、软件项目管理

1. 项目管理概述

软件**项目管理**对象是**软件工程**项目，目标是在预定**成本、进度、质量**下完成项目。管理重点包括：人员 People、产品 Product、过程 Process、项目 Project

2. 软件进度管理

进度管理包括：**活动**定义、**活动**排序、**活动**资源估计、**活动**历时估计、制定进度计划、进度控制。

WBS 是**进度管理**基础：

WBS 要点	考试关键词
以可交付成果为导向	定义整个工作范围
最底层是工作包	工作包应由唯一主体负责
树形结构不宜过深	一般不超过 6 层
每个工作包有交付成果	便于责任分配和进度控制

3. 软件配置管理 SCM

SCM 是标识、组织和控制修改的技术，目标是标识变更、控制变更、确保变更正确实现、向相关人员报告变更。

核心内容：

内容	作用
版本控制	记录文件变更，支持并行开发、分支与合并
变更控制	管理外部 需求变更 和内部修复变更

易错提醒：配置管理不只是“版本管理”，还包括**变更控制**、状态记录和一致性维护。

4. 软件质量管理

软件质量是软件与明确和隐含需求相一致的程度。质量因素可从三类理解：

类别	典型因素
产品运行	正确性、健壮性、效率、完整性、可用性、风险
产品修改	可理解性、可维修性、灵活性、可测试性
产品转移	可移植性、复用性、互操作性等相关理解

SQA 软件质量保证关注预防缺陷、过程可见、独立审查、贯穿全过程。

SQA 主要任务：

1. **SQA** 审计与**评审**。
2. **SQA** 报告。
3. **处理**不符合问题。

5. 软件风险管理

风险管理目标是预防风险，基本**活动**包括：

体系	阶段
Boehm	风险估计：辨识、分析、排序；风险控制：计划、 处理 、监督
Charette	分析：辨识、估计、评价；管理：计划、控制、监督
CMU-SEI	基于实践的全面 风险管理 ，包含需求方因素

第 6 章 数据库设计基础知识

一、本章考试定位

本章在系统架构设计师考试中属于**基础高频 + 案例支撑型章节**。综合知识常考数据库概念、三级模式、**关系代数**、**完整性**约束、范式、**E-R** 转关系模式、**索引**与 **NoSQL**；案例题可能要求识别**实体**、**联系**、主外键、**规范化**缺陷、数据库性能优化方案；论文中可作为“数据架构设计、数据层选型、读写分离、缓存、**NoSQL**、**反规范化**”的理论支撑。教材第 6 章覆盖数据库基本概念、**关系数据库**理论、**数据库设计**步骤、应用程序与数据库交互、**NoSQL** 数据库等内容。

二、数据库基本概念

1. 数据、信息、DB、DBS、DBMS

数据是描述事物的符号记录，可以是文字、图形、图像、声音等；**信息**是具有特定含义的数据，反映现实世界事物的状态。考试中常考二者关系：**数据是信息的载体，信息是有语义的数据**。

数据库 DB：长期存储在计算机内、有组织、可共享的相关数据集合。

数据库系统 DBS：采用数据库技术，有组织、动态存储大量关联数据，支持多用户访问的计算机系统。

数据库管理系统 DBMS：数据库系统的核心软件，负责数据定义、数据操纵、运行管理、建立与维护。

易错点：

DB 侧重“数据集合”；**DBMS** 侧重“管理软件”；**DBS** 是包含 **DB**、**DBMS**、应用、用户、硬件、管理员等的完整系统。

2. 数据管理技术三阶段

阶段	核心特点	缺点/考点
人工管理	数据少，数据与程序一一对应，无专门管理软件	数据不保存，程序与数据强依赖，冗余大
文件系统	数据可长期保存，逻辑结构与物理结构开始分离	文件相互孤立，数据冗余、不一致、共享差
数据库系统	采用 数据模型 ，数据面向整个应用系统， DBMS 统一管理	重点优势 ：共享性高、独立性高、冗余低、 完整性/安全性 控制强

真题对应：常以“文件系统阶段缺点”“数据库系统区别于文件系统的主要特征”设问。答案关键词一般是**数据共享**、**数据独立性**、**统一控制**、**减少冗余**。

3. 数据模型三要素

数据模型是描述数据的一组概念和定义，三要素为：

要素	含义	记忆关键词
数据结构	描述对象类型及其 联系	静态特性
数据操作	对对象实例可执行的操作，如查询、插入、删除、修改	动态特性
数据约束	完整性 规则集合	正确、有效、相容

数据模型发展：**层次模型** → **网状模型** → **关系模型** → **第三代/NoSQL 等新型数据模型**。**层次模型**是树形结构；**网状模型**更灵活，可表示多个父节点；**关系模型**用表格结构描述**实体**及**联系**，是当前最常用模型之一。

三、DBMS 功能与三级模式

1. DBMS 的主要功能

DBMS 主要功能包括：

- 1. **数据定义 DDL**：定义**外模式**、模式、**内模式**，**完整性**、安全权限等。
- 2. **数据操纵 DML**：检索、插入、修改、删除。
- 3. **数据库运行管理**：**并发控制**、安全控制、**完整性**检查、事务管理、日志与恢复。
- 4. **数据组织、存储和管理**：管理用户数据、**数据字典**、存取路径。
- 5. **数据库建立与维护**：初始建立、数据转换、转储恢复、重组重构、性能监测。
- 6. **通信与转换**：与其他 **DBMS**、文件系统或网络软件交互。

易错点：

DDL 是“定义结构”，**DML** 是“操作数据”，**DCL** 通常用于权限控制；运行管理强调**并发**、**安全**、**完整性**、**恢复**。

2. 数据控制功能

控制功能	含义	易考关键词
安全性 Security	防止非法使用导致泄露、篡改、破坏	权限、口令、访问控制
完整性 Integrity	保证数据正确性和相容性	防止合法用户录入不符合语义的数据
并发控制	协调多事务并发执行	防止丢失修改、不可重复读、脏读等
故障恢复	将数据库恢复到正确/一致状态	日志、备份、冗余数据

3. 数据库三级模式结构

数据库系统通常采用**外模式—概念模式—内模式**三级模式。教材图 6-1 展示了用户通过外部视图访问数据，DBMS 在**视图层**、**逻辑层**和**物理层**之间完成映射与屏蔽。

层次	又称	描述内容	作用
外模式	用户模式/子模式	用户需要使用的部分数据	面向用户，保证 安全性
概念模式	模式	全部数据的逻辑结构和特征	数据库全局逻辑视图
内模式	存储模式	数据物理结构和存储方式	面向存储和访问路径

两级映像：
外模式/概念模式映像保证逻辑**数据独立性**；**概念模式/内模式映像**保证物理**数据独立性**。
易错提醒：
概念模式通常只有一个；**外模式**可以有多个；**内模式**描述物理存储。**视图层**不等同于 SQL 视图本身，但 SQL 视图是实现**外模式/用户视图**的重要手段。

四、关系数据库核心考点

1. 关系基本术语

术语	含义	高频易错
属性 Attribute	表中的列	描述 实体 特征
域 Domain	属性 取值范围	关系模型 通常要求原子值
度 Degree	关系中 属性 个数	不是 元组个数
元组 Tuple	表中的一行	对应一条记录
候选码	能唯一标识元组的 属性 或 属性组	可有多
主码/主键	从 候选码 中选定一个	唯一且非空
主 属性	包含在任何 候选码 中的 属性	非主 属性 不在任何 候选码 中
外码	本关系中 不是 码，但对应其他关系的码	外码 可为空，取决于业务规则
全码	关系模式所有 属性 共同构成 候选码	常见于三元 联系 场景

关系模式形式化表示：
 $R(U, D, dom, F)$
其中，R 为关系名，U 为**属性**名集合，D 为域，dom 为**属性**到域的映射，F 为**属性**间数据依赖。通常简写为：
 $R(A_1, A_2, \dots, A_n)$

2. 关系完整性约束

类型	规则	例子
实体完整性	主码唯一且非空	学号 不能 为空且 不能 重复
参照完整性	外码 要么为空，要么等于被参照关系某个 主码 值	员工表部门号 必须 存在于部门表
用户定义完整性	由具体业务语义决定	账户号范围、工资下限、成绩 0~100

易错提醒：
参照完整性不是要求**外码**一定非空，而是**外码值若非空，必须能在被参照关系中找到对应主码值**。

五、关系代数与公式熟练

关系操作的特点：**操作对象是集合，操作结果仍是集合**。**关系代数**中，**五种基本运算是**：并、差、笛卡尔积、**投影**、**选择**；交、**连接**、**除**等可由基本运算导出。

1. 传统集合运算

运算	记号	条件	结果
并	$R \cup S$	R、S 结构相同	属于 R 或 S 的元组
差	$(R - S)$	R、S 结构相同	属于 R 但不属于 S 的元组
交	$R \cap S$	R、S 结构相同	同时属于 R 和 S 的元组
笛卡尔积	$R \times S$	无需同结构	元组拼接，列数相加、行数相乘

笛卡尔积基数公式：

$$|R \times S| = |R| \times |S|$$

若多个域 $(D_1, D_2, \dots, D_n)$ 的基数分别为 $(m_1, m_2, \dots, m_n)$ ，则笛卡尔积基数为：

$$M = \prod_{i=1}^n m_i$$

2. 专门关系运算

运算	记号	方向	类比 SQL
选择	$(\sigma_F(R))$	水平方向，选行	WHERE
投影	$(\pi_A(R))$	垂直方向，选列	SELECT 字段
连接	$R \bowtie S$	横向拼接满足条件的元组	JOIN
自然连接	$R \ltimes S$	等值连接并去掉重复属性	NATURAL JOIN
除	$R \div S$	同时涉及行与列	“全部满足”类查询

选择：

$$\sigma_F(R) = \{ t \mid t \in R \wedge F(t) = \text{True} \}$$

投影：

$$\pi_A(R) = \{ t[A] \mid t \in R \}$$

θ 连接可理解为：

$$R \bowtie_{\theta} S = \sigma_{\theta}(R \times S)$$

考试口诀：

选择选行、投影选列、连接按条件拼表、自然连接去重列、除法找“对所有……都满足”的对象。

3. 外连接与聚集函数

外连接用于保留不匹配元组并用 null 填充：左外连接保留左表全部，右外连接保留右表全部，全外连接两边都保留。

聚集函数：sum、avg、count、min、max。若要去重后统计，使用 distinct，如 count-distinct。

易错提醒：

自然连接不是简单笛卡尔积，它要求同名属性相等，并去掉重复属性；如果没有公共属性，自然连接退化为笛卡尔积。

六、关系数据库设计理论：函数依赖与范式

1. 函数依赖

设关系模式 $(R(U))$ ， $(X, Y \subseteq U)$ 。若任意合法关系实例中，两个元组在 X 上取值相同，则在 Y 上取值也必相同，称 X 函数决定 Y，记为：

$$X \rightarrow Y$$

注意：

函数依赖是语义约束，不能只根据某一时刻的数据偶然现象判断。例如某时刻学生年龄都不同，不能推出“年龄 $\rightarrow$ 学号”。

2. 完全依赖、部分依赖、传递依赖

类型	含义	例子
完全函数依赖	X 能决定 Y，且 X 的任一真子集不能决定 Y	((学号, 课程号) $\rightarrow$ 成绩)
部分函数依赖	X 能决定 Y，但 X 的某个真子集也能决定 Y	((学号, 课程号) $\rightarrow$ 姓名)，实际学号即可决定姓名
传递依赖	$X \rightarrow Y, Y \rightarrow Z$ ，且 Y 不是 X, Z 间接依赖 X	学号 $\rightarrow$ 系号，系号 $\rightarrow$ 系名

3. 多值依赖

多值依赖记为：

$$X \twoheadrightarrow Y$$

含义是：给定 X 后，Y 的一组值只由 X 决定，与其他属性无关。4NF 主要用于消除非平凡且非函数依赖的多值依赖。

七、范式体系与规范化

范式层次：

$$1NF \geq 2NF \geq 3NF \geq BCNF \geq 4NF \geq 5NF$$

教材重点到 4NF。规范化的目的：减少冗余，避免插入异常、删除异常、修改异常。

范式	判定要点	消除的问题
----	------	-------

1NF	每个属性值不可再分，属性原子化	非原子属性
2NF	在 1NF 基础上，非主属性完全依赖于码	非主属性对码的部分依赖
3NF	在 2NF 基础上，消除非主属性对码的传递依赖	传递依赖导致的冗余
BCNF	每个非平凡函数依赖的决定因素都必须包含码	更严格地消除主属性相关异常
4NF	每个非平凡多值依赖的决定因素都必须包含码	非函数多值依赖

真题对应：
给出关系模式和函数依赖，要求判断最高范式、找候选码、指出部分依赖/传递依赖、分解到 3NF 或 BCNF。
易错提醒：
3NF 不等于 BCNF。BCNF 要求每个决定因素都是超码；只考虑函数依赖时，最高常考到 BCNF；考虑多值依赖时，才进入 4NF。

八、数据库设计步骤

数据库设计属于系统设计范畴，任务是在给定应用环境、硬件、操作系统和 DBMS 条件下，建立性能良好的数据库模式和应用系统。教材将数据库设计分为六个阶段。

阶段	主要任务	输出/工具
用户需求分析	分析功能、性能、限制、数据与处理要求	数据流图、数据字典、需求说明
概念结构设计	抽象现实世界，建立独立于 DBMS 的概念模型	E-R 图
逻辑结构设计	将 E-R 图转换为 DBMS 支持的数据模型	关系模式、完整性约束、用户视图
物理结构设计	确定数据分布、存储结构、访问方式	索引、聚簇、存储参数
数据库实施	建库、编程、数据加载、试运行	DDL 脚本、加载程序
运行与维护	性能监测、备份恢复、重组重构	DBA 运维方案

考试口诀：
需概逻物实运：需求分析 → 概念设计 → 逻辑设计 → 物理设计 → 实施 → 运行维护。

九、需求分析与概念结构设计

1. 需求分析三类要求

类型	关注内容
信息要求	系统要保存哪些数据，数据间完整性要求
处理要求	要实现哪些操作，频度、响应时间、处理方式、安全完整性要求
系统要求	安全性、使用方式、并发用户数、可扩充性

结构化分析 SA 常用数据流图 DFD + 数据字典 DD。需求分析是后续逻辑设计、物理设计和应用程序设计的基础。

2. E-R 方法

概念结构设计常用 E-R 方法，用实体、属性、联系描述现实世界。抽象方法包括：

方法	含义
分类 Classification	抽象出具有共同特征的类型
聚集 Aggregation	定义某类对象的属性
概括 Generalization	从已有类型定义新类型，形成超类/子类

概念结构设计步骤：选择局部应用 → 逐一设计分 E-R 图 → 合并为全局 E-R 图。

3. 分 E-R 图合并冲突

冲突类型	表现	处理
属性冲突	类型、取值范围、计量单位不一致	统一属性定义
命名冲突	同义异名或同名异义	统一命名
结构冲突	同一对象在不同图中被设计为实体或属性	统一抽象层次

合并优化：实体类型合并、冗余属性消除、冗余联系消除。

十、逻辑结构设计：E-R 图转关系模式

逻辑结构设计主要包括：确定数据模型、E-R 图转换、确定完整性约束、确定用户视图。

1. 实体转换

E-R 图中的每个实体通常转换为一个关系模式：

- 实体名 → 关系名；
- 实体属性 → 关系属性；
- 实体标识符 → 关系码。

2. 联系转换规则

联系类型	转换方法	主码处理
1:1	可并入任一方实体	加入另一方主码和联系属性
1:N	并入 N 方实体	在 N 方加入 1 方主码和联系属性
M:N	必须转换为独立关系模式	两端实体主码组合通常作为该关系主码

易错提醒：

多对多联系不能简单并入任一实体，必须单独成表。一对多联系通常把“一方主键”放到“多方表”中作为外键。

3. 逻辑设计后续工作

- 根据语义确定函数依赖。
- 判断关系模式范式，通常检查是否达到 3NF 或 BCNF。
- 不满足要求则分解。
- 根据性能要求进行修正，必要时反规范化。
- 确定检查约束、主码约束、参照完整性约束。
- 根据业务和用户类别建立视图，提高安全性和独立性。

十一、反规范化与物理设计

1. 反规范化

规范化会减少冗余，但可能导致查询时多表连接过多、性能下降。为提升读性能，可以有选择地反规范化。

常见反规范化手段：

手段	含义	风险
冗余列	保存可由其他表获得的数据	数据不一致
派生列	保存计算结果	更新维护复杂
表重组	合并或重构表	设计复杂
水平分割	按行拆表	查询路由复杂
垂直分割	按列拆表	需要关联恢复完整记录

易错提醒：

反规范化不是“从未规范化”，而是在规范化基础上为性能引入受控冗余。引入冗余后，需要应用程序同步、批处理同步或触发器同步等机制处理一致性问题。

2. 物理设计三项工作

工作	内容
确定数据分布	集中/分布式管理，按应用、频度、响应时间分布
确定存储结构	顺序、哈希、堆、B+树等
确定访问方式	索引、聚簇、访问路径

3. 索引设计

适合建索引：主码、外码、查询频繁字段、只读表、范围查询字段、排序字段、聚集函数字段。

不适合建索引：小表、频繁更新字段、取值很少字段、过长字段、大文本/多媒体字段、很少出现在查询条件中的字段。

易错提醒：

索引能提升检索性能，但会增加插入、删除、修改维护成本。不是索引越多越好。

4. 聚簇设计

适合建立聚簇索引的条件：

- 聚簇码值相对稳定。

- 2. 表主要用于查询。
- 3. 通过**聚簇**码访问或**连接**是主要应用。
- 4. 每个**聚簇**码对应元组数适中。

十二、数据库实施与运行维护

1. 数据库实施

实施阶段主要包括：

- 1. 用 **DDL** 建立数据库结构。
- 2. 描述数据库模式、子模式、空间、**完整性**、**安全性**和物理存储参数。
- 3. 数据整理、录入、校验与加载。
- 4. 应用程序和数据库联合调试。
- 5. 试运行和评价。

重点提醒：

数据加载前**必须**校验数据质量；测试**不是**证明系统“能运行”，而是尽可能发现问题。

2. 运行维护

维护内容	重点
性能监测与改善	I/O 量、CPU 时间、响应时间
备份与 故障恢复	制定备份方案，恢复到一致状态
数据库重组	不改变逻辑/物理结构， 清除 碎片和废弃空间
数据库重构	修改表结构、视图、约束、表分解与合并

易错提醒：

重组不改变数据库逻辑结构；重组会改变结构。视图可提高逻辑独立性和**安全性**，但修改视图时要防止引发数据泄露。

十三、应用程序与数据库交互

教材将常见接口分为库函数、嵌入式 **SQL**、通用数据接口标准和 **ORM**。

接口类型	代表	特点
库函数级接口	Oracle OCI	底层、灵活、强依赖特定数据库
嵌入式 SQL	C、COBOL、Java 中嵌入 SQL	需预编译器处理
通用接口	ODBC、JDBC、ADO.NET	屏蔽异构数据库差异
ORM	Hibernate、Mybatis、JPA	对象与关系表映射，提高开发效率

易错提醒：

ODBC 面向异构数据库统一访问；**JDBC** 是 Java 访问数据库的标准 API；**ORM 不是**数据库本身，而是对象与**关系数据库**之间的映射技术。

十四、NoSQL 数据库

NoSQL 可理解为 Non-Relational 或 Not Only **SQL**，泛指非关系型数据库。它区别于**关系数据库**，不保证传统关系数据的 **ACID** 特性，更适合大规模、高并发、灵活**数据模型**等场景。

1. NoSQL 四类数据库

类型	数据结构/特点	典型场景
列式存储数据库	按列组织，列族管理	海量数据、分布式存储、分析型场景
键值数据库	Key-Value 哈希访问	缓存、会话、简单高速读写
文档数据库	JSON 等半结构化文档	内容管理、网页数据、灵活字段
图数据库	节点和边表示关系	社交网络、知识图谱、路径关系

2. NoSQL 共同特点

- 1. 易扩展：弱化关系结构，便于横向扩展。
- 2. 大数据量、高性能：结构简单，读写性能高。
- 3. **数据模型**灵活：无需预先固定字段。
- 4. 高可用：可通过复制、分布式架构实现。

3. NoSQL 四层体系框架

层次	内容
数据持久层	内存、硬盘、内存+硬盘、可插拔存储
数据分布层	CAP 支持、多数据中心、动态部署
数据逻辑模型层	表达数据逻辑结构
接口层	REST、Thrift、Map/Reduce、Get/Put、特定语言 API

适用场景：**数据模型**简单、需要更强灵活性、高性能、无需高度一致性、给定 key 易映射复杂值的环境。

易错提醒：

NoSQL 不是“完全不用 SQL”，也不是必然优于关系数据库。强事务、强一致、复杂关联查询、严格完整性约束场景仍优先考虑关系数据库。

第 7 章 系统架构设计基础知识

一、本章考试定位

第 7 章是系统架构设计师的核心基础章，属于 综合知识高频章节、案例分析常用支撑章节、论文写作基础章节。考试通常不会只考“背概念”，而是把**架构风格、质量属性、ABSD 流程、复用、产品线、DSSA**等放入具体系统场景中考查。

本章备考主线可以压缩为一句话：

软件架构 = 构件 + 外部可见属性 + 构件关系；架构设计 = 用合适的视图、风格、复用资产和演化机制，满足功能需求与质量属性。

二、7.1 软件架构概念

1. 软件架构的定义

软件架构不是可运行程序，而是对系统结构的抽象表达。它关注的是：

核心要素	备考理解
构件	模块、类、服务、数据库、中间件、子系统等
外部可见属性	接口、性能、安全性、可靠性、可维护性等对外表现
构件关系	调用、依赖、通信、部署、数据流、控制流、事件关系等

教材中的核心口径是：**软件体系结构**包括系统的一个或多个结构，结构中包括软件**构件**、**构件外部可见属性**及其**相互关系**。**体系结构**用于分析设计有效性、比较备选方案、降低构造风险。

记忆公式

$$SA = Component + Property + Relationship$$

即：

软件架构 = 构件 + 外部可见属性 + 相互关系

不要把**软件架构**简单理解为“模块划分图”或“部署图”。模块划分只是架构的一种**视图**，架构还必须体现**质量属性**、接口约束、交互关系和演化约束。

2. 软件架构设计与生命周期

软件架构贯穿软件生命周期，考试常考“某阶段架构工作的重点”。

生命周期阶段	架构关注点	高频考法
需求分析	从需求模型转换到架构模型，保持 可追踪性	问题空间 vs 解空间 、Use Case 到类图/架构模型
设计阶段	构件 、 连接器 、ADL、多 视图 表示	ADL 、 4+1 视图 、关注点分离

实现阶段	从高层架构到代码实现、配置管理、基于架构测试	模型映射、构件组装、中间件支持
构件组装	连接子实现、构件复用、失配检测	构件失配、连接子失配、全局架构假设冲突
部署阶段	软硬件映射、拓扑、CPU/内存资源、性能与可靠性分析	部署视图、部署方案质量属性评估
后开发阶段	维护、演化、动态架构、架构恢复与重建	动态软件架构、遗留系统架构重建

动态软件架构

动态架构强调系统在运行时可以发生结构变化，例如运行时创建**构件**、切换**连接子**、在线升级、运行时重配置。考试常把它和“静态架构”对比。

架构恢复与重建

架构重建是从已经实现的系统中提取架构**视图**，常见方法包括：

- 1、 手工重建
- 2、 工具支持的手工重建
- 3、 查询语言辅助自动聚集
- 4、 数据挖掘等其他技术

3. 软件架构的重要性

作用	应用理解
满足系统品质	架构决定 性能、安全性、可靠性、可维护性等质量属性
统一涉众目标	让客户、开发、测试、运维对系统结构达成共识
支持项目计划	支撑成本估算、进度安排、人员分工、风险管理
指导系统开发	给设计人员和实现人员提供结构框架、接口规范和约束
管理复杂性	用抽象、分解、分层、 构件化 降低复杂度
支持复用	架构层复用比代码复用粒度更大、收益更高
降低维护费用	架构文档和可扩展机制有利于维护、适应和扩充
支持影响分析	需求变更或 构件 变更时，可分析影响范围

客观题常问“架构设计的作用不包括哪项”；案例题常要求说明“为什么要先做架构评审或架构设计”；论文题可把这些作用作为“架构设计价值”的论证段。

三、7.2 基于架构的软件开发方法 ABSD

1. ABSD 概念

ABSD，即 Architecture-Based Software Design，基于**体系结构**的软件设计方法。它由商业需求、质量需求和**功能需求**共同驱动。其重要特点是：需求尚未完全确定时，也可以并行开展架构设计，尤其适合产品线系统、长期运行系统和需求难以一次性穷尽的大型系统。

ABSD 三大基础

基础	含义
功能分解	使用模块内聚、耦合等方法分解功能
架构风格 选择	通过合适 架构风格 满足质量和商业需求

软件模板使用	利用已有系统结构形成可复用模板
--------	-----------------

记忆公式

ABSD = 功能分解 + 架构风格 + 软件模板

2. ABSD 关键概念

概念	备考要点
设计元素	系统 → 概念子系统 → 概念 构件 → 实际 构件 /类
视角与视图	从不同角度描述架构，如逻辑 视图 、进程 视图 、实现 视图 、配置 视图
用例	捕获 功能需求 ，描述用户获得结果值的功能点
质量场景	捕获 非功能需求 ，如变更、 性能 、 可靠性 、交互性场景

用例主要对应 **功能需求**；**质量场景**主要对应 **非功能需求/质量属性**。不要把“**质量场景**”写成普通业务流程。

3. ABSD 开发模型

ABSD 模型包括六个子过程：
体系结构需求 → 体系结构设计 → 体系结构文档化 → 体系结构复审 → 体系结构实现 → 体系结构演化
需、设、文、审、实、演

4. 体系结构需求

体系结构需求主要来自三类目标：

- 1、 系统质量目标
- 2、 系统商业目标
- 3、 系统开发人员的商业目标

需求过程包括：

步骤	内容
需求获取	获取 功能需求 和 质量属性
标识构件	生成初始逻辑结构
需求评审	检查需求真实性、 类分组 合理性、 构件 合并合理性

标识构件又可细分为：

生成类图 → 对类进行分组 → 把类打包成构件

5. 体系结构设计

体系结构设计是迭代过程，主要步骤为：

步骤	内容
提出架构模型	先选择合适的 架构风格 ，形成目标结构
映射 构件	将已 标识构件 映射到架构模型
分析 构件 交互	明确 构件 关系、接口、调用、数据交换
产生架构	在中间结构基础上精化
设计评审	邀请独立人员评审架构合理性

当案例题要求“说明架构设计过程”时，可按以下顺序答：

选风格 → 建模型 → 映射构件 → 分析交互 → 精化架构 → 独立评审

6. 体系结构文档化、复审、实现与演化

子过程	重点
文档化	输出架构规格说明、质量设计说明；文档必须最新、完整、面向使用者
复审	识别潜在风险，检查架构是否满足功能与质量需求
实现	根据架构说明书实现 构件 、查找/开发 构件 、组装和测试
演化	根据需求变化修改架构，保证新需求与原架构集成

体系结构演化六步：

- 1、需求变化归类
- 2、制订**体系结构演化**计划
- 3、修改、增加或删除**构件**
- 4、更新**构件**相互作用
- 5、构件**组装**与测试
- 6、**技术评审**

“实现”不是随意编码，而是要符合架构说明中的结构性设计决策；“演化”不是简单改代码，而是需求变化驱动下的架构级变更。

四、7.3 软件架构风格

软件架构风格是某一类系统组织方式的惯用模式，它定义了**构件**、**连接件**及二者组合的约束。考试最常考“风格识别”和“场景匹配”。

1. 数据流体系结构风格

风格	构件	连接件	适用场景	易错点
批处理	独立应用程序	数据媒介	数据完整后分阶段处理，如传统报表、批量结算	每一步必须等前一步结束
管道-过滤器	过滤器	管道	连续数据转换，如编译器、流式处理、日志处理	一个过滤器输出是下一个输入

批处理：整批数据，阶段串行。
管道过滤器：**数据流动**，边处理边传递。

2. 调用/返回体系结构风格

风格	核心思想	高频考点
主程序/子程序	单线程控制，主程序调用子程序	层次调用、过程调用
面向对象	数据和操作封装为对象	抽象数据类型、封装、对象交互
层次型	上层使用下层服务，下层为上层提供接口	相邻层交互、 虚拟机 思想、可替换性
C/S	客户端请求服务端资源	两层 C/S、三层 C/S、胖/瘦客户机

C/S 两层与三层对比

类型	组成	特点
两层 C/S	客户应用程序 + 数据库服务器 + 网络	客户端承担较多交互和部分业务，常称 胖客户机
三层 C/S	表示层 + 功能层 + 数据层	增加应用服务器，业务逻辑集中在功能层，客户机更“瘦”

3. 以数据为中心的体系结构风格

风格	重点	适用场景
仓库风格	中央数据结构 + 独立 构件 ， 构件 围绕仓库读写数据	数据集中共享、统一维护
黑板风格	黑板 + 知识源 + 控制机制，多知识源协同求解	语音识别、模式识别、复杂非结构化问题

黑板风格适合复杂、非结构化、需要多个知识源协作的问题；**仓库风格**更强调共享数据中心。

4. 虚拟机体系结构风格

风格	构成	特点
解释器风格	解释引擎、代码存储区、状态数据结构、执行进度数据结构	灵活但效率较低
规则系统风格	规则集、规则解释器、规则/数据选择器、工作内存	适合基于规则推理的系统

解释器风格的优点是灵活、可扩展；缺点常考为 **执行效率较低**。

5. 独立构件体系结构风格

风格	构件	连接件	特点
进程通信	独立进程	消息传递	点到点、同步/异步、远程调用
事件系统	模块或事件处理过程	事件触发/广播	触发者不知道哪些 构件 会响应

事件系统又称**隐式调用**。它的关键特征是：**构件**不是直接调用另一个**构件**，而是**发布**事件，由系统自动调用**注册**在该事件上的处理过程。

看到“发布、订阅、注册、监听、通知、事件处理器”，优先联想 **事件系统风格**。看到“过滤、转换、流、管道”，优先联想 **管道-过滤器风格**。看到“知识源、黑板、控制模块、分级解空间”，优先联想 **黑板风格**。

五、7.4 软件架构复用

1. 软件产品线与架构复用

软件产品线是一组软件密集型系统，它们共享公共、可管理的特性集，围绕**核心资产库**进行管理、复用和集成。**核心资产**包括架构及可裁剪元素，也包括设计文档、用户手册、项目管理历史、测试计划和测试**用例**等。

2. 软件架构复用类型

类型	含义
机会复用	开发过程中发现资产可复用，就直接复用

系统复用	开发前就有计划地确定哪些资产需要复用
------	--------------------

机会复用偏“临时发现”；系统复用偏“事前规划”。产品线更接近系统化、策略性复用。

3. 架构复用的原因

架构复用的直接收益：

- 1、 减少开发工作量
- 2、 缩短开发时间
- 3、 降低开发成本
- 4、 提高生产率
- 5、 提高产品质量
- 6、 改善互操作性
- 7、 简化维护

4. 可复用资产对象

可复用资产	应用理解
需求	多个系统存在相同或相似需求
架构设计	经验证的架构可迁移到新产品
元素	构件、模块、接口、服务等
建模与分析	性能分析、容错方案、负载均衡方案
测试	测试用例、测试数据、测试工具、测试计划
项目规划	成本、进度、团队安排经验
过程、方法和工具	产品线级流程、规范、标准、工具环境
人员	产品线培养的人员可服务系列产品
样本系统	已投产系统可作为原型和参考
缺陷消除	产品族中的共性缺陷可一次解决、多次受益

教材强调，复用粒度正由函数、库、类、接口、包等小粒度复用，向架构、产品线、测试与过程等大粒度复用发展。

5. 软件架构复用基本过程

软件架构复用包括三个阶段：

- 1、 构造/获取可复用资产：资产应可靠、易理解、可修改、可广泛使用。
- 2、 管理可复用资产：核心设施是构件库，关键问题是构件分类与构件检索。
- 3、 使用可复用资产：根据需求检索资产，选择、定制、扩展、配置、组装和集成。

记忆公式

复用过程 = 获取资产 + 管理资产 + 使用资产

六、7.5 特定领域软件体系结构 DSSA

1. DSSA 定义

DSSA，即 Domain Specific Software Architecture，特定领域软件体系结构，是在一个特定应用领域中，为一组应用提供组织结构参考的标准软件体系结构。其目标是在相关应用中共享架构，支持同一领域多个应用系统的生成。

2. DSSA 必备特征

- 1、 有严格定义的问题域和解域
- 2、 具有普遍性，可用于领域内具体应用开发
- 3、 对领域**构件**组织模型有恰当抽象
- 4、 具备领域中固定、典型、可复用的元素

3. 垂直域与水平域

类型	含义	例子理解
垂直域	面向一个特定系统族	银行业务系统族、航空订票系统族
水平域	跨多个系统族的共有功能区域	权限管理、日志审计、工作流、报表

垂直域强调“某一类完整系统族”；**水平域**强调“多个系统都共有的功能区域”。

4. DSSA 基本活动

活动	目标	产出
领域分析	获得领域模型	识别边界、信息源、共性需求、变化范围
领域设计	获得 DSSA	建立能适应多个系统需求的高层设计
领域实现	开发和组织可重用信息	构件 、模型、框架、基础设施等

DSSA 活动不是一次性线性过程，而是反复、逐步求精的过程。

5. DSSA 参与人员

角色	主要任务
领域专家	提供领域知识、需求规约、样本系统选择、复审领域模型
领域分析人员	控制领域分析、获取知识、组织领域模型、验证一致性
领域设计人员	根据领域模型开发 DSSA，验证 DSSA 准确性并建立模型联系
领域实现人员	开发或提取可重用构件，验证构件，建立 DSSA 与构件联系

6. DSSA 建立过程

DSSA 建立过程分为五个阶段：

- 1、 定义领域范围
- 2、 定义领域特定元素
- 3、 定义领域特定的设计和实现约束
- 4、 定义领域模型和**体系结构**
- 5、 产生、搜集可重用产品单元

定范围 → 定元素 → 定约束 → 定模型架构 → 收集产品单元

第 8 章 系统质量属性与架构评估

一、本章备考定位

本章核心不是背“**质量属性**名词”，而是掌握 **质量属性**如何被场景化表达、架构如何被评估、不同**质量属性**之间如何权衡。考试中常见三类考法：

- 1. 综合知识选择题：考**质量属性**分类、场景六要素、**敏感点/权衡点**、**SAAM/ATAM/CBAM** 区分。

2. 案例分析题：给出电商、政务、金融、工业互联网、云平台等场景，要求识别**质量属性**、构造**质量属性场景**、判断**敏感点**和**权衡点**。
3. 论文题支撑：架构设计论文中，必须会写“为什么选择该架构策略、如何满足**性能/可用性/安全性/可修改性**、如何评估权衡”。
- 一句话总纲：架构关注**质量属性**，**质量属性**靠场景表达，**架构评估**靠场景、度量和权衡。

二、软件系统质量属性

1. 质量属性的基本概念

软件系统属性分为 **功能属性** 和 **质量属性**。**功能属性**回答“系统做什么”，**质量属性**回答“系统做得怎么样”。**软件架构**设计重点关注**质量属性**，因为架构层面的结构、构件、连接件、部署方式、数据流和控制流，会直接影响系统的**性能、可用性、安全性、可修改性**等。

教材按 GB/T 16260.1 的口径，将软件质量从管理角度划分为六大特性：

质量特性	关键含义	常见子特性
功能性	能否满足规定功能	适合性、准确性、 互操作性 、依从性、 安全性
可靠性	能否持续正确运行	容错性 、易恢复性、成熟性
易用性	用户是否容易理解、学习和操作	易学性、易理解性、易操作性
效率	时间和资源使用是否合理	时间特性、资源特性
维护性	修改、分析、 测试 是否方便	可测试性 、 可修改性 、稳定性、易分析性
可移植性	能否迁移到其他 环境	适应性、易安装性、一致性、可替换性

考试提醒：**质量属性**是可测量或可**测试**的属性，不能只写“系统要好用、要快、要安全”，必须转化为可度量表达，例如“95%请求在 2 秒内**响应**”“故障后 5 分钟内恢复”。

2. 开发期质量属性与运行期质量属性

教材将**质量属性**按生命周期分为 **开发期质量属性** 和 **运行期质量属性**。这组分类是选择题高频考点。

类型	质量属性	备考理解
开发期质量属性	易理解性	设计是否容易被开发人员理解
开发期质量属性	可扩展性	适应新需求、增加新功能的能力，也称灵活性
开发期质量属性	可重用性	系统或构件能否在其他场景复用
开发期质量属性	可测试性	是否容易通过 测试 证明满足需求
开发期质量属性	可维护性	缺陷修复、功能增加、质量提升时修改是否容易
开发期质量属性	可移植性	能否从一个运行 环境 迁移到另一个运行 环境
运行期质量属性	性能	响应时间 、吞吐量、容量、速度
运行期质量属性	安全性	合法用户可用，非法访问被阻止
运行期质量属性	可伸缩性	用户量、数据量增加时仍维持服务质量
运行期质量属性	互操作性	与其他系统交换数据、调用服务的难易程度
运行期质量属性	可靠性	一定时间内持续无故障运行
运行期质量属性	可用性	正常工作时间占总时间比例
运行期质量属性	鲁棒性	非正常输入、故障或异常 环境 下仍能合理运行

易错点：**可扩展性、可维护性、可移植性**常被混淆。**可扩展性**偏“加新功能”，**可维护性**偏“改缺陷/改系统”，**可移植性**偏“换平台/换**环境**”。

三、面向架构评估的质量属性

1. 性能

性能是系统对事件的**响应**能力，常用 **响应时间、吞吐量、容量、资源利用率** 衡量。教材强调**性能**可用单位时间处理事务数或完成事务所需时间来定量表达。

常用表达：

- 响应时间**：一次请求从提交到返回的时间。
- 吞吐量**：单位时间内处理的事务数。
- 容量**：系统可承载的最大用户数、连接数或数据量。
- 抖动**：响应时间波动程度，实时系统尤其关注。

公式熟练：

$$\text{吞吐量} = \text{完成事务数} / \text{时间}$$

$$\text{平均响应时间} = \text{所有请求响应时间之和} / \text{请求总数}$$

考试应用：看到“高并发、秒杀、峰值请求、接口响应慢、批处理窗口、实时控制”，优先想到 **性能**。

2. 可靠性

可靠性是系统在规定条件和时间内完成规定功能的能力，教材常用 **MTTF** 和 **MTBF** 衡量。**可靠性**包含 **容错** 和 **健壮性** 两个方面。

概念	含义	考试判断
MTTF	Mean Time To Failure，平均失效等待时间	不可修复系统常用

概念	含义	考试判断
MTBF	Mean Time Between Failure, 平均失效间隔时间	可修复系统常用
MTTR	Mean Time To Repair, 平均修复时间	与可用性计算有关
容错	错误发生时系统仍能保持正确行为, 并进行内部修复	如重试、冗余、故障切换
健壮性	错误输入或异常使用时系统按预定义方式处理	如输入校验、异常终止、降级

公式熟练：

$MTBF = MTTF + MTTR$

当修复时间很短且失效率近似为常数时，教材指出 MTTF 与 MTBF 近似相等。

易错点：容错 ≠ 健壮性。容错强调“错误发生后还能继续正确服务或恢复”，健壮性强调“不被错误输入击垮，至少能按预定义方式处理”。

3. 可用性

可用性是系统能够正常运行的时间比例，常与故障恢复速度有关。看到“系统全年可用率、故障恢复时间、停机窗口、降级运行、容灾切换”，优先判断为可用性。

公式熟练：

$可用性 = MTTF / (MTTF + MTTR)$

或：

$可用性 = 正常运行时间 / (正常运行时间 + 故障停机时间)$

常见数量级记忆：

可用性	年停机时间大致含义
99%	年停机约 3.65 天
99.9%	年停机约 8.76 小时
99.99%	年停机约 52.56 分钟
99.999%	年停机约 5.26 分钟

考试提醒：可用性更关注“服务是否可访问、多久恢复”，可靠性更关注“是否少出故障、能否持续正确运行”。

4. 安全性

安全性是系统在向合法用户提供服务的同时，阻止非授权用户使用或拒绝服务的能力。教材将安全性进一步分为机密性、完整性、不可否认性、可控性。

子属性	含义	常见措施
机密性	信息不泄露给未授权对象	加密、访问控制、脱敏
完整性	信息完整、准确、防篡改	摘要、签名、校验
不可否认性	发送方/接收方不能否认行为	数字签名、审计日志
可控性	对信息传播和内容具有控制能力	权限管理、数据分级、审计追踪

考试应用：看到“越权访问、数据泄露、篡改、抵赖、审计、拒绝服务、认证授权”，优先判断为 安全性。

5. 可修改性

可修改性是系统以较高性价比快速变更的能力，通常通过某类具体变更的代价衡量。教材把可修改性细分为可维护性、可扩展性、结构重组、可移植性。

方面	核心含义	架构设计关注
可维护性	错误发生后修复系统	局部修改、影响面小
可扩展性	增加新特性或替换构件	松耦合、接口稳定
结构重组	重新组织构件及关系	构件关系清晰、可配置
可移植性	适应不同平台/环境	硬件软件无关、平台隔离

考试提醒：可修改性是 ATAM 和 SAAM 的重点质量属性，案例题常要求判断某个架构是否易于增加新功能、替换组件、迁移平台。

6. 功能性、可变性与互操作性

属性	考点理解	常见题眼
功能性	系统完成期望工作的能力，需要多个构件协作	“是否完成业务功能”“构件是否正确协同”
可变性	架构经扩充或变更形成新架构的能力	“产品线”“系列产品”“框架扩展”
互操作性	与其他系统交换数据、调用服务的能力	“异构系统”“跨语言调用”“外部接口”“数据交换”

易错点：可变性比可修改性更偏架构族/产品线；互操作性不是“内部模块调用顺畅”，而是强调与外部系统或异构环境交互。

四、质量属性场景

1. 质量属性场景六要素

质量属性场景是把抽象质量需求转化为可测试需求的工具。教材将质量属性场景划分为六个部分：刺激源、刺激、环境、制品、响应、响应度量。

六要素	含义	例子
刺激源 Source	谁发出刺激	用户、外部系统、攻击者、管理员
刺激 Stimulus	发生了什么	请求到达、故障、攻击、需求变更
环境 Environment	在什么状态下发生	正常、过载、降级、运行时、设计时
制品 Artifact	作用到哪个对象	系统、服务、数据库、通信信道、模块
响应 Response	系统采取什么动作	处理请求、记录日志、拒绝访问、恢复服务
响应度量 Measurement	如何判断达标	响应时间、恢复时间、吞吐量、错误率

速记口诀：源激环制响量。

案例写法模板：

在【环境】下，当【刺激源】对【制品】产生【刺激】时，系统应【响应】，并满足【响应度量】。

示例：

在正常运行且并发 5000 用户访问时，当用户提交订单请求到订单服务，系统应完成库存校验、订单生成和支付跳转，95% 请求响应时间不超过 2 秒。

2. 六类典型质量属性场景

场景类型	关注点	刺激示例	响应度量
可用性场景	故障发生、检测、恢复、降级	崩溃、错误、通信中断	可用时间、故障修复时间、降级运行时间
可修改性场景	修改成本和影响范围	增加/删除/修改功能或容量	影响元素数量、修改成本、对其他功能影响
性能场景	响应速度和处理能力	定期/随机/偶然事件到达	等待时间、期限、吞吐量、抖动、丢失率
可测试性场景	缺陷发现和测试准备难度	架构/设计/代码/系统交付	覆盖率、测试时间、故障发现概率
易用性场景	学习、操作、纠错、满意度	用户学习、操作、出错、定制	任务时间、错误数、满意度、成功操作比例
安全性场景	授权、访问、攻击、数据保护	显示/修改/删除数据，访问服务，降低可用性	攻破概率、攻击检测率、恢复范围、服务保持比例

易错提醒：质量属性场景不是“描述功能流程”，而是 围绕一个质量属性写可测量需求。比如“用户下单”是功能场景；“95%下单请求 2 秒内完成”才是性能场景。

五、系统架构评估

1. 架构评估的含义

系统架构评估是在架构分析和评估基础上，对架构策略选取进行决策。它可以针对系统的一致性、正确性、质量属性和规划结果，提供描述性、预测性和指令性分析结果。教材将系统架构评估方法分为三类：基于调查问卷或检查表、基于场景、基于度量。

方法类别	核心做法	优点	缺点/限制
问卷/检查表	设计检查项，借助专家经验评估	简单、易实施	主观性强
基于场景	分析架构对具体场景的支持程度	适合质量属性分析，考试最常考	场景选择影响结论
基于度量	建立质量属性与度量映射，从架构文档获取数据	定量化较强	度量定义和数据获取较难

考试重点：SAAM 和 ATAM 都属于基于场景的评估方法；CBAM 在 ATAM 基础上加入成本收益分析。

2. 敏感点、权衡点、风险承担者与场景

概念	定义	考试判断
敏感点	一个或多个构件及关系的特性，对某个质量属性影响明显	“缓存大小影响性能”“模块耦合影响可修改性”
权衡点	同时影响多个质量属性的敏感点	“加密级别提高安全性但降低性能”
风险承担者/利益相关者	对架构有利益诉求并影响架构决策的人	用户、架构师、开发、测试、运维、安全专家、项目经理
场景	从利益相关者角度描述与系统交互	在架构评估中常用刺激、环境、响应描述

经典例子：提高加密级别可能提升安全性，但增加处理时间、降低性能，因此“加密级别”是 权衡点。

易错提醒：敏感点不一定是权衡点，权衡点一定是多个质量属性的敏感点。

六、SAAM、ATAM、CBAM 三大评估方法

1. SAAM 方法

SAAM 是较早形成文档并广泛使用的软件架构分析方法，最初主要用于分析 可修改性，后来扩展到可移植性、可扩充性等质量属性。SAAM 的核心是 场景技术。

SAAM 五个步骤：

1. 场景开发。
2. 架构描述。
3. 单个场景评估。
4. 场景交互。
5. 总体评估。

备考抓手：

维度	SAAM 关键词
主要技术	场景技术
重点 质量属性	可修改性
输入	问题描述、需求声明、架构描述
适用	比较不同架构，发现潜在问题点
局限	不重点考虑已有知识库可重用性

选择题常见设问：“某方法把**质量属性**具体化为场景，主要分析**可修改性**，可用于比较不同架构”，答案通常是 **SAAM**。

2. ATAM 方法

ATAM 是在 **SAAM** 基础上发展起来的架构权衡分析方法，关注多个相互影响的**质量属性**，尤其是 **性能、安全性、可修改性、可用性**。它用于系统开发前，对架构方案进行评价和折中。

ATAM 四个主要活动领域：

1. 场景和需求收集。
2. 架构视图和场景实现。
3. 属性模型构造和分析。
4. 折中。

ATAM 三类场景：

场景类型	含义	例子
用例场景	系统典型使用	用户登录、提交订单
增长场景	系统未来修改或扩展	增加支付方式、扩展新业务
探测场景	极端压力或过载修改	峰值流量 10 倍增长、节点大面积故障

ATAM 效用树：

效用树结构 = 根节点“效用” → 质量属性 → 属性分类 → 质量属性场景

优先级通常用：

(重要度，难易度) = (H/M/L, H/M/L)

例如：

(H, L)

表示该场景 重要度高、实现难度低，通常应优先关注。教材强调 **ATAM** 会修剪**效用树**，保留重要场景，并按重要性和实现难易度排序。

考试重点：**ATAM** 不只是“评价架构好不好”，更重要的是找出 风险、非风险、**敏感点、权衡点**。

3. CBAM 方法

CBAM 是**成本效益分析法**，建立在 **ATAM** 之上，用于从经济角度对架构设计决策的**成本**和**收益**建模。它使用 **ATAM** 的评估结果，帮助干系人根据**投资回报率**选择架构策略。

CBAM 八个步骤速记：

1. 整理场景，选取优先级最高的约 1/3。
2. 对场景求精，得到最坏、当前、期望、最好**响应**级别。
3. 确定场景优先级。
4. 分配**效用**。
5. 建立“策略—场景—**响应**级别”关系。
6. 用内插法确定期望**响应**级别**效用**。
7. 计算各架构策略总**收益**。
8. 根据**成本**约束下的 **ROI** 选择架构策略。

公式熟练：

ROI = 收益 - 成本 / 成本

或在架构策略排序题中简化解为：

ROI ≈ 架构策略收益 / 架构策略成本

考试判断：看到 成本、收益、效用、投资回报率、按 **ROI** 排序选择架构策略，优先选 **CBAM**。

4. 三大方法对比速记表

方法	全称	核心关注	关键词	高频考法
SAAM	Scenarios-based Architecture Analysis Method	场景化评估，尤其 可修改性	场景、 可修改性 、比较架构	问“最早、场景、 可修改性 ”

方法	全称	核心关注	关键词	高频考法
ATAM	Architecture Tradeoff Analysis Method	多质量属性权衡	效用树、敏感点、权衡点、风险	问“性能/安全/可修改/可用的折中”
CBAM	Cost Benefit Analysis Method	成本收益决策	效用、收益、成本、ROI	问“投资回报率选择架构策略”

速记：SAAM 看场景，ATAM 看权衡，CBAM 看收益。

七、其他架构评估方法

教材还介绍了多种其他方法，考试一般不深考推导，主要考“方法一用途一关键词”对应关系。

方法	核心用途	备考关键词
SAEM	从外部质量属性和内部质量属性建立架构评估框架	外部属性、内部属性、度量准则
SAABNet	使用贝叶斯信念网络表达和使用定性知识	BBN、不确定知识、质量属性预测
SACMM	度量架构修改，比较两个图结构架构差异	图内核、架构距离、重命名稳定性
SASAM	比较预期架构与实际架构	静态评估、映射、SAVE、架构一致性
ALRRA	评估软件架构可靠性风险	动态复杂度、动态耦合度、FMEA、CDG
AHP	多准则决策，计算权重并排序	层次分析法、两两比较、权重排序
COSMIC+UML	评估架构可维护性	复杂度、耦合度、内聚度、UML 组件图

SACMM 公式核心：

$$d(A_x,A_y)=\sqrt{(K(A_x,A_x)-2K(A_x,A_y)+K(A_y,A_y))}$$

其中 K 表示两个架构之间的相似度内核。考试通常不要求复杂推导，但要知道 SACMM 关注 架构修改距离、图结构比较、架构演化度量。

AHP 五步速记：

1. 确定总目标。
2. 建立递阶层次结构。
3. 构造判断矩阵并求相对权重。
4. 计算总排序。
5. 根据排序辅助决策。

八、ATAM 方法架构评估实践

教材第 8.3 节用 Event 框架中的 Hoover 架构和 Banking 架构说明 ATAM 实践流程。该部分适合案例题和论文题使用，不建议死背类图细节，重点掌握 ATAM 评估步骤、效用树、风险/非风险、敏感点/权衡点。

1. ATAM 四阶段

阶段	名称	主要任务
阶段 1	演示 Presentation	介绍 ATAM、业务驱动因素、待评估架构
阶段 2	调查和分析	确定架构方法、生成效用树、分析架构方法
阶段 3	测试	头脑风暴场景、投票排序、再次分析架构方法
阶段 4	报告 ATAM	输出评估发现

2. ATAM 实践步骤

步骤	内容	考点
第 1 步	介绍 ATAM	说明评估过程、技术、预期结果
第 2 步	介绍业务驱动因素	明确业务目标、干系人、约束
第 3 步	介绍待评估架构	描述架构、质量要求、技术约束
第 4 步	确定架构方法	说明架构如何满足关键需求
第 5 步	生成质量属性效用树	明确重要质量属性和优先级
第 6 步	分析架构方法	找风险、非风险、敏感点、权衡点
第 7 步	头脑风暴和优先场景	利益相关者提出并投票选择场景
第 8 步	再次分析架构方法	根据高优先级场景复核架构
最终	报告 ATAM	汇总效用树、场景、问题、风险和架构方法

教材最后报告通常包括：效用树、生成的场景、分析问题、风险和非风险、确定的架构方法。

3. Hoover 架构与 Banking 架构的考试化结论

教材案例中，Hoover 架构与 Banking 架构都围绕事件框架展开，但质量属性表现不同。备考时抓下面结论即可：

对比点	Hoover 架构	Banking 架构
应用程序信息嵌入程度	应用与框架分离较好	应用信息嵌入多个组件
可修改性	较强，修改影响面小	较弱，修改牵涉组件多
可重用性	较强，组件较独立	较弱，组件依赖较高
概念完整性	控制机制较少，行为一致	控制机制较多，一致性不足

对比点	Hoover 架构	Banking 架构
安全性	数据封装相对较好	数据分散，保密性较差
性能	处理涉及组件较少，性能较好	处理涉及组件较多，性能较弱

案例结论：Hoover 架构更符合 可变性、可修改性、功能性、概念完整性、安全性、性能 等质量属性要求；Banking 架构存在应用信息嵌入过多、组件耦合较高、控制机制偏多等风险。

九、真题对应与应用模板

1. 选择题高频点

题干关键词	正确方向
场景、可修改性、比较不同架构	SAAM
效用树、敏感点、权衡点、风险	ATAM
成本、收益、效用、ROI	CBAM
架构修改距离、图内核	SACMM
贝叶斯信念网络、不确定知识	SAABNet
预期架构与实际架构映射比较	SASAM
可靠性风险、FMEA、动态复杂度	ALRRA
多准则决策、两两比较、权重	AHP
刺激源、刺激、环境、制品、响应、响应度量	质量属性场景
同时影响安全性和性能	权衡点

2. 案例分析答题模板

问题：请识别系统中的质量属性需求，并构造质量属性场景。

答题模板：

- 质量属性：性能/可用性/安全性/可修改性/可靠性。
- 刺激源：用户、外部系统、管理员、攻击者。
- 刺激：请求到达、故障发生、非法访问、功能变更。
- 环境：正常运行、峰值负载、故障状态、运行时/设计时。
- 制品：系统、服务、数据库、接口、通信链路。
- 响应：处理、拒绝、记录、恢复、降级、告警。
- 响应度量：响应时间、吞吐量、恢复时间、错误率、修改成本。

示例：

性能场景：在秒杀活动高峰期，外部用户向订单服务提交下单请求，系统应完成库存校验和订单创建，并保证 95% 请求在 2 秒内响应，失败率不超过 1%。

3. 架构评估答题模板

问题：说明如何采用 ATAM 对某系统架构进行评估。

可按以下结构作答：

- 介绍 ATAM 方法，明确评估目标和参与者。
- 说明业务驱动因素，包括核心业务目标、质量属性目标和约束。
- 描述候选架构，包括构件、连接件、部署和关键架构策略。
- 收集质量属性场景，形成效用树。
- 按重要性和实现难度对场景排序。
- 分析架构方法，识别风险、非风险、敏感点和权衡点。
- 通过头脑风暴补充场景，并由利益相关者投票确定优先级。
- 输出 ATAM 报告，包括效用树、场景、分析问题、风险、非风险和架构建议。

4. 论文写作可用段落

在论文中写“架构评估与质量属性保障”时，可这样组织：

在架构方案确定后，我采用质量属性场景对关键非功能需求进行明确描述，重点围绕性能、可用性、安全性和可修改性构造场景。每个场景均从刺激源、刺激、环境、制品、响应和响应度量六个方面展开，确保质量需求可验证、可度量。随后采用 ATAM 思路组织架构评估，邀请业务负责人、架构师、开发、测试、运维和安全人员共同识别关键场景，构建效用树并按重要性和实现难度排序。对候选架构进行分析后，识别出若干敏感点和权衡点，例如缓存策略提升性能但增加一致性维护成本，数据加密增强安全性但增加响应延迟。最后结合风险、非风险和架构策略，确定采用分层解耦、缓存、限流降级、冗余部署和统一认证授权等方案，以保证系统在满足业务功能的同时达到主要质量属性要求。

第 9 章 软件可靠性基础知识

一、本章考试定位

本章核心不是单纯背“可靠性定义”，而是掌握一条工程闭环：
可靠性目标 → 运行剖面 → 可靠性设计 → 可靠性测试 → 可靠性数据 → 可靠性模型 → 可靠性评价
综合知识常考概念、公式、模型分类、容错技术对比；案例分析常考“如何设计高可靠系统”“如何组织可靠性测试”“如何根据失效数据进行评价”；论文可用于“高可靠架构设计”“质量属性权衡”“容错与集群设计”等主题。

二、软件可靠性基本概念

1. 软件可靠性的定义

软件可靠性是指软件产品在**规定条件**下、在**规定时间区间**内完成**规定功能**的能力。也可理解为：在**规定条件**和**规定时间**内，软件**不引起系统失效的概率**。
三个限定词必须一起出现：

关键词	含义	考试抓点
规定条件	软件运行环境、硬件平台、操作系统、数据库、中间件、输入数据范围等	可靠性不是脱离环境的绝对属性
规定时间	通常指软件运行阶段的 运行时间 ，也可用 自然时间 、 运行时间 、 执行时间 度量	执行时间 最精确
规定功能	软件为完成给定服务必须具备的功能	不同任务路径不同，可靠性也可能不同

易错提醒：可靠性不是“没有**缺陷**”，而是在规定使用条件下“不发生**失效**”的概率；**缺陷**是内在问题，**失效**是外在表现。教材强调**软件可靠性**与**规定时间**、**规定条件**、**规定功能**密切相关。

2. 软件可靠性与硬件可靠性的区别

比较点	软件	硬件	考点结论
复杂性	逻辑高度复杂	相对简单	软件 失效 主要来自 设计 错误
物理退化	不存在物理磨损	会老化、磨损	软件不会因“使用久了”自然磨损
唯一性	复制不改变软件本身	每个硬件个体不完全相同	软件副本 故障 通常具有一致性
版本更新	需求 变更、 缺陷 修复频繁	定型后较稳定	软件可靠性 评估更困难

真题对应：常以选择题形式问“软件与硬件可靠性的差异”。看到“软件会物理老化”“软件**失效**主要由物理退化导致”均为错误表述。

三、软件可靠性的定量描述

1. 三种时间

时间类型	含义	可靠性度量价值
自然时间	日历时间，如一天、一个月	最粗略
运行时间	软件从启动到结束的时间	常用
执行时间	CPU 执行程序指令的时间总和	最准确

易错提醒：题目中如果问“度量**软件可靠性**最准确的时间”，优先选**执行时间**。

2. 失效概率 F(t)

从软件开始运行到时刻 t 为止，出现**失效**的概率记为 F(t)。

特点：

- F(0)=0：初始时刻**失效概率**为 0。
- F(t) 随时间**单调递增**。
- F(+∞)=1：运行时间无限增长时，**失效概率**趋近于 1。

3. 可靠度 R(t)

可靠度表示软件在**规定条件**和**规定时间**内不发生**失效**的概率。

$R(t) = 1 - F(t)$

因此：

$$R(0) = 1, \quad R(+\infty) = 0$$

公式熟练：题目给出失效概率 $F(t)$ ，求可靠度，直接用 $1-F(t)$ 。

4. 失效强度 $f(t)$

失效强度是单位时间内软件发生失效的概率，可理解为失效概率函数的变化率：

$$f(t) = \lim_{\Delta t \rightarrow 0} \frac{F(t + \Delta t) - F(t)}{\Delta t} = F'(t)$$

易错提醒：失效强度不是“缺陷数量”，而是单位时间发生失效的概率强度。

5. MTTF、MTTR、MTBF

指标	中文	含义	记忆
MTTF	平均失效前时间	从开始运行到发生失效的平均时间	不可修复系统常用
MTTR	平均恢复前时间	从故障出现到修复成功的平均时间	越短越好
MTBF	平均故障间隔时间	两次故障之间的平均时间	可修复系统常用

关系式：

$$MTBF = MTTF + MTTR$$

真题对应：常给出 MTTF 和 MTTR，让计算 MTBF；或问“哪个指标越短表示易恢复性越好”，答案是 MTTR。

四、可靠性目标

可靠性目标是客户对软件可靠性满意程度的定量期望，常用可靠度、失效强度、MTTF 等指标描述。确定可靠性目标时，必须平衡：

可靠性要求、交付时间、开发成本、失效严重程度。

1. 失效严重程度类

失效严重程度类是对用户造成相同影响程度的失效集合。常见划分依据有：

划分依据	典型表现
成本影响	额外运行成本、修复成本、恢复成本、业务机会损失
系统能力影响	系统崩溃、重要数据丢失、系统停止响应、操作无响应

教材示例中，严重程度最高的失效可表现为“系统崩溃，重要数据不可恢复”；较低级别失效可能只是部分操作无响应且可替代。

易错提醒：可靠性目标不能只看“失效概率”，还要看“失效后果”。低概率但高灾难后果的失效，仍属于高可靠设计重点。

五、广义可靠性测试与狭义可靠性测试

类型	含义	关键词
广义可靠性测试	为最终评价软件可靠性而综合运用建模、统计、试验、分析和评价等手段	全流程、评价导向
狭义可靠性测试	为获取可靠性数据，按预定测试用例，在预期使用环境中实施测试	可靠性试验、数据采集

可靠性测试目的：

- 1. 发现需求、设计、编码、测试、实施中的缺陷。
- 2. 为使用和维护提供可靠性数据。
- 3. 确认软件是否达到可靠性定量要求。

图示理解：教材图 9-1 体现了“确定可靠性目标、定义运行剖面、设计测试案例、实施测试、收集可靠性数据、建模或选取模型、可靠性评价、排错反馈”的闭环过程。

六、软件可靠性建模

1. 影响软件可靠性的因素

从技术角度看，主要包括：

因素	说明	考点
运行剖面	软件实际运行环境和使用方式	同一软件在不同运行剖面下可靠性不同
软件规模	代码量、功能量、模块规模	规模越大，缺陷概率通常越高
软件内部结构	模块复杂度、耦合关系、控制路径	结构越复杂，可靠性风险越高
开发方法与环境	结构化方法、工具、人员水平、需求变化频度	工程方法影响缺陷引入
可靠性投入	可靠性设计、管理、测试、评价投入	早期投入更有效

易错提醒：可靠性不是只靠测试提高，需求、设计、编码、测试、实施全生命周期都影响可靠性。

2. 软件可靠性模型的组成

一个软件可靠性模型通常包括：

- 1. 模型假设：如测试用例代表实际运行剖面、失效独立发生等。
- 2. 性能度量：如失效强度、残留缺陷数、可靠度。
- 3. 参数估计方法：用历史失效数据估计模型参数。
- 4. 数据要求：不同模型要求不同可靠性数据。

3. 三个共同假设

假设	含义	易错点
代表性假设	测试用例能代表实际运行剖面	测试数据才能用于预测运行可靠性
独立性假设	失效在不同时刻独立发生	一个失效不影响另一个失效
相同性假设	不区分失效严重程度	只看发生时间，不看后果等级

真题对应：模型假设常以“哪项不是软件可靠性模型共同假设”形式出现。

4. 软件可靠性模型分类

教材列出的主要模型类型可按名称识记：

模型类别	核心思想
种子法模型	人为植入错误，根据发现比例估计残留错误
失效率类模型	研究程序失效率变化
曲线拟合类模型	用回归分析研究缺陷数、失效率、失效间隔
可靠性增长模型	用增长函数描述排错后可靠性改善
程序结构分析模型	根据模块调用关系形成可靠性分析网络
输入域分类模型	按输入域样本及使用概率推断可靠性
执行路径分析模型	计算路径执行概率和错误路径概率
非齐次泊松过程模型	以单位时间失效次数预测累计失效数
马尔可夫过程模型	用状态转移描述可靠性行为
贝叶斯模型	结合先验分布和当前测试失效信息评价可靠性

易错提醒：种子法的缺点是“诱导错误与真实错误的类比性难以估量”；输入域分类模型的难点是“输入域概率分布和运行剖面描述”。

七、软件可靠性管理

软件可靠性管理是软件工程的一部分，目标是全面提高和保证软件可靠性。其核心是把可靠性活动贯穿软件生命周期。教材按阶段列出了可靠性活动：需求阶段确定可靠性目标、验收标准和数据规范；设计阶段进行可靠性设计、预测和文档编制；编码阶段结合单元测试开展可靠性测试与排错；测试阶段开展建模、评价和数据收集；实施阶段结合验收测试进行评价和模型调整。

生命周期活动速记

阶段	可靠性活动重点
需求分析	可靠性目标、影响因素、验收标准、管理框架、数据规范
概要设计	可靠性度量、验收方案、可靠性设计、数据收集
详细设计	可靠性设计、可靠性预测、计划调整
编码	单元测试中的可靠性测试、排错、数据收集
测试	集成/系统测试中的可靠性测试、建模、评价
实施	验收测试、排错、模型调整、可靠性评价

易错提醒：可靠性管理不是测试阶段才开始；从需求阶段就要确定目标和数据规范。

八、软件可靠性设计

本章最高频、最实用的考点是：保障软件可靠性最有效、最经济、最重要的手段是在软件设计阶段进行可靠性控制。可靠性设计是在常规软件设计中引入可靠性技术，使程序在满足功能和性能需求的同时满足可靠性要求。教材强调可靠性设计不能脱离总体设计框架，也不能无限扩大，必须在功能、用户需求、费用等约束下权衡。

1. 容错设计技术

适用于软件失效后果特别严重的场景，如飞行控制、空中交通管制、核反应堆安全控制等。

(1) 恢复块设计

思想：为同一功能准备多个设计差异的程序块。正常执行主块，若主块失败，则切换到备份块。

关键词：动态冗余、后备文本、验收测试、失败后恢复。

(2) N 版本程序设计

思想：由不同团队、不同方法、不同语言或工具实现多个版本，对同一输入的输出结果进行多数表决。

关键词：多版本、多数表决、设计不相关性。

要求：

- 需求说明必须完整、精确。
- 不同版本的设计过程尽可能不相关。

易错提醒：N 版本程序设计不是复制同一套代码，而是要求实现差异和设计不相关。

(3) 冗余设计

思想：在主系统之外设计不同路径、不同算法或不同实现方式的备份模块，故障时替换。

与硬件冗余不同：软件如果只是复制一套相同系统，通常意义不大，因为相同输入和环境下可能在同一缺陷处失败。

2. 检错技术

适用于不需要在线容错或不能采用冗余技术，但对可靠性要求较高的部分。

四个设计要素：

要素	说明
检测对象	检测点、检测内容，放在易错处或影响大的地方
检测延时	故障发生到被发现之间的时间
实现方式	返回值判断、运行时间监控、状态标志位等
处理方式	停止运行、报警、局部停机、人工干预等

易错提醒：检错技术能发现并报警，但通常不能自动修复故障；容错技术才强调故障后继续提供服务。

3. 降低复杂度设计

思想：在保证功能的前提下，简化结构、缩短代码、优化数据流、降低模块复杂性和结构复杂性。

考试表达：软件复杂性是产生缺陷的重要根源，复杂度超过一定阈值后，缺陷数可能急剧上升，可靠性急剧下降。

4. 系统配置技术

(1) 双机热备

由两台服务器、共享磁盘阵列、**双机热备**软件组成。通过“心跳”检测主备状态，主机**故障**时备机**接管**。
三种模式：

模式	特点
双机热备 Active/Standby	一台工作，一台监控待命；可靠但资源利用率低
双机互备	两台分别运行不同应用，互为备份；对性能要求高
双机双工	两台均活动并运行相同应用；兼顾 负载均衡 和互备

(2) 服务器集群

多台独立服务器组合成单一系统，对外提供统一服务。某节点**故障**时，应用可被其他节点自动**接管**。
真题对应：常考“**双机热备**、互备、双工、集群”的区别。看到“心跳检测”“主备**切换**”优先想到**双机热备**；看到“多台服务器组成单一系统、节点**接管**”优先想到集群。

九、软件可靠性测试

1. 可靠性测试活动

主要包括：
确定可靠性目标 → **开发运行剖面** → **设计测试用例** → **实施测试** → **分析测试结果**。
可靠性测试最好在**受控的自动测试环境**下，由专业**测试**机构完成。可靠性测试虽然不能保证残存**缺陷**最少，但可以帮助确认软件是否达到较高可靠性要求。

2. 定义运行剖面

运行剖面用于描述软件真实使用情况。可用马尔可夫链建模，将**输入域**抽象为状态集，用弧表示**状态转换**，并为弧分配**转换概率**。
运行剖面开发包括：

层次	含义
用户级分层	谁在使用系统，以及如何使用
用法级分层	系统能够做什么，即功能使用方式

概率来源优先级：
1. 实际用户数据、已有系统数据、**原型数据**。
2. **用户访谈**或观察。
3. 原型**测试**分析结果。
4. **领域专家意见**。
5. 没有数据时才平均分配概率，这是最差方案。
易错提醒：**高安全系统**不能只按“使用频率”分配**测试**资源。起飞、降落等低频但高风险路径必须单独**设计运行剖面**和补充测试。

3. 可靠性测试用例设计

测试用例应真实反映**运行剖面**，并把统计方法用于用例生成和资源分配。
典型**测试用例**包括：**测试用例**标识、被测对象、**测试**环境及条件、**测试**输入、操作步骤、预期输出、输出判定标准、特殊需求。
可靠性测试应重点考虑：

特殊情况	考查点
用户误操作	是否能屏蔽、提示、纠正
错误提示	是否准确、可理解
异常退出	错误操作是否导致崩溃
数据可靠性	输入校验、冗余过滤、校验、清洗

特殊情况	考查点
异常情况	硬件 故障 、网络 故障 、模块 失效 后的影响与补救

4. 可靠性测试实施

实施要点：

- 测试前检查需求、设计、程序、数据、支持环境是否一致。
- 可靠性测试必须是受控测试。
- 为保证统计有效性，每个测试用例应使用同一软件版本；新版本意味着新测试开始。
- 非统计测试如覆盖测试最好在可靠性测试之前完成。
- 测试过程必须真实记录失效数据。
- 测试结束应形成可靠性测试报告。

可靠性数据四类：

数据类型	记录内容
失效时间数据	发生一次失效所累积经历的时间
失效间隔时间数据	本次失效与上次失效之间的间隔时间
分组时间内失效数	某时间区间发生的失效次数
分组时间累积失效数	到某区间为止的累计失效次数

易错提醒：更换软件版本后继续沿用旧版本可靠性测试数据，会影响统计结论有效性。

十、软件可靠性评价

软件可靠性评价是选用或建立可靠性数学模型，运用统计技术，对测试和运行期间收集的失效数据进行处理，进而评估和预测软件可靠性的过程。包括三件事：选择可靠性模型、收集可靠性数据、可靠性评估和预测。

1. 如何选择可靠性模型

选择模型时看四点：

选择因素	判断标准
模型假设适用性	模型假设是否符合实际系统
预测能力与质量	是否能较准确预测未来可靠性
输出值是否满足评价需求	能否给出可靠度、MTTF、故障密度、达标日期、成本等
使用简便性	数据是否易收集，模型是否易理解，工具是否易用

易错提醒：不存在适用于所有软件系统的“最好模型”；模型必须服务于评价目的和数据条件。

2. 可靠性数据收集

可靠性数据主要是软件失效数据，是评价基础。数据收集贯穿全生命周期，而不是只在测试阶段进行。

常见问题：

- 术语和规范不统一。
- 数据收集缺乏连续性。
- 工具和手段不足。
- 数据完整性难保证。
- 数据质量和准确性难保证。

改进措施：

- 尽早确定模型和数据规范。
- 制定可实施的数据收集计划。
- 重视可靠性测试数据整理和分析。
- 利用数据库存储、统计、分析可靠性数据。

3. 可靠性评估和预测

可回答的问题：

1. 是否达到可靠性目标。

- 2. 是否满足投入使用或**终止测试条件**。
- 3. 若未达标，还需投入多少时间、人力、资金。
- 4. **维护、升级、修改**后是否仍能达到交付可靠性水平。

辅助分析方法：

方法	用途
失效数据图形分析	分析累计 失效 数、单位时间 失效 数、 失效 间隔
EDA 试探性数据分析	发现循环相关、短期 失效 数激增、 失效 集中时间段

高频易错：**测试**期间没有**失效**，不能推出**可靠度**等于 1。除非**完全测试**，但**完全测试**几乎不可能；无**失效**时只能保守估计。

十一、公式与概念速记表

项目	公式/结论
可靠度	$R(t) = 1 - F(t)$
初始 可靠度	$R(0) = 1$
长时间 可靠度	$R(+\infty) = 0$
初始 失效概率	$F(0) = 0$
长时间 失效概率	$F(+\infty) = 1$
失效强度	$f(t) = F'(t)$
平均失效前时间	$MTTF = \int_0^{\infty} R(t) dt$
平均故障间隔时间	$MTBF = MTTF + MTTR$
易恢复性	MTTR 越短越好
可靠性目标	可靠度、失效强度、MTTF 等
运行剖面	对软件实际使用条件、输入分布、操作概率的描述

第 10 章 软件架构的演化和维护

一、本章核心主线

本章可以按一条主线记忆：

为什么要演化 → 演化什么 → 如何演化 → 如何评估 → 如何维护 → 网站架构如何逐步演进。

软件架构通常经历**初始设计**、**实际使用**、**修改完善**、**退化弃用**等阶段，其中“**修改完善**”就是架构演化和维护的核心过程。架构演化的目的，是让软件系统适应用户需求变化、业务环境变化、运行环境变化、**质量属性**要求变化，本质上属于软件整体结构在生命周期内的持续调整。

二、10.1 软件架构演化和定义的关系

1. 演化的重要性

软件架构演化的本质是软件整体结构的演化，覆盖架构需求获取、架构建模、架构文档、架构实现和架构维护等阶段。因此教材强调：软件架构是演化来的，而不是一次性设计出来的。

考试常考三点：

考点	备考理解
为什么架构要演化	为适应新需求、新业务环境、新运行环境，维持架构有用性
架构演化的价值	保证 性能、可靠性、安全性、可维护性 等 质量属性 持续满足需求
为什么从架构层演化更合理	架构是系统蓝图，便于观察复杂性、变化性和 波及效应 ，修改成本相对较低

2. 架构定义决定演化关注点

若将软件架构定义为：

$SA = \{components, connectors, constraints\}$

则**软件架构演化**主要关注三类元素：

架构元素	含义	演化表现	易错点
Components 组件	计算元素、数据存储、重要模块	增加、删除、修改模块	组件 变化通常会引发消息或连接关系变化
Connectors 连接件	组件 之间的交互关系	增加、删除、修改交互消息	连接件 演化经常伴随 组件 演化
Constraints 约束	组件 与 连接件 之间的拓扑、配置、参数、 约束 数据	增加、删除、修改 约束 数据	约束 变化可能直接影响架构正确性

易错提醒：不要把架构演化理解为“只改代码”。考试中更强调架构层面的**组件**、**连接件**、**约束**及其波及效应。

三、10.2 面向对象软件架构演化过程

本节以 **UML 顺序图** 为基础讨论架构演化，重点考查对象、消息、复合片段、**约束**四类演化。

1. 对象演化

操作	全称	含义	典型场景
AO	AddObject	增加对象	增加新功能，或把已有对象的部分职责独立出来
DO	DeleteObject	删除对象	移除功能，或合并对象以降低复杂度

考试关键句：对象如果没有与现有对象发生交互，对当前架构正确性或时态属性通常没有实际影响。因此，**对象演化**往往伴随**消息演化**。

2. 消息演化

消息是**顺序图**的核心元素，包含名称、源对象、目标对象、时序等信息。**消息演化**最容易影响对象交互、架构正确性和时态属性。

操作	全称	含义	记忆关键词
AM	AddMessage	增加消息	新增交互
DM	DeleteMessage	删除消息	移除交互， AM 的逆向
SMO	SwapMessageOrder	交换消息顺序	改变时序关系
OM	OverturnMessage	反转消息发送方与接收方	交互方向变化
CMM	ChangeMessageModule	改变消息发送或接收对象	交互对象变化

消息演化与**约束**关系可分为三类：

类型	是否关联约束	是否违背约束	结论
第 1 类	无关	不违背	一般不影响正确性
第 2 类	直接关联	不违背	一般仍正确
第 3 类	直接关联	违背	属于 不正确演化

易错提醒：消息是**顺序图**演化的核心。对象、复合片段、**约束**的演化经常要回到消息层面判断。

3. 复合片段演化

复合片段描述对象交互中的控制流，属于**连接件**范畴。常见会产生分支的复合片段包括：**ref**、**loop**、**break**、**alt**、**opt**、**par**。

操作	全称	含义	考点
AF	AddFragment	增加复合片段	增加控制流
DF	DeleteFragment	删除复合片段	移除控制流， AF 的逆向
FTC	FragmentTypeChange	改变复合片段类型	常视为删除旧片段 + 添加新片段
FCC	FragmentConditionChange	改变执行条件	同时影响满足条件与不满足条件两类转移

易错提醒：**复合片段演化**不是单纯“改框”。它会改变消息执行流程，可能导致原**约束**被破坏，因此演化后必须验证。

4. 约束演化

操作	全称	含义
AC	Add Constraint	增加 约束 ，需要判断当前设计是否满足新 约束
DC	Delete Constraint	删除 约束 ，一般更容易满足演化后的 约束

约束缺乏可视化描述，因此“修改**约束**”通常可理解为：删除原**约束** + 增加新**约束**。

四、10.3 软件架构演化方式分类

1. 三种典型分类方法

分类角度	分类结果
按实现方式和实施粒度	基于过程和函数、面向对象、基于 组件 、基于架构的演化
按研究方法	演化支持、版本和工程管理工具、架构变换形式方法、成本收益分析

分类角度	分类结果
按是否发生在运行期	静态演化、动态演化

2. 软件架构演化时期

时期	英文	发生阶段	重点
设计时演化	Design-Time Evolution	编译前，模型和代码形成前	调整架构模型
运行前演化	Pre-Execution Evolution	编译后、执行前	不考虑程序运行状态，但要考虑体系结构
有限制运行时演化	Constrained Runtime Evolution	运行中但满足特定 约束	安全模式下的规定演化
运行时演化	Runtime Evolution	系统运行中	最难，包括增删 组件 、替换 组件 、改变拓扑

真题对应：选择题常用“是否停机”“是否运行中”“是否预先规定**约束**”来区分四种时期。

五、静态演化

1. 静态演化需求

静态演化发生在系统未运行或停止运行时，主要有两类需求：

类型	含义
设计时演化需求	在架构开发和实现过程中调整原架构，保证实现与架构一致
运行前演化需求	软件发布后因 运行环境变化 而修改升级

静态演化对应常见维护类型：**更正性维护**、**适应性维护**、**完善性维护**。

2. 静态演化一般过程

可按五步记忆：

软件理解 → 需求变更分析 → 演化计划 → 系统重构 → 系统测试

步骤	要点
软件理解	查文档、分析架构、识别组成元素和关系
需求变更分析	找出新旧需求差异
演化计划	确定范围、成本和方案
系统重构	按计划调整系统结构
系统测试	验证演化后系统正确性与不足

3. 静态演化原子操作

一次完整架构演化可拆成一系列**原子演化操作**。原子操作是逻辑语义上粒度最小的架构修改操作，不等于物理上完全不可拆。

与可维护性相关的操作

操作	含义
AMD	增加模块间依赖
RMD	删除模块间依赖
AMI	增加模块间接口
RMI	删除模块间接口
AM	增加模块
RM	删除模块
SM	拆分模块
AGM	聚合模块

与可靠性相关的操作

操作	含义
AMS/RMS	增加/删除 顺序图 中的交互消息
AO/RO	增加/删除 顺序图 中的交互对象
AF/RF/CF	增加/删除/修改消息片段
AU/RU	增加/删除用例
AA/RA	增加/删除参与者

易错提醒：**可维护性**更偏**组件图**、模块依赖、接口、聚合拆分；**可靠性**更偏用例图、部署图、**顺序图**、交互复杂度和运行风险。

4. 正交软件架构

正交软件架构通过分层和线索化降低修改影响范围。同一层次中的**组件**不允许相互调用，因此每次变动通常只影响一条线索。
演化过程：

需求变动归类 → 制订架构**演化计划** → 修改/增加/删除**组件** → 更新**组件**相互作用 → 形成演化后架构

六、动态演化

动态演化发生在系统运行期间，需要在**不停机**或尽量不影响服务的情况下完成演化，难度高于**静态演化**。

1. 动态演化需求来源

来源	示例
软件内部执行导致结构改变	服务器端软件为客户请求创建 新组件
外部请求导致系统重配置	操作系统升级无需重启，在运行中完成结构调整

适合**动态演化**的系统通常是长期运行、停机代价高、风险高的系统，如航空航天、生命维持、金融、交通等。

2. 软件动态性三级

级别	名称	含义	考试关键词
级别 1	交互动态性	固定结构下的数据动态交互	结构不变
级别 2	结构动态性	可添加/删除 组件 和 连接件 实例	主流研究与应用
级别 3	架构动态性	架构基本构造可被重定义	真正动态架构

3. 动态演化内容

内容	例子
属性改名	重新定义服务响应时间等非功能指标
行为变化	更换加密算法，HTTP 改 HTTPS
拓扑结构改变	增删 组件 、增删 连接件 、改变关联关系
风格变化	两层 C/S 调整为三层 C/S 或 C/S 与 B/S 混合结构

4. DSA 与 DR

技术	全称	关注点	对应动态性
DSA	Dynamic Software Architecture	运行时变化的系统框架结构	架构动态性
DR	Dynamic Reconfiguration	运行时修改 组件 、 连接件 、配置关系	结构动态性

5. DSA 实施动态演化的四步

捕捉并分析需求变化 → 获取或生成架构**演化策略** → 选择并实施**演化策略** → 演化后评估与检测

DSA 要满足三个关键要求：

要求	含义
保存当前架构信息	包括拓扑结构、 组件 状态、 组件 数量等
设置 监控管理机制	监视需求变化，判断能否演化、何时演化、演化范围
保证 操作原子性	任一操作失败，整个操作集应撤销，避免系统不稳定

6. DSA 描述语言

语言	视角	记忆点
π-ADL	行为视角	基于 π 演算 ，描述动态行为，适合移动系统建模
Pilar	反射视角	模型修改反映到系统，系统变化也反映到模型
LIME	协调视角	分离计算部分与协调部分，支持移动和分布式系统

7. 动态重配置四种模式

模式	要点
主从模式	主 组件 分配任务，从 组件 执行并返回结果
中央控制模式	中央控制器控制多个 组件 ，常用于实时系统
客户端/服务器模式	客户端请求服务端，事务完成后可添加/删除 组件
分布式控制模式	功能分布在多个控制 组件 中，如环形、顺序模式

8. 动态重配置难点

四个难点要熟记：

- 1. **约束**定义困难。
- 2. **性能约束**难以静态衡量，需要运行时评估。
- 3. 某些重配置方案只能解决部分**性能约束**，难以管理所有方面。
- 4. 重配置必须同时保证**组件**系统完整性和重配置策略正确、安全。

七、10.4 软件架构演化 18 项原则

本节是选择题高频区，建议按“成本、进度、风险、结构、质量、适应性”分组记忆。

序号	原则	核心含义	度量或判断
1	演化成本控制 ECC	演化成本应明显小于重新开发成本	$CoE \ll CoRD$
2	进度可控	演化应在预期时间内完成	$t_task = T_task - T'_task $ ，越小越好
3	风险可控	经济、时间、人力、技术、环境风险可控	分别检验
4	主体维持	主体行为保持稳定，平均增量平稳	$AIG = \text{主体规模变更量} / \text{主体规模}$
5	系统总体结构优化	演化后整体结构更合理	看整体 可靠性 和 性能
6	平滑演化 IWR	生命周期内演化速率趋于稳定	$IWR = \text{变更总量} / \text{项目规模}$
7	目标一致	阶段目标与最终目标一致	$o_task = O_task - O'_task $ ，越小越好
8	模块独立演化	模块自身演化尽量相互独立	检查修改是否局部化
9	影响可控	模块变更对其他模块影响可预测	计算修改影响范围
10	复杂性可控	架构复杂度保持可控	$CC < \text{阈值}$
11	有利于重构	演化后更便于重构	检查复杂度指标
12	有利于重用	维持或提高可重用性	高内聚、低耦合
13	设计原则遵从	不破坏架构设计原则	$RCP = CDP / DP $ ，越小越好
14	适应新技术 TI	避免过度依赖特定技术	$TI = 1 - DDT$
15	环境适应性	易适应新的软硬件环境	看兼容性
16	标准依从性	不违背相关标准	人工判定
17	质量向好 QI	演化后质量指标更好	$EQI \geq SQ$
18	适应新需求	不降低对新需求的适应能力	$RNR = ANR / NR $ ，结合教材语义理解

易错提醒：

- 影响可控**强调“修改带来的**波及范围**”。
- 模块独立演化强调“每个模块是否能局部变化”。
- 复杂性可控**强调演化后是否仍易维护、易扩展、易分析、易测试。
- 质量向好**不是只看功能是否新增，而是看**质量属性**是否改善。

八、10.5 软件架构演化评估方法

1. 两类评估

类型	含义	方法
演化过程已知	已知每一步怎么改	度量每个中间版本，比较 质量属性 变化
演化过程未知	不知道具体怎么改	比较演化前后度量结果，反推可能的演化操作

2. 演化过程已知的评估流程

可按下面顺序记忆：

$$A_0 \rightarrow A_1 \rightarrow A_2 \rightarrow \cdots \rightarrow A_n$$

每次原子演化形成一个中间版本，对每个版本计算**质量属性**值：

$$Q_0, Q_1, ..., Q_n$$

再计算相邻版本**质量属性**距离：

$$D(i-1, i)$$

最终比较演化前后**质量属性**差异。

3. 可维护性距离

可维护性由 6 个指标构成向量，例如：

$$A = (a_1, a_2, ..., a_6), \quad B = (b_1, b_2, ..., b_6)$$

按归一化笛卡儿距离思想理解：

$$D_m(A, B) = \sqrt{\sum_{i=1}^6 \left(\frac{a_i - b_i}{\max(a_i, b_i)} \right)^2}$$

值越大，说明两个架构版本在**可维护性**上的差距越大。

4. 可靠性距离

可靠性度量值为实数，可理解为简化的归一化距离：

$$D_r(A, B) = \frac{|a - b|}{\max(a, b)}$$

值越大，**可靠性**差异越大。

5. 重要结论

结论	考试意义
质量属性距离适合比较同一架构的不同版本	不适合比较两个完全不同系统
架构质量属性距离与物理结构差异不一定正相关	质量近似不代表结构近似
D(0, n) 不等于各相邻版本距离简单相加	原子演化影响不具有简单累加性
距离曲线可帮助发现关键演化点	可用于故障定位、风险控制、维护决策

6. 演化过程未知时的评估

步骤：度量演化前后版本 → 计算质量属性距离 → 反推可能演化操作 → 找出作用位置 and 对象 → 分析演化驱动原因 → 判断质量变化是否符合预期。

如果目标是重构以提升可维护性，但度量结果显示图复杂度升高、耦合度升高，则说明本次演化可能没有达到目标，甚至引入了新的质量问题。

九、10.6 大型网站系统架构演化实例

大型网站架构演化的核心挑战是：庞大用户、高并发访问、海量数据。本节很适合案例题和论文素材。

阶段	架构形态	解决问题	易考点
1	单体架构	小规模访问，一台服务器部署应用、数据库、文件	简单但扩展能力弱
2	垂直架构	应用、数据库、文件分离	不同服务器承担不同职责
3	使用缓存	缓解数据库访问压力	80%访问集中在 20%数据；本地缓存快但受内存限制，远程缓存容量更强
4	应用服务器集群	提升并发处理能力	负载均衡调度请求
5	数据库读写分离	缓解数据库瓶颈	写主库、读从库，主从复制
6	CDN + 反向代理	加快响应，减轻后端压力	CDN 靠近用户，反向代理在中心机房
7	分布式文件系统和分布式数据库	解决文件与数据持续增长	分布式数据库是最后手段，常先业务分库
8	NoSQL 和搜索引擎	支持复杂数据存储与检索	通过统一数据访问模块降低复杂性
9	业务拆分	应对复杂业务	按产品线拆分应用，应用独立部署
10	分布式服务	复用公共业务，降低连接复杂度	将用户管理、商品管理等公共业务独立为服务

案例题答题模板

当系统出现性能瓶颈时，不要只说“升级服务器”。应按瓶颈定位回答：

数据库读压力大：缓存、读写分离、分库分表。

应用并发不足：应用服务器集群、负载均衡。

静态资源访问慢：CDN、反向代理。

文件量大：分布式文件系统。

业务复杂度高：业务拆分、服务化、分布式服务。

十、10.7 软件架构维护

1. 架构维护的定位

软件架构是连接需求、设计、实现的桥梁。架构维护与架构演化密不可分，维护要对演化过程进行追踪和控制，确保架构演化满足需求。

架构维护主要包括：架构知识管理、架构修改管理、架构版本管理。

2. 架构知识管理

教材给出：

架构知识 = 架构设计 + 架构设计决策

也就是说，不仅要知道“采用了什么架构”，还要知道“为什么采用这种架构”。

内容	备考理解
架构知识管理含义	记录架构方案、设计决策、设计依据、质量属性等
为什么需要	防止关键设计知识沉没或随人员流动而丢失
现实问题	文档化成本高、动机不足、知识难共享、难搜索

易错提醒：架构知识管理不是普通文档管理，重点在于记录架构设计决策及其理由。

3. 架构修改管理

核心做法是建立隔离区域 Region of Quiescence，使修改对其他部分影响较小甚至没有影响。

要明确：

修改规则。

修改类型。

影响范围。

副作用。

4. 架构版本管理

架构版本管理为演化控制、使用、评价提供依据，也为架构演化量化度量奠定基础。

常见分析手段：

组件—连接件模型。

邻接矩阵。

可达矩阵。

波及效应分析。

5. 可维护性度量实践

本节可作为公式题和案例题素材。

指标	含义	备考理解
CCN 圈复杂度	独立执行路径条数	越高测试和维护越难，实践上常以 $CCN \leq 10$ 为宜
FFC 扇入扇出度	主动调用与被调用频率	越高说明关联关系越多
CBO 模块间耦合度	模块与其他模块交互频繁程度	越高越易受其他模块修改影响
RFC 模块响应	执行所需功能数量	越大复杂度越高
TCC 紧内聚度	模块内聚程度	高内聚更好
LCC 松内聚度	模块内聚程度	结合 TCC 判断

教材实践中给出示例公式：

$$CCN = (totalE - totalN) + 2L$$
$$FFC = CCN \times (E + W) \times (X + R)^2$$
$$RFC = S + E + R$$

其中，L 为组件图数目，totalE 为外部组件相关边数，totalN 为外部组件数；S 为内部组件数，E 为依赖出边数，X 为依赖入边数，R 为使用接口数，W 为提供接口数。

第 11 章 未来信息综合技术

一、本章命题定位

第 11 章属于新技术综合应用章，考查重点不是深层算法推导，而是“概念识别 + 架构对比 + 应用场景判断 + 安全与协同理解”。教材将本章划分为 CPS、人工智能、机器人、边缘计算、数字孪生、云计算、大数据等内容，这些技术虽然分散，但共同主线是：感知物理世界 → 数据采集与传输 → 计算分析 → 智能决策 → 反馈控制/优化服务。

考试高频问法：单选常考定义、分类、特征、服务模式；案例题常把边缘计算、云计算、AI、大数据、数字孪生放在智能制造、智慧园区、工业互联网、视频监控等场景中，要求判断“为什么放边缘、为什么上云、为什么需要数字孪生或 CPS”。

二、信息物理系统 CPS

1. 核心概念

CPS，即 Cyber-Physical Systems，是控制系统、嵌入式系统在网络化、智能化条件下的扩展。它把感知、计算、通信、控制与自动控制技术融合起来，使物理空间中的人、机、物、环境与信息空间中的模型、数据、知识形成映射、交互和协同。其本质是构建一个基于数据自动流动的闭环体系：状态感知 → 实时分析 → 科学决策 → 精准执行。

记忆公式：

$$CPS = \text{感知} + \text{计算} + \text{通信} + \text{控制} + \text{数据闭环} + \text{资源优化}$$

2. 三层体系架构

层级	核心含义	典型功能	考点关键词
单元级 CPS	不可分割的最小 CPS 单元	感知、计算、通信、控制、自决策	智能部件、机器人、智能机床
系统级 CPS	多个单元级 CPS 通过工业网络协同	互联互通、即插即用、边缘网关、数据互操作、协同控制、监视诊断	局部优化、自组织、自配置
SoS 级 CPS	多个系统级 CPS 的系统之系统	数据存储、数据融合、分布式计算、大数据分析、数据服务	全局优化、资产管理、运营优化

教材强调：单元级构成基本闭环，系统级解决多个单元互联协同，SoS 级强调跨系统、跨平台的数据汇聚与全局优化。

3. 技术体系

CPS 技术体系可概括为 总体技术、支撑技术、核心技术，也可浓缩为“四要素”：

一硬：感知和自动控制，是硬件支撑。

- 一软：工业软件，是规则、流程、模型和数据处理的核**心**。
- 一网：工业网络，是**互联互通**和数据传输载体。
- 一平台：工业云和**智能服务平台**，是数据汇聚、服务封装和上层应用支撑基础。

4. 应用与建设路径

CPS 的典型应用包括**智能设计**、**智能生产**、**智能服务**、**智能应用**。其中，**智能设计**强调**仿真**预演，**智能生产**强调设备互联和资源调度，**智能服务**强调**健康管理**、**预测维护**、**远程诊断**，**智能应用**强调**无人装备**、**产业链互动**和**价值链共赢**。建设路径不能一步到位，应按 体系设计 → 单元级建设 → 系统级建设 → SoS 级建设 逐步推进。

易错提醒：CPS 不等于普通嵌入式系统。嵌入式系统更偏单设备**控制**，CPS 强调物理空间与信息空间的**映射**、**交互**、**协同**和**闭环优化**。

三、人工智能 AI

1. 基本概念与分类

人工智能是利用数字**计算机**或由其**控制**的机器来模拟、延伸和扩展人的智能，实现**感知**环境、获取知识并利用知识取得较优结果的理论、方法、技术及系统。按能否真正推理、思考和解决问题，教材分为**弱人工智能**与**强人工智能**。

类型	含义	备考判断
弱人工智能	面向特定任务，表现出智能，但没有真正自主意识	当前主流应用基本属于弱 AI
强人工智能	能真正思维、推理、有知觉或自我意识	技术挑战极大，教材口径认为短期难实现

2. 发展脉络

重点记住几个节点：1950 年图灵测试；1956 年达特茅斯会议标志 AI 研究领域诞生；20 世纪 70 年代第一次寒冬；20 世纪 80 年代专家系统带来复兴；1997 年 IBM 深蓝获胜体现规则型 AI 的里程碑意义；2006 年后**深度学习**兴起；2010 年后在**大数据**、算力和算法推动下进入快速发展阶段。

3. 关键技术

技术	核 心 理解	常见应用
NLP 自然语言处理	让 计算机 理解、生成和处理自然语言	机器翻译、问答系统、语义理解
计算机视觉	让 计算机 提取、处理、理解图像或视频	自动驾驶、机器人、智能医疗
知识图谱	结构化语义知识库，节点和边表达实体及关系	搜索、反欺诈、精准营销
人机交互 HCI	人与 计算机 之间的信息交换	图形、语音、体感、脑机 交互
VR/AR	构建或增强数字化环境，增强沉浸与 交互	仿真 、培训、设计、展示
机器学习 ML	从数据中学习规律并预测未来数据	分类、回归、推荐、风控、识别

机器学习可按学习模式分为**监督学习**、**无监督学习**、**半监督学习**、**强化学习**；按方法可分为**传统机器学习**与**深度学习**。**监督学习**有标签，**无监督学习**无标签，**半监督学习**少量标签加大量无标签，**强化学习**依靠环境反馈形成策略。

记忆公式：

机器学习 = 经验 E + 任务 T + 性能度量 P

当系统通过经验 E 使任务 T 上的性能 P 得到改善，就可称为**机器学习**。

易错提醒：**深度学习**不是“**无监督学习**”的同义词。**深度学习**是学习方法类别，监督/无监督/**强化学习**是学习模式类别，两者**不是同一分类维度**。

四、机器人技术

1. 发展阶段

机器人发展可按教材记为三代：**第一代示教再现型机器人**、**第二代感觉型机器人**、**第三代智能型机器人**。示教再现型强调按示教轨迹重复动作；感觉型具备力觉、触觉、视觉、听觉等**感知**能力；智能型具备多传感器、逻辑推理、判断和自主决策能力。

2. 机器人 4.0 核心技术

机器人 4.0强调把“云端大脑”与**边缘计算**结合，形成**规模化**、**智能化**、**低时延**服务能力。**核心技术**包括：**云-边-端**无缝协同计算、持续学习与协同学习、**知识图谱**、**场景自适应**、**数据安全**。其中，云侧适合**高性能计算**和**知识存储**，边缘侧适合**近端处理**与**协同共享**，机器人端主要承担**实时操作**。

3. 分类

按**控制**方式：操作机器人、程序机器人、示教再现机器人、智能机器人、综合机器人。按应用行业：工业机器人、服务机器人、特殊领域机器人。工业机器人常见于搬运、焊接、装配、喷漆、检查等场景；特殊领域机器人用于核电站、海底、宇宙空间等人员难以进入的场合。

易错提醒：示教再现机器人不等于智能机器人。前者是“记住并重复”，后者强调根据环境变化调整动作。

五、边缘计算

1. 概念与定位

边缘计算是把数据处理、应用运行和部分功能服务从中心云下沉到靠近数据源、终端或用户的网络边缘侧。其价值在于减少数据回传、降低时延、减轻中心云压力，并增强本地自治能力。教材强调边缘计算不是替代云计算，而是与云计算形成互补协同。

2. 三种落地形态

形态	核心含义	判断关键词
云边缘	云服务在边缘侧延伸，仍强依赖云	云服务下沉
边缘云	在边缘侧构建中小规模云服务能力	MEC、CDN、边缘数据中心
云化网关	用云化技术重构嵌入式网关	协议转换、资源调度、应用管理

3. 特点

边缘计算的特点可记为：联接性、数据第一入口、约束性、分布性。联接性强调多协议、多接口、多网络接入；数据第一入口强调近源采集和数据全生命周期价值；约束性强调工业现场环境、防尘、防爆、抗振、电磁干扰、功耗和空间限制；分布性强调分布式计算、存储、调度、智能和安全能力。

4. 边云协同

边云协同包括六类：资源协同、数据协同、智能协同、应用管理协同、业务管理协同、服务协同。考试中最常考的是智能协同：云端进行 AI 模型训练，边缘节点执行推理；数据协同：边缘采集和初步处理，云端做海量存储、分析和价值挖掘。

5. 安全与应用场景

边缘安全要覆盖基础设施、应用、数据、接入、协议转换、网络和安全运营。典型应用场景包括智慧园区、安卓云与云游戏、视频监控、工业物联网、Cloud VR。视频监控场景强调边缘图像识别、智能存储和边云协同；工业物联网强调协议多样、数据异构、预测性维护、质量检测 and 工艺数据安全。

易错提醒：边缘计算适合局部、实时、短周期处理；云计算适合全局、非实时、长周期的大数据处理。二者是互补，不是替代。

六、数字孪生体

1. 核心概念

数字孪生体是物理实体对象的数字模型，通过实测、仿真和数据分析实时感知、诊断、预测物理实体状态，并通过优化和指令反向调控物理实体行为。教材还强调“数字孪生体”可指技术体系、某个物理对象的数字化模型、某类场景下的数字化模型，或物理实体与数字孪生体及使能实体共同构成的数字孪生系统。

2. 关键技术

数字孪生体的三项核心技术是：建模、仿真、基于数据融合的数字线程。系统工程和 MBSE 是顶层框架技术，物联网是底层伴生技术，云计算、机器学习、大数据、区块链等属于外围使能技术。

技术	作用
建模	将对物理世界的理解数字化、模型化，是数字孪生的源头
仿真	验证模型有效性，模拟物理世界状态和参数
数字线程	支撑多视图、多阶段、多尺度模型之间的数据贯通
MBSE	以模型为中心进行系统工程管理
IoT/云/边/大数据/AI	提供数据采集、存储、计算、分析、学习与优化能力

3. 应用

数字孪生体主要应用于制造、产业、城市和战场。制造领域中，设计阶段用于模型设计、模拟仿真和性能验证；制造阶段用于数字生产线、过程监控、能力评估和参数优化。城市领域中，数字孪生城市可实现城市信息集成、实时监控、可视化、科学决策和智能管理。

易错提醒：数字孪生不是简单的三维模型。三维模型偏“形”，数字孪生强调“模型 + 数据 + 实时交互 + 仿真预测 + 反馈优化”。

七、云计算

1. 概念

云计算既可指一种系统平台，也可指一类通过互联网访问的可扩展应用。平台层面强调按需动态部署、配置、重配置和取消服务；应用层面强调用户通过互联网和标准浏览器访问云端计算、存储和服务能力。

2. 三类服务方式

服务模式	提供内容	用户控制粒度	典型判断
SaaS 软件即服务	直接提供应用软件	最少	用户订购、浏览器使用
PaaS 平台即服务	提供开发环境、平台、API、运行环境	中等	开发者在平台上构建应用

服务模式	提供内容	用户控制粒度	典型判断
IaaS 基础设施即服务	提供计算、存储、网络、虚拟机等基础资源	最大	虚拟服务器、存储资源池

教材给出一个重要对比：**灵活性**：SaaS → PaaS → IaaS 依次增强；**方便性**：IaaS → PaaS → SaaS 依次增强。PaaS 在**云计算**体系中起支撑作用，为上层 SaaS 应用提供 API 和平台能力。

3. 部署模式

云计算按**部署模式**可分为**公有云**、**社区云**、**私有云**、**混合云**。**公有云**面向公众，**私有云**面向单个组织，**社区云**面向有共同任务和安全需求的组织群体，**混合云**是多种云的组合。安全性、**控制力**和**成本**是场景选型的常见依据。

易错提醒：SaaS、PaaS、IaaS 是**服务模式**；**公有云**、**私有云**、**社区云**、**混合云**是**部署模式**。二者分类维度不同。

八、大数据

1. 定义与特征

大数据是指**规模**或**复杂性**超出常用软件工具在合理成本和时限内完成捕获、管理、处理、分析和可视化能力的**数据集合**。教材综合多种定义，重点强调 **Volume** 数据量、**Velocity** 速度、**Variety** 多样性、**Value** 价值，并补充可变性和**复杂性**。

记忆公式：

大数据 = 海量 + 多样化 + 快速处理 + 价值

2. 研究挑战与处理流程

大数据面临的核心问题包括**异构性**、**规模**、**时间性**、**复杂性**、**隐私性**。研究挑战可归纳为**数据获取**、**数据结构化**、**数据集成**、**数据分析建模**、**结果解释与交互**。教材给出的**大数据**分析流程是：**数据获取/记录** → **信息抽取/清洗/注记** → **数据集成/聚集/表现** → **数据分析/建模** → **数据解释**。

阶段	主要任务	易考点
数据获取/记录	压缩、在线分析、元数据、数据来源追踪	数据从哪里来、保留什么
信息抽取/清洗	从复杂结构数据中提取可分析信息，处理虚假与错误数据	脏数据、虚假数据
数据集成/表示	统一异构数据结构与语义	数据孤岛、语义不一致
查询/建模/分析	处理噪声、动态、异构、关联和不可信数据	可扩展算法、SQL 与挖掘结合
解释	让决策者理解结果、验证过程、追踪出处	数据出处、可视化、可解释性

3. 应用领域

大数据主要应用于**制造业**、**服务业**、**交通行业**和**医疗行业**。**制造业**中，**大数据**可支持预测性维护、质量改进、流程优化和服务化转型；**服务业**中，可用于个性化推荐、金融反欺诈、旅游价格预测等；**交通行业**用于智慧交通和出行路线优化；**医疗行业**用于电子病历、决策支持和**医疗资源优化**。

易错提醒：**大数据**不是“数据越多越好”。考试更重视“数据是否能清洗、集成、分析、解释并产生价值”。

九、横向对比：案例题最容易混淆的点

技术	解决的核心问题	最适合的关键词
CPS	物理空间与信息空间闭环控制	感知-分析-决策-执行、资源优化
AI	从数据和知识中形成智能判断	NLP、视觉、机器学习、推理
机器人	在物理世界执行感知、移动、操作任务	感知-认知-行动
边缘计算	近端处理、低时延、本地自治	实时、低带宽、数据不出场
数字孪生	用数字模型镜像、仿真、预测物理实体	建模、仿真、数字线程
云计算	弹性资源、集中计算、平台化服务	SaaS/PaaS/IaaS、公有/私有/混合
大数据	海量多源数据的价值挖掘	4V、清洗、集成、分析、解释

选型口诀：需要**低时延**和**本地自治**，选**边缘计算**；需要集中训练、海量存储和全局分析，选**云计算**；需要物理对象实时镜像和**仿真**预测，选**数字孪生**；需要物理系统闭环**控制**和**资源优化**，选 **CPS**；需要从数据中识别、分类、预测或决策，选 **AI/机器学习**；需要对海量异构数据进行价值挖掘，选**大数据**。

第 12 章 信息系统架构设计理论与实践

一、本章考试定位

第 12 章不是单纯“概念章”，而是把信息系统、**企业架构**、**TOGAF/ADM**、**信息化**工程、**SOA** 集成案例串成一条完整工程主线。综合知识题常考定义、分类、模型、**ADM** 阶段、**信息化**规划方法；案例题常考“旧系统集成、信息孤岛、**数据交换**、**服务化改造**、**迁移规划**、**架构治理**”；论文题可直接用于“企业**信息化**总体架构设计”“**SOA/企业应用集成**”“**遗留系统改造**”“**数据架构**与流程架构协同”等主题。

本章主线可压缩为一句话：

信息系统架构 = 业务战略牵引 + 业务流程重组 + 应用系统支撑 + 信息基础设施保障；架构师通过 ADM 等方法，把现状架构逐步迁移为目标架构，并在治理、集成、安全、演化中保持系统价值。

二、12.1 信息系统架构基本概念及发展

1. ISA 的基本含义

信息系统架构 ISA 是一种体系结构，用来反映政府、企业或事业单位信息系统各组成部分之间的关系，以及信息系统与业务、技术之间的关系。考试理解不要只把 ISA 理解成“软件结构图”，它更强调业务、组织、流程、数据、应用、技术之间的整体关系。

记忆式：

ISA = 信息对象 + 业务关系 + 技术关系 + 组织管理关系

现代信息系统架构有两个层次：

层次	含义	考试提示
概念层次	包含艺术、科学、方法、建设风格	偏规划、原则、模型、方法论
物理层次	架构工作后形成的物理结构及相互作用	偏硬件、网络、部署、运行结构

易错点：信息系统架构不只是软件架构，也不只是硬件拓扑；它必须服务企业总体业务架构。

2. 信息系统架构的发展

发展线索按“框架化、标准化、企业化”记：

阶段/代表	关键词	备考价值
Zachman 框架	信息、流程、网络、人员、时间、基本原理六视角	企业架构框架的早期代表
美国联邦企业架构框架	公共业务流程、技术引入、信息流、投资协调	政府/大型组织架构治理场景
TOGAF	行业标准架构框架，核心是 ADM	本章最高频方法论考点
企业信息化推动	市场、环境、经营、技术、用户、法律需求	论文和案例常用背景

当前 The Open Group 官方仍将 TOGAF 作为企业架构方法论和框架发布，并提供 TOGAF Standard, 10th Edition 下载；但教材第 12 章按 TOGAF 9 体系讲解 ADM，软考答题应优先按教材口径掌握，除非题干明确考查当前标准版本。([开放组织][2])

3. 信息系统架构的定义

教材采用较通俗的定义：软件或计算机系统的信息系统架构，是该系统的一个或多个结构，结构由软件元素、元素外部可见属性及其关系组成。

考试抓三件事：

关键词	含义	常见考法
软件元素	比构件更一般的抽象	架构不是只列类或对象
外部可见属性	其他元素能感知或依赖的服务、性能、接口等	内部实现细节通常不是架构
元素关系	连接、交互、依赖、约束	架构图必须体现关系

六个理解要点：

- 1. 架构是对系统的抽象，不关心纯内部实现细节。
- 2. 架构由多个结构组成，单一视图不能代表大型系统全貌。
- 3. 任何软件都有架构，但不一定有及时准确的架构文档。
- 4. 架构既描述静态大粒度结构，也描述关键动态行为。
- 5. 架构具有基础性，体现复用方案和代价高昂的关键决策。
- 6. 架构隐含决策，需要围绕功能需求、质量属性、约束、冲突进行权衡。

易错提醒：文档过时不等于架构不存在；“架构文档”只是架构描述，不能等同于真实运行架构。

三、12.2 信息系统架构

1. 架构风格

架构风格是某一应用领域中系统组织方式的惯用模式。它定义一个系统家族，包含构件/连接件词汇表和组合约束。其价值是支持架构级复用，但具体选型必须结合项目特点。

通用架构风格按五类记：

风格类别	典型形式	场景提示
数据流风格	批处理序列、管道/过滤器	报表批处理、编译器、流水处理
调用/返回风格	主程序/子程序、面向对象、层次结构	传统应用、分层系统
独立构件风格	进程通信、事件系统	发布/订阅、消息通知
虚拟机风格	解释器、规则系统	脚本、规则引擎、专家系统
仓库风格	数据库、超文本、黑板系统	数据中心、多知识源协同

2. 信息系统架构分类

信息系统架构可分为物理结构和逻辑结构。

分类	核心含义	典型内容
物理结构	硬件资源在空间上的拓扑分布	集中式、分布式、C/S
逻辑结构	功能子系统的综合体	供应、生产、销售、人事、财务等子系统

物理结构

类型	优点	缺点	考试判断
集中式	资源集中、便于管理、利用率较高	单点脆弱、扩展和维护困难、用户主动性弱	早期主机/单机系统
分布式	扩展方便、适应变化、局部故障不致整体瘫痪	标准统一难、协调和规划难	现代企业主流
一般分布式	服务器提供文件/程序/数据服务	业务逻辑不一定集中管理	文件服务型
C/S	客户机发请求，服务器提供加工后的信息	客户端和服务端职责分离	信息系统常见结构

逻辑结构综合

综合方式	含义	例子
横向综合	同一管理层次不同职能综合	基层人事与工资业务一体化
纵向综合	同一职能不同管理层次综合	工厂会计系统与公司会计系统集成
纵横综合	信息模型和处理模型综合	公用数据库、统一信息处理系统

易错提醒：子系统性能好，不代表整体系统性能好；信息系统开发必须重视子系统之间的协调、一致和共享。

3. 信息系统架构的一般原理

信息系统架构的基本原理可概括为：

识别稳定部分 + 隔离变化部分 + 在稳定基础上重组易变部分 = 系统柔性

信息系统之所以比计算机体系结构、网络体系结构更复杂，是因为它不仅包含计算机、网络和数据，还包含人、组织、管理、战略、业务流程等因素。架构设计的目标不是“把系统做死”，而是在稳定组成成分和稳定关系的支撑下，适应环境、技术和业务变化。

四、信息系统常用 4 种架构模型

1. 单机应用模式 Standalone

单机应用是运行在一台物理机器上的独立应用程序，可以多进程或多线程。它适合独立专业软件或早期复杂算法系统，如 CAD/CAM、图形图像处理等。考试不应误认为“单机=简单”，大型单机专业软件也可能非常复杂。

2. C/S 模式

C/S 本质上可理解为基于 TCP/IP 的进程间通信结构：客户端发送请求，服务器处理并返回结果。

两层 C/S

结构	特点	易错点
前台客户端 + 后台数据库	典型胖客户端	业务逻辑往往在客户端，与界面耦合
常见工具	PowerBuilder、Delphi、VB 等	维护和升级成本较高

三层 C/S 与 B/S

三层 C/S 通常包括表示层、业务逻辑层、数据层。B/S 是最典型的三层 C/S 应用模式：浏览器通过 HTTP 与 Web 服务器交互，浏览器降低了客户端开发和维护成本。

常见通信方式包括 Socket、自定义消息机制、RPC、CORBA/IIOP、Java RMI、JMS、HTTP。

易错提醒：B/S 不是脱离 C/S 的新物种，本质是采用通用客户端界面的三层 C/S；浏览器与 Web 服务器之间仍基于 TCP/IP，只是应用层协议标准化为 HTTP。

多层 C/S

典型结构：



中间件/应用层主要价值：

- 1. 提高可伸缩性和并发能力。
- 2. 承担请求转发和复杂业务逻辑处理。
- 3. 隔离 Web 服务器与企业数据库，增强数据安全。

3. MVC 模式

MVC 是多层 C/S 的一种标准化抽象。

组成	含义	J2EE 场景对应
----	----	-----------

Model	数据和业务逻辑	应用逻辑、持久化
View	展示层	浏览器/JSP/页面
Controller	控制器	Web 服务器/控制逻辑

MVC 的核心不是“三个英文单词”，而是**表示层与模型层分离，控制器负责协调视图和模型**。它可降低界面变化对业务逻辑的影响，提高**可维护性**。

易错提醒：**MVC** 可以落在**三层**或**四层**体系中，不等于固定三台服务器，也不等于只能用于 Java。

4. SOA 模式

SOA 面向服务架构用于多个独立**应用系统**之间的通信、调用和集成。在 **SOA** 中，一个**多层 C/S** 应用整体可被视为对外提供完整功能的服务。应用之间一般通过消息、**中间件**、交易**中间件**、消息**中间件**等完成通信。

概念	考试理解
服务	对外提供完整业务能力的独立应用或功能集合
SOA 本质	消息机制或远程过程调用
Web Service	SOA 在 Web 应用之间的典型实现
SOAP/WSDL	Web Service 常见协议/描述方式

易错提醒：**SOA** 中的“服务”不是操作系统后台 daemon，而是具备独立业务功能的服务单元。

5. 企业数据交换总线

企业数据交换总线是不同企业应用之间进行信息交换的公共通道，常见于银行、电信等**信息化**程度较高的行业。它本质上可理解为**连接器**软件系统，可基于消息**中间件**、交易**中间件**或 **CORBA/IIOP** 构建，负责数据、请求、响应的接收和分发。

考试判断：

场景	优先关键词
多个 异构系统 之间交换数据	数据交换总线
实时交易路由和请求分发	总线/连接器/中间件
可靠大数据量传输	通常需单独设计，不宜简单等同于实时交易总线

五、企业信息系统总体框架

企业 **ISA** 不能用单一模型描述，应是多维、分层、集成化模型。总体框架由四部分组成：**战略系统、业务系统、应用系统、信息基础设施**。

层次	部分	作用
第一层	战略系统	高层决策支持、战略规划约束信息系统建设
第二层	业务系统	业务过程建模、流程优化、BPR
第二层	应用系统	用应用软件实现业务控制，提高运行效率
第三层	信息基础设施	数据、技术、管理支撑平台

1. 战略系统

战略系统包括**高层决策**支持系统和企业战略规划体系。其意义有两个：

1. 信息系统要支持高层管理者决策。
 2. 企业战略规划会影响信息系统建设方向。
- 长期规划较稳定，短期规划更易随环境和企业运行状况调整。

2. 业务系统

业务系统由完成一定业务功能的物质、能量、信息、人组成。企业常见**业务系统**包括生产、销售、采购、人事、会计等。业务过程可继续分解为业务活动，每个活动有角色、数据和执行顺序。

BPR 业务流程重组的本质：以**业务流程**为中心，打破职能部门壁垒，优化或重组现有业务过程，以提升效率、成本、质量、交货期等指标。

3. 应用系统

应用系统指企业信息系统中的应用软件部分，常见包括 TPS、MIS、DSS、ES、OAS、CAD/CAPP/CAM、MRP II 等。

从架构角度，**应用系统**包含两部分：

部分	稳定性	说明
内部功能实现部分	数据相对稳定，算法和控制结构更易变	受功能需求影响大
外部界面部分	相对变化多	受用户界面要求影响大

4. 企业信息基础设施 EII

EII 根据企业当前业务和可预见发展趋势，构建信息设备、通信网络、数据库、系统软件和支持软件等环境。

组成	内容
技术基础设施	计算机、网络、系统软件、支持软件、 数据交换 协议
信息资源设施	数据和信息、交换形式与标准、信息处理方法
管理基础设施	信息系统部门组织、人员分工、管理制度和方法

六、12.3 信息系统架构设计方法

1. TOGAF 概述

TOGAF 是开放式**企业架构**框架标准，为标准、方法论和**企业架构**人员之间的沟通提供一致性保障。教材按 **TOGAF 9** 介绍其六个组件：

组件	含义
架构开发方法 ADM	TOGAF 核心，提供分步开发 企业架构 的方法
ADM 指南和技术	支持应用 ADM 的技术与指南
架构内容框架	交付物、构件块、元模型、制品结构
企业连续体和工具	对架构活动输出分类、存储和 复用
TOGAF 参考模型	TRM、II-RM
架构能力框架	组织、流程、技能、角色、职责

核心思想：
模块化架构、内容框架、扩展指南、**架构风格**灵活性。

2. ADM 架构开发方法

ADM 是本章最重要考点。它详细定义开发**企业架构**所需步骤及其关系，是 **TOGAF** 的核心。**企业架构**资源库为 **ADM** 提供**可复用资产**，**ADM** 各阶段产出的交付物又不断补充和刷新资源库。

ADM 记忆口诀

准需愿业信技机迁治变

口诀	阶段	核心任务
准	准备阶段	组织、原则、框架、工具、治理准备
需	需求管理	识别、存储、管理并反馈需求
愿	A 架构愿景	范围、 约束 、期望、价值主张、工作说明
业	B 业务架构	基线业务、目标业务、 差距分析
信	C 信息系统架构	数据架构 + 应用架构
技	D 技术架构	硬件、软件、通信、平台基础设施
机	E 机会和解决方案	工作包、 过渡架构 、实施策略
迁	F 迁移规划	项目优先级、路线图、迁移计划
治	G 实施治理	架构契约 、合规审查、实施监督
变	H 架构变更管理	持续监控、变更控制、新一轮 ADM

ADM 三类迭代

迭代层次	含义
ADM 整体迭代	一个架构开发循环结束后进入下一轮
多阶段间迭代	如 技术架构 后返回 业务架构 重新校正
阶段内部迭代	在一个阶段内反复完善复杂架构内容

ADM 各阶段抓分表

阶段	一句话考点	常见输出
准备	明确谁做、按什么原则做、用什么框架做	企业架构 组织模型、定制框架、治理框架、架构工作要求书
A 架构愿景	启动一次 ADM 循环，确定范围、 约束 、目标和价值	架构工作说明书、 架构愿景 、沟通计划、能力评估
B 业务架构	描述现行业务和目标业务，做 差距分析	基线/目标 业务架构 、 业务流程 、业务角色、业务数据模型
C 数据架构	定义业务运行所需数据源和数据类型	业务数据模型、逻辑数据模型、数据实体/业务功能矩阵
C 应用架构	定义处理数据并支撑业务运行的 应用系统	基线/目标 应用架构 、应用交互需求、 应用架构

		视图
D 技术架构	把应用组件映射到技术组件	技术平台、网络通信、硬件网络说明、负载分布
E 机会和解决方案	从差距到项目，形成实施思路	工作包、项目清单、 过渡架构 、风险注册表
F 迁移规划	把实施项目排优先级并形成路线图	实施迁移计划、架构路线图、过渡架构、架构契约
G 实施治理	保证实施项目符合架构	签字 架构契约 、合规建议、实施后更新文档
H 架构变更管理	让架构持续适应变化	架构更新、变更请求、新架构工作要求书
需求管理	贯穿所有阶段	更新的架构需求说明、需求影响评估

易错提醒：

1. **需求管理**不是某个线性阶段，而是贯穿 **ADM 全生命周期**。
2. 阶段 C 包含**数据架构**和**应用架构**，两者可顺序开展，也可并行开展。
3. 阶段 E 开始直接关注实施，阶段 F 才形成详细**迁移规划**。
4. 阶段 G 不是开发实现本身，而是架构监督、合规审查和契约治理。
5. 阶段 H 不是简单维护，而是通过变更管理触发新的 **ADM** 循环。

ADM 架构活动范围四维度

维度	需要回答的问题
企业范围或焦点	企业边界在哪里？本次架构聚焦多少范围？
架构领域	是否覆盖业务、数据、应用、技术四个领域？
详述垂直范围	细化到什么层级？与 系统设计 、开发工作的边界在哪里？
时间周期	目标架构 覆盖多长时间？是否需要中间 目标架构 ？

七、12.3.2 信息化总体架构方法

教材引用《“十四五”推进国家政务**信息化**规划》提出的政务**信息化**目标：到 2025 年，推进政务**信息化**工作迈入以数据赋能、协同治理、智慧决策、优质服务为主要特征的“融慧治理”新阶段。2026 备考时，这部分应作为**信息化**背景和政策口径理解，答题仍以教材概念、总体规划方法和工程方法为主。（[国家发展和改革委员会][3]）

1. 信息化的一般概念

信息化是培育和发展以智能化工具为代表的新生产力，并使之造福社会的历史过程。国家**信息化**体系可按六要素记忆：**资源、网络、应用、产业、人才、政策**

要素	含义
开发利用信息资源	数据和信息是核心资产
建设国家信息网络	通信网络与 信息基础设施
推进信息技术应用	在农业、工业、科技、国防和社会生活中应用
发展信息技术和产业	技术研发、装备制造、信息服务
培育 信息化 人才	人才是持续推进基础
制定完善 信息化 政策	制度、标准、法规保障

2. 信息化生产力四方面

方面	内容
信息网络体系	信息资源、信息系统、公用通信网络
信息产业基础	技术研发、装备制造、信息咨询服务
社会运行环境	管理体制、政策法规、制度文化等
效用积累过程	劳动者素质、现代化水平、生活质量提升

3. 信息化建设六大特征

特征	考试解释
易用性	影响推广应用，是用户接受系统的关键
健壮性	支撑并发、大数据量、长期运行性能
平台化、灵活性、扩展性	通过配置、流程、门户、模块等适应变化
安全性	权限、日志、加密、部署平台、安全控制
门户化、整合性	从单系统走向统一入口和系统整合
移动性	移动端接入和移动服务能力

易错提醒：信息化建设不是“买软件”“上系统”，而是战略、业务、组织、流程、数据、技术和管理制度的系统工程。

4. 信息化架构两种模式

模式	关注点	典型场景	风险
数据导向架构	数据对象、主题域、数据质量	数据中心、BI、MDM	容易忽视流程编排
流程导向架构	端到端流程整合、服务编排	SOA、流程平台、企业集成	容易缺乏完整数据模型

高频结论：

MDM 偏数据导向，SOA 偏流程导向；二者必须结合。

SOA 解决服务编排和业务组合，但不能天然解决数据存储和数据质量；MDM 解决主数据集中管理和统一视图，但需要 SOA 提供敏捷分发和路由能力。

5. 信息系统生命周期

记忆口诀：

规分设实运

阶段	核心任务	关键成果
系统规划	环境、目标、现状、必要性、可行性	可行性报告、系统设计任务书
系统分析	回答“做什么”，提出逻辑模型	系统说明书
系统设计	回答“怎么做”，形成物理模型	系统设计说明书
系统实施	设备购置、程序开发、培训、转换、测试	进度报告、测试分析报告
运行维护	维护、评价、修改、效益评估	运行记录、维护评价

易错提醒：系统分析是逻辑设计，系统设计是物理设计；验收时系统说明书也是重要依据。

6. 信息化总体规划三种常用方法

方法	英文	核心思想	优点	不足
关键成功因素法	CSF	从组织目标识别关键成功因素和关键信息	抓重点，目标明确	可能覆盖不够全面
战略目标集转化法	SST	将使命、目标、战略转化为信息系统目标	全面，疏漏少	突出重点不如 CSF
企业系统规划法	BSP	自上而下识别目标、过程、数据；自下而上设计系统	支持企业目标，适应组织变化	目标导引不如 CSF 直接

考试口诀：

CSF 抓关键，SST 转战略，BSP 过数矩阵。

八、12.4 信息系统架构案例分析

1. 价值驱动的体系结构

价值驱动架构强调：系统存在是为了给利益相关方创造价值。架构设计不能只从功能点出发，还要从价值、限制因素和变化驱动因素出发。

价值模型三个驱动因素

因素	含义	例子
价值期望值	对功能、质量、效用的期望	响应时间、安全性、准确性
反作用力	实现期望的困难或阻力	网络带宽、负载、成本、环境限制
变革催化剂	导致期望或限制变化的事件	法规变化、市场变化、业务模式变化

体系结构挑战 = 限制因素使满足价值期望变难。

制定架构策略的流程：

- 1. 识别并排序价值背景。
- 2. 定义效用曲线和期望值优先级。
- 3. 识别反作用力和变革催化剂。
- 4. 找出满足期望最困难的领域。
- 5. 优先处理高价值、高影响、难替代、影响范围大的挑战。

架构策略四个表达维度：

维度	说明
组织	子系统和组件如何组织、部署、连接
操作	组件如何交互、同步/异步、诊断与纠错
可变性	哪些功能可配置、何时配置、分歧点关系

演变	如何支持重大变更并保持稳定性
----	----------------

论文用法：写质量属性权衡、架构选型、遗留系统演化时，可用“价值期望—限制因素—架构挑战—架构策略”作为论证框架。

2. Web 服务在 HL7 上的应用

HL7 是医疗卫生领域的**数据交换**和**互操作**标准，HL7 Web Services Basic Profile 用 Web 服务实现 HL7 消息传输。教材案例重点不在医疗知识，而在“领域模型 + 标准消息 + Web 服务**适配器** + **互操作**架构”。

HL7 关键概念

概念	含义
RIM 参考信息模型	医疗领域公共信息模型框架
消息结构	HL7 应用之间通过消息交换完成交互
Wrapper	封装消息传输和确认所需信息
交互	触发事件引起消息在应用间转移
应用程序角色	应用在交互中承担的发送/接收职责
Storyboard	描述用例、角色、消息类型和交换顺序

参考架构

HL7 Web 服务架构可分为：

层次	职责
业务逻辑	生成/解析 HL7 XML 消息，验证业务规则，处理应用层确认
Web 服务 适配器	封装 SOAP、处理消息分发、通信确认、安全 约束 、消息持久化
Web 服务基础设施	传输、代理、服务端 Stub、网络交互

HL7 Web 服务适配器开发步骤

1. 设计消息和数据类型，生成 XSD。
2. 选择**适配器**模式。
3. 开发 WSDL 服务契约。
4. 生成 Stub 和代理实现。
5. 开发**适配器**业务逻辑。

案例启示：标准化行业消息模型可以降低**互操作**成本；**适配器**可以隔离**业务系统**与通信协议变化。

3. 以服务为中心的企业整合

航空公司案例的本质是**遗留系统**服务化集成。问题背景包括：主机系统、UNIX 系统、.NET 应用、COBOL/PRO*C/Oracle/IMS/TPF 等异构环境并存，既有 Information Hub 难以开放**复用**，事件支持弱，点到点集成耦合强，跨系统业务协作硬编码严重。

服务化整合思路

步骤	重点
业务环境分析	分析 Ramp Coordination 等流程和业务活动
服务建模	识别 可复用服务 ，如 Retrieve Flight BO、Check Spot、Check Loading
IT 环境分析	梳理已有系统、 接口 、协议、数据格式
高层架构设计	引入信息服务、事件服务、流程服务、传输服务
系统开发与治理	通过 ESB 、 BPEL 、服务适配等降低耦合

架构设计启示

问题	架构解法
异构系统 多	用服务封装差异
点到点耦合 强	用 ESB 和事件服务降低耦合
流程变化频繁	用流程服务和编排提高灵活性
业务活动重复	抽象 可复用服务
遗留系统 难替换	通过 适配器 和平滑演进保护投资

第 13 章 层次式架构设计理论与实践

一、本章考试定位

第 13 章不是单纯背“**分层架构**”，而是考你能否把一个复杂系统按职责拆开，并说明每层的作用、**接口**、耦合关系、数据访问方式、**事务控制**和实际案例落地。

考试题型	常见考法	本章对应能力
综合知识	MVC/MVP/MVVM、DAO、DTO、ORM、事务 ACID、连接池、XML Schema、物联网三层结构	概念辨析、术语识别
案例分析	给出电商、管理系统、 物联网 系统，要求划分层次、指出设计问题、改进 数据访问层 或 业务层	架构诊断、方案改造
论文	分层架构 设计、 数据访问层 设计、 物联网 系统架构、电子商务系统架构	架构论证、质量属性权衡

本章主线可以压缩为：

表现层负责交互 → **中间层**封装业务 → **数据访问层**屏蔽数据源 → **数据层**保存数据 → 通过**接口**、工厂、**事务**、**连接池**、**ORM**降低耦合并提升可维护性。

二、13.1 层次式体系结构概述

1. 核心概念

层次式架构是将系统组织为若干层，每一层为上层提供服务，同时作为下层的客户。常见结构为：

表现层 / 展示层 → 中间层 / 业务层 → 数据访问层 / 持久层 → 数据层

考试中看到“前端界面、**业务逻辑**、**DAO**、**数据库**”分离，基本就要联想到**层次式架构**或三层/**N 层架构**。

2. 层次式架构的价值

价值	考试化理解
关注分离	每层只处理本层职责，避免界面代码、业务代码、SQL 代码混杂
降低耦合	上层通过 接口 调用下层，不直接依赖具体实现
提高复用	同一 业务层 可被不同 表现层 复用，同一 数据访问接口 可适配不同 数据库
便于维护	修改某层实现时，只要 接口 不变，对其他层影响较小
支撑系统级复用	形成可传递的架构经验，适合相似 质量属性 和功能需求的系统

3. 两个高频易错点

第一，**污水池反模式**。请求只是机械穿过多个层，每层几乎不处理**业务逻辑**，导致分层形式存在但价值不足。若大量请求只是“**表现层**→**业务层**→**DAO**→**数据库**”的空转，应考虑开放部分层或简化调用路径。

第二，**过度分层导致系统庞大**。分层不是越多越好。层太多会带来部署复杂、性能下降、代码膨胀、可靠性降低等问题。考试中若题干强调“小系统、简单 CRUD、过度抽象、性能下降”，通常要指出**过度分层**或不必要的分布式部署。

三、13.2 表现层框架设计

1. MVC、MVP、MVVM 对比

模式	结构	核心交互	适用场景	易错提醒
MVC	Model、View、Controller	Controller 接收请求，调用 Model 处理，再选择 View 显示	Web 应用、传统 表现层 分离	View 可查询 Model 状态，不能把 业务逻辑 写进 View
MVP	Model、View、Presenter	View 不直接访问 Model，所有交互集中到 Presenter	需要强化测试、降低 View 与 Model 耦合的场景	Presenter 依赖抽象 View 接口，便于单元测试
MVVM	Model、View、ViewModel	View 与 Model 通过 ViewModel 和 DataBinding 双向绑定	数据驱动、界面状态频繁变化的应用	双向绑定方便但问题定位较难

2. MVC 考试重点

MVC 把输入、处理、输出拆成三个部分：

- Model：业务数据和**业务逻辑**。
- View：显示数据、接收输入，不承担实际业务处理。
- Controller：接收**用户请求**，解释输入，调用模型，并选择视图返回结果。

答题模板：

答题模板： 该系统**表现层**可采用 MVC 模式。**Controller** 负责接收请求和**流程控制**，**Model** 封装业务数据与**业务逻辑**，**View** 负责**结果展示**。这样可实现业务处理与界面显示分离，提高**可维护性**、**可扩展性**和界面**复用**能力。

3. MVP 考试重点

MVP 从 MVC 演变而来，核心是进一步限制 View 与 Model 的直接交互：

View → Presenter → Model

优点：

- Model 与 View 完全分离。
- 所有交互集中在 **Presenter**，逻辑更清晰。
- Presenter 可**服务**多个 View。
- UI 处理逻辑可脱离真实界面做单元测试。

易错点：MVP 不是把 **Controller** 简单改名为 **Presenter**，而是强调 **View** 通过**接口**与 **Presenter** 协作，**View** 不直接读取 **Model**。

4. MVVM 考试重点

MVVM 的核心是 **ViewModel** + **DataBinding**:

View ⇌ ViewModel ⇌ Model

ViewModel 负责状态处理、**数据转换**、**命令映射**和**双向绑定**。它适合数据状态变化频繁、界面响应要求高的系统。
易错点: **MVVM** 并不是“**View** 直接绑定**数据库**”，而是 **View** 通过 **ViewModel** 间接与 **Model** 交互。

5. 使用 XML 统一 Web Form 与 Windows Form 外观

教材中强调 **XML** 可作为 **GUI** 描述载体: 控件类型、位置、可见性、事件绑定等可写入 **XML** 节点和属性, 再由不同**解析器**生成不同界面形态。

考试化理解:

内容	作用
XML 描述 GUI	用统一方式保存界面结构和控件属性
解析器	针对 Web、桌面等不同表现技术生成界面
策略模式思想	不直接绑定某一种表现 API, 而是按不同策略解析 XML
价值	一套界面描述, 多种展示实现, 降低界面迁移成本

6. UIP 设计思想

UIP 即 User Interface Process Application Block, 强调把用户界面流程、导航、**状态管理**、**工作流控制**从普通 UI 代码中剥离出来。

组件	职责
User Interface Components	用户可见界面, 负责收集输入、显示结果
User Interface Process Components	用户不可见, 负责导航、 工作流 、状态和视图管理

案例题中若题干出现“**复杂导航**、**跨页面状态**、用户中途离开后恢复任务、**界面流程复用**”, 应联想到 **UIP** 思想。

7. 表现层动态生成设计

基于 **XML** 的界面管理技术包括:

- 界面配置**: 运行前读取**配置文件**, 完成界面**静态定义**。
- 界面动态生成**: 运行时按 **XML 配置**和数据生成界面元素。
- 界面定制**: 运行过程中用户修改**菜单**、**工具栏**、**快捷键**、大小、位置等, 并保存结果。

考试关键词: **界面描述与功能代码分离**、**配置化**、**定制化**、**移植性**、**适应一定程度数据库结构变化**。

四、13.3 中间层架构设计

1. 业务逻辑层组件设计

业务逻辑组件通常包括:

部分	作用
接口	定义业务组件必须实现的方法, 对外暴露稳定 契约
实现类	调用 DAO 等数据访问组件, 完成具体 业务规则
DAO 依赖	业务逻辑 以多个 DAO 为基础, 组合完成业务功能
IoC/DI 配置	由 容器注入 DAO , 降低业务类与具体 DAO 实现的耦合

关键思想: **控制器**面向**业务接口**编程, 不直接依赖具体**业务实现类**。

2. Facade 与业务层解耦

教材中**业务逻辑层**通过统一 **Facade 接口**向 Web 层提供**服务**。这样前台 Web 层只依赖 **Facade**, 不直接依赖大量**业务实现类**。

考试答法:

答题模板: 可在**业务层**设置 **Facade 接口**, 对**表现层**屏蔽复杂业务组件和 **DAO** 组合细节。**表现层**只提交业务参数并接收结果, **业务规则**、**事务**处理和**数据访问**组合由**中间层**完成, 从而降低 Web 层与**业务层**耦合。

3. 工作流设计

工作流是业务流程的部分或全部自动化, 使文档、信息或任务按**规则**在参与者之间流转。**工作流**管理联盟 **WFMC** 参考模型中常考五类**接口**:

接口	名称	作用
Interface 1	过程定义导入/导出 接口	支持过程定义转换, 早期 WPD , 后发展为 XPDL
Interface 2	客户端应用程序 接口	工作流 机与任务表处理器交互, 组织用户任务
Interface 3	应用程序调用 接口	工作流 机直接调用后台应用工具执行活动
Interface 4	工作流 机协作 接口	支持不同厂商 工作流 系统之间任务传递
Interface 5	管理和监视 接口	用户、角色、审查、资源、过程 状态管理

工作流思想的核心价值:

应用逻辑与过程逻辑分离; 不改具体功能代码, 通过修改过程模型改变系统功能。

4. 业务逻辑层实体设计

业务逻辑层实体用于表示业务数据及相关状态, 可作为业务过程的输入/输出参数传递, 但它不直接访问**数据库**, 也不主动启动**事务**。

常见表示方式:

表示方式	优点	缺点/注意点
------	----	--------

表示方式	优点	缺点/注意点
XML	标准、灵活、跨平台互操作好	大量数据、高性能 事务 场景不宜只靠 XML
通用 DataSet	可表示复杂关系、可 序列化 、支持绑定、 排序过滤 、XML 互换	弱类型，编译期检查不足
有类型 DataSet	代码可读性好、使用方便、编译期类型检查	结构变化后维护成本可能上升

易错点：**业务实体不是 DAO**。**业务实体**保存业务数据状态，**DAO** 负责**数据库**访问。

5. 业务逻辑层框架：Domain Model—Service—Control

组成	职责
Domain Model	领域业务对象，包含业务相关属性
Service	业务过程实现单元，通过 接口 和 契约 提供 服务
Control	服务控制器 ，负责 服务 之间的切换和执行顺序 控制

答题关键词：**业务容器**、**服务松耦合**、**接口契约**、**控制与服务分离**、**可复用**、**可扩展**。

五、13.4 数据访问层设计

1. 五种数据访问模式

模式	核心思想	适用/考法
在线访问	每次 数据库 操作通过连接与 数据源 交互	最基础、最常用，但连接占用影响性能
DAO	封装底层数据访问，分离高层 业务逻辑 与 数据源 操作	J2EE 经典模式，常考“ 接口 +实现+工厂+ DTO ”
DTO	跨进程或网络传输数据的 容器 ，不含复杂 业务逻辑	远程调用、分布式系统中减少网络交互
离线数据模式	数据取出后按预定义结构存在本地，操作独立于连接	适合离线操作、 DataSet 、 SDO 等场景
ORM/O-R Mapping	对象与关系 数据库 记录之间映射	减少 SQL 和 CRUD 代码，常与 Hibernate 结合考

2. DAO 模式

典型 DAO 包含：

1. **DAO** 工厂类
2. **DAO** **接口**
3. **DAO** **接口**的具体**实现类**
4. 数据传输对象 **DTO**

考试重点：**DAO** 具体类包含访问特定**数据源**的逻辑，**业务层**应依赖 **DAO 接口**而不是具体**数据库**实现。

3. DTO 模式

DTO 用于跨进程、跨网络传递数据，本身不应包含复杂**业务逻辑**。

两种实现选择：

实现方式	优点	缺点
内置集合对象	简单、通用、代码少	类型弱，编译期不易发现字段错误
自定义类	强类型、可读性好、编译期检查	编码工作量较大

4. 工厂模式在数据访问层应用

目的：屏蔽不同**数据库**访问类差异，使客户端不直接依赖 **SQL Server**、**Oracle**、**OleDb** 等具体实现。

核心结构：

DataAccess 接口 → **AbstractDataAccess 抽象类** → **MSSqlDataAccess / OracleDataAccess / OleDbDataAccess** → **DataAccessFactory 创建对象**

考试答法：

答题模板：**数据访问层**可采用**工厂模式**。先定义统一 **DataAccess 接口**，再为不同**数据库**提供具体实现，最后由 **Factory** 根据**配置**创建对应访问对象。这样当**数据库**类型变化时，只需修改**配置**或**工厂创建**逻辑，**表现层**和**业务层**无须改动，从而提高**可维护性**和**数据库迁移**能力。

易错点：若程序中大量使用某**数据库**专有函数，即使有**工厂模式**，也不能做到真正**数据库**无关。

5. ORM 与 Hibernate

ORM 把对象模型和关系模型之间建立映射，使开发者像操作对象一样操作**数据库**记录。

优点：

- 降低学习和开发成本。
- 减少重复 **CRUD** 代码。
- 降低**数据访问层**出错概率。
- 通过**缓存**机制优化性能。
- 隔离数据存储与**业务层**，便于分层跟踪和优化。

Hibernate 本质上是提供**数据库服务**的对象关系映射中间件，对 **JDBC** 做轻量封装，支持 **Java** 类与数据表映射、**查询**与恢复机制。

易错点：**ORM** 不是不要**数据库**设计，也不是完全不需要 **SQL**；**复杂查询**、**性能优化**、**事务边界**仍要架构师关注。

6. XML Schema

XML Schema 用于描述 **XML** 文档的合法结构、内容和限制，可看作 **XML** 文档的类型约束机制。

对比点	DTD	XML Schema
-----	-----	------------

对比点	DTD	XML Schema
数据类型	较弱	丰富，支持 string、integer、Boolean、time、date 等
扩展能力	较弱	支持 complexType、simpleType、自定义类型
复用	较弱	支持继承、组件化和复用
命名空间	支持较弱	与 XML Namespace 紧密结合
适用性	传统结构约束	更适合复杂 XML 数据描述与验证

7. 事务处理设计

事务是一组操作，要么全部成功提交，要么失败回滚。必须遵循 ACID 原则：

原则	英文	含义
原子性	Atomicity	事务中的操作要么全部成功，要么全部失败
一致性	Consistency	失败时数据恢复到事务执行前一致状态
隔离性	Isolation	事务提交前，对其他事务不可见
持久性	Durability	已提交结果即使故障也应保持正确

J2EE 中常见事务类型：

JDBC 事务：通过 Connection 关闭 auto-commit，手工 commit/rollback。

JTA 事务：通过 UserTransaction 等接口控制事务。

容器管理事务：由应用服务器容器管理事务边界。

易错点：

不宜在程序中混用多种事务模型，例如 JTA 中嵌套 JDBC 事务。

事务应尽可能短，不要跨多个长时间方法或用户交互过程。

事务边界通常应放在业务层，而不是表现层。

8. 连接对象管理设计

数据库连接是影响性能的重要资源。频繁创建和关闭连接会带来较大开销，因此常采用连接池。

连接池基本思想：

- 1. 系统初始化时创建一定数量连接。
- 2. 请求到来时优先分配空闲连接。
- 3. 使用结束后不真正关闭，而是归还连接池。
- 4. 空闲不足时按策略等待、复用或扩展连接。

考试关键词：资源池、连接复用、减少创建销毁开销、性能优化、连接分配与释放策略。

六、13.5 数据架构规划与设计

1. 数据库设计与类设计融合

类和类之间关系的识别是数据模型设计的关键。建模没有唯一正确答案，但有更优模型。好的数据模型不仅考虑当前开发成本，还要考虑系统生命周期内的维护、演化和变化适应能力。

考试化理解：

类设计偏向对象行为与封装。

数据库设计偏向持久化、约束、查询和一致性。

二者需要融合，不能简单把每个类机械映射成一张表，也不能让数据库结构完全绑死对象模型。

2. 数据库设计与 XML 设计融合

XML 文档分两类：

类型	特征	常见用途
数据为中心的 XML 文档	结构规则、内容同构、关注数据本身	Web 数据存储与传输
文档为中心的 XML 文档	结构不规则、内容零散、顺序重要	网页描述、产品介绍、邮件等

XML 文档存储方式：

存储方式	优点	缺点
文件存储	保留原始文本，简单直接	查询结构化数据困难，容量大时管理难
数据库存储	检索快、管理方便、安全性和并发控制较好	需要设计映射和 XML 数据库管理机制

易错点：XML 具备存储、模式、查询语言和编程接口等数据库特征，但在索引、安全、多用户访问、事务管理等方面不能简单等同于成熟 DBMS。高并发、高完整性、高性能系统不宜仅把 XML 文件当数据库使用。

七、13.6 物联网层次架构设计

物联网常用三层架构：

层次	功能	关键技术/对象	类比
感知层	识别对象、采集信息	RFID、二维码、传感器、摄像头、GPS、M2M 终端、传感器网关	皮肤和五官
网络层	传递、处理、存储感知数据	移动通信网、互联网、专网、企业网、云计算平台、感知数据管理	神经中枢和大脑
应用层	与行业需求结合，实现智能化服务	智能交通、智能家居、物流监控、远程抄表、移动支付	社会分工与人机交互

答题模板：

答题模板： 该物联网系统可采用**感知层、网络层、应用层**三层架构。**感知层**通过**传感器/RFID**/终端设备采集物理世界数据；**网络层**通过无线、有线或互联网传输并处理数据，可结合云平台完成存储、**查询**、分析；**应用层**面向具体行业场景提供**监控、查询、控制、预测**和**人机交互服务**。

八、13.7 层次式架构案例分析

1. PetShop 案例

PetShop 体现了**分层架构**逐步成熟的过程。

版本/结构	特点	考试启示
PetShop 2.0	没有明显 数据访问层	性能可能较好，但 业务逻辑 与数据访问 职责混乱
PetShop 3.0	将数据访问逻辑独立成层	职责更清晰， 便于维护 和 数据库迁移
PetShop 4.0	延续分层，引入 缓存、异步 处理、 Membership	在保持分层基础上优化性能和扩展性

PetShop 4.0 关键设计思想：

- 数据访问层**采用**面向接口编程**。
- IDAL 抽象**数据访问接口**，SQLServerDAL 和 OracleDAL 实现该接口。
- DALFactory 负责创建具体 DAL 对象。
- BLL 不直接访问**数据库**，而通过 IDAL 访问**数据层**。
- 对订单插入等变化点，可通过策略接口支持**同步/异步实现**。
- Messaging 模块支持**异步订单处理**，CacheDependency 支持**缓存**。

案例题答法：

答题模板： 原系统若**业务层**直接访问**数据库**，会导致**职责混乱**和**数据库迁移**困难。应将数据访问逻辑抽象为独立 DAL 层，定义 IDAL 接口，由 SQL Server、Oracle 等具体 DAL 实现接口，再通过 DALFactory 创建对象。**业务逻辑层**只依赖 IDAL 接口，从而**降低耦合**，提高可**维护性**和可**移植性**。

2. 电子小票物联网案例

电子小票系统采用**感知层、网络层、应用层**三层**物联网架构**。

层次	在电子小票系统中的体现
感知层	小票智能硬件采集收银机待打印购物小票信息
网络层	通过 Wi-Fi/GPRS 将小票信息传输到 电子小票 云平台，并提供数据处理、存储、设备管理、商家管理、会员管理、商品管理
应用层	通过微信公众平台向顾客推送 电子小票、查询 历史小票和 会员积分 ；向商家提供经营数据分析、精准推荐、畅销品排行

法规与安全提醒：**电子小票**涉及**购物记录、会员积分**、用户识别、**消费偏好**等数据，案例作答时应补充**数据安全、最小必要、访问控制、传输加密、日志审计、授权查询、数据备份**和**异常处置**等措施。

九、真题对应与答题方向

1. 综合知识常见问法

问法	正确抓手
哪种模式将输入、处理、输出分为模型、视图、控制器？	MVC
哪种模式中 View 不直接访问 Model，交互集中在 Presenter？	MVP
哪种模式强调 ViewModel 和双向数据绑定？	MVVM
哪种模式将底层数据访问与高层业务逻辑分离？	DAO
跨进程/网络边界传输数据的对象是什么？	DTO
对象与关系数据库记录之间映射是什么？	ORM
事务四大特性是什么？	ACID
物联网三层结构是什么？	感知层、网络层、应用层

2. 案例分析常见问法

题干场景	应答方向
界面代码中写大量流程控制和状态管理	引入 UIP，分离 UI 组件与 UI 过程组件
系统需同时支持 Web 和桌面客户端	用 XML 描述 GUI，解析成不同表现形式
业务层直接拼 SQL 访问数据库	增加 DAO/DAL 层， 面向接口编程
系统需支持 SQL Server 和 Oracle 切换	使用 工厂模式+数据访问接口
订单处理性能压力大	缓存、异步消息、连接池、事务边界 优化
小票/会员/消费数据上云	按 物联网 三层架构设计，并补充 数据安全 措施

3. 论文可用主题

本章适合支撑以下**论文题**：

- 1. 论**层次式架构**在信息系统中的应用
- 2. 论企业应用系统的**数据访问层**设计
- 3. 论**面向接口编程**在**分层架构**中的应用
- 4. 论电子商务系统的**分层架构**设计
- 5. 论**物联网**系统的层次架构设计

6. 论系统架构中的解耦、复用与可维护性设计

论文结构可套用：

项目背景 → 业务与质量需求 → 层次划分 → 表现层设计 → 业务层设计 → 数据访问层设计 → 数据层/事务/连接池/缓存 → 安全与合规 → 实施效果。

第 14 章 云原生架构设计理论与实践

一、章节定位：本章考什么

本章是“新技术架构 + 云计算 + 微服务 + 容器 + DevOps + 安全治理”的综合章节，最容易在三类题型中出现：

题型	常见考法	备考重点
综合知识	概念辨析、技术特征、架构原则、反模式判断	云原生定义、七大原则、架构模式、容器/K8s/Serverless/服务网格
案例分析	企业上云改造、微服务拆分、容器化部署、弹性扩容、稳定性治理	从“问题—方案—效果”角度答题
论文	云原生架构设计、微服务架构治理、容器平台建设、DevOps 与可观测性	写清业务背景、架构设计、关键技术、风险治理、实施效果

本章不是单纯背名词，重点在于能把“云原生技术”落到系统架构设计场景中：为什么改造、怎么拆分、怎么部署、怎么治理、怎么保证弹性与稳定性。

二、云原生架构产生背景

1. 云原生的基本含义

云原生 Cloud Native 可以拆成两层理解：

组成	含义	考试记忆
Cloud	应用运行在云端，依托云基础设施和云服务	不是简单“把系统搬到云上”
Native	应用从设计之初就面向云环境构建	能充分利用弹性、分布式、自动化、按需服务

考点句：云原生不是传统系统的简单云迁移，而是让应用在设计、开发、部署、运维全生命周期中充分利用云平台的弹性、分布式、自助、按需和自动化能力。

2. 企业为什么需要云原生

传统问题	云原生解决方向
烟囱式 IT 建设，资源重复投入	统一 IaaS/PaaS 能力，资源和服务复用
开发、测试、运维割裂	DevOps、CI/CD、自动化交付
瀑布流程周期长	敏捷迭代、持续交付
高峰流量难支撑	弹性伸缩、快速扩容
运维依赖人工经验	自动恢复、自动调度、可观测性
新老应用并存	云原生改造、应用网关、服务网格、灰度迁移

易错点：云原生的价值不只是“降本”，还包括业务敏捷、弹性承载、稳定性提升、创新能力提升、数字化转型支撑。

三、云原生架构定义与核心内涵

1. 教材口径定义

云原生架构是基于云原生技术的一组架构原则和设计模式集合，目标是把云应用中的大量非业务代码从业务系统中剥离出来，让云设施接管弹性、韧性、安全、可观测性、灰度发布等非功能特性，使业务应用具备轻量、敏捷、高度自动化的特点。

2. 云原生代码结构三部分

组成	内容	价值判断
业务代码	实现业务逻辑，如订单、支付、用户、库存	核心价值
三方软件	业务库、基础库、中间件依赖	辅助价值

组成	内容	价值判断
非功能性代码	高可用、安全、监控、灰度、容灾等	应尽量由云设施或平台接管

高频判断：云原生的核心不是把所有代码都重写，而是让**业务代码**更聚焦业务价值，让**非功能能力**由云平台、PaaS、中间件、**服务网格**等基础设施承担。

3. 云原生带来的三大变化

变化	解释	案例化表达
代码结构变化	存储、网络、线程等复杂能力逐渐 服务化	对象存储 、块存储、数据库、缓存由云服务提供
非功能特性大量委托	高可用、容灾、安全、 灰度 、 可观测 性由平台承担	虚拟机热迁移、 容器 自愈、云服务高可用
软件交付高度自动化	容器 镜像、声明式配置、面向终态交付	通过 CI/CD 、IaC、GitOps、 Operator 自动交付

真题对应方向：题干给出“开发环境与生产环境不一致、交付依赖手工文档、上线窗口压力大”，通常应答 **容器化打包 + 自动化交付 + 声明式配置 + CI/CD**。

四、云原生架构七大原则

原则	核心含义	考点关键词	易错提醒
服务化原则	按业务能力拆分服务，服务内部高内聚、服务间低耦合	微服务 、小服务、接口契约、服务治理	不是越细越好
弹性原则	部署规模随业务量自动伸缩	自动扩缩容、按需资源、降低闲置成本	弹性 不等于只买更大服务器
可观测原则	通过日志、 链路追踪 、度量掌握系统运行状态	Logging、Tracing、Metrics、SLO、traceid/spanid	可观测性 强于传统监控
韧性原则	依赖异常时仍尽量持续提供服务	限流、降级、熔断 、反压、重试、隔舱、容灾	韧性 关注故障抵御能力
全过程自动化原则	交付、部署、配置、运维尽量 自动化	IaC、GitOps、OAM、 Operator 、 CI/CD	没有 自动化的微服务 会放大复杂度
零信任原则	默认不信任 任何网络内外实体，以 身份为中心 访问控制	Identity、认证、授权、 mTLS、RBAC	IP、主机、地理位置不能天然可信
架构持续演进原则	架构应可迭代、可治理、可迁移	增量迁移、 灰度 、适配器、 服务网格 、数据迁移	存量系统迁移要控制成本与 风险

记忆口诀：服弹观韧，自零演。即：**服务化、弹性、可观测、韧性、自动化、零信任、持续演进**。

五、主要云原生架构模式

1. 服务化架构模式

服务化架构是云时代构建云原生应用的标准模式。它以应用模块为粒度拆分系统，以接口契约定义服务关系，以 **HTTP、gRPC** 等标准协议实现互联互通，并结合 **DDD、TDD、容器化**部署提升代码质量和迭代速度。

内容	要点
典型形式	微服务 、小服务 MiniService
核心收益	模块独立部署、独立扩缩容、独立升级
适用场景	大型系统、多团队协作、不同模块生命周期不同
风险	服务数量过多、治理能力不足、接口调用开销增加

易错点：小服务不是“单体”，也不是无限拆分的**微服务**，而是一组关系密切、共享数据、避免过细粒度调用损耗的服务组合。

2. Mesh 化架构模式

Mesh 化架构把 **RPC**、缓存、消息、数据库访问等中间件能力从业务进程中剥离出来，通过独立的 **Mesh** 进程完成流量控制、安全策略、微隔离、**熔断**、**限流**、**降级**、重试等能力。

对比项	传统 SDK 模式	Mesh 化模式
治理逻辑位置	业务进程内 SDK	独立 Sidecar/Mesh 进程
升级影响	SDK 升级可能影响 业务代码	中间件升级对业务更透明

对比项	传统 SDK 模式	Mesh 化模式
治理能力	分散在各应用中	平台统一治理
典型能力	RPC 调用、缓存访问	流量治理、安全、 可观测 、灰度

考点句：Mesh 的核心价值是 **非侵入式服务治理**，让**业务代码**更薄，让流量控制、安全和**可观测**能力下沉到基础设施。

3. Serverless 模式

Serverless 把“部署和运行环境管理”从开发运维中收走。开发者只关注业务逻辑，云平台负责调度、运行、关闭或复用业务进程。

适合场景	不适合场景
事件驱动数据计算	强状态同步应用
计算时间短的请求/响应应用	长时间后台密集计算任务
没有复杂互调的周期任务	频繁外部 I/O、强依赖低时延场景

易错点：Serverless 不是“没有服务器”，而是 **服务器管理对用户不可见**；也不是所有应用都适合 Serverless。

4. 存储计算分离模式

在云环境中，推荐把 session、结构化数据、非结构化数据等交由云服务或分布式存储管理，使应用尽量无状态，从而获得更好的**弹性**、可用性和分区容错能力。

设计点	说明
无状态应用	更容易水平扩展和迁移
有状态数据	下沉到数据库、缓存、 对象存储 、分布式存储
特殊场景	会话数据过大或远端访问影响性能时，可用日志 + 快照/检查点恢复

CAP 对应：有状态应用更容易受到一致性 C 的约束；无状态应用不直接维护数据一致性，更容易获得可用性 A 和分区容错性 P。

5. 分布式事务模式

模式	一致性	性能	侵入性	适用判断
XA	强一致	较差	中等	强一致 要求高、性能压力小
BASE/消息最终一致	最终一致	高	中等	异步、解耦、可容忍短暂不一致
TCC	应用层控制事务	较高	强	业务可显式 Try/Confirm/Cancel
SAGA	正向事务 + 补偿事务	较高	强	长事务、可补偿业务
Seata AT	自动回滚、代码量低	高	低	适合受 AT 模式约束的场景

易错点：XA **强一致但性能差**；BASE 性能高但通用性有限；TCC/SAGA 灵活但开发维护成本高；Seata AT **不是万能方案**，有适用场景限制。

6. 可观测架构

可观测架构由三类数据构成：

类型	作用	关键词
Logging	记录详细运行信息	debug、warning、error、fatal
Tracing	追踪一次请求的完整调用链	traceid 、 spanid 、 链路追踪
Metrics	量化系统状态	QPS 、延迟、错误率、容量、并发度

考试表达：**可观测**性的目标是度量服务 **SLO**，并通过监控、分析、定位和优化提升 **SLA**。

7. 事件驱动架构 EDA

事件驱动架构本质是应用或组件之间的集成模式。事件具有 schema，可校验有效性，并可通过 QoS 机制保障处理可靠性。

场景	作用
服务解耦	上游不直接依赖下游同步处理
增强韧性	下游失败不直接拖垮上游
CQRS	命令与查询分离

场景	作用
Event Sourcing	通过事件回放重建状态
数据变化通知	订单完成后通知积分、信用、库存等服务
开放式接口	事件发布者不关心订阅者
事件流处理	Kafka 日志处理、IoT 数据处理

六、典型云原生架构反模式

反模式	问题	正确处理
庞大单体应用	代码耦合、责任不清、整体扩容、局部故障影响全局	按业务域拆分，明确边界和接口
单体硬拆微服务	小系统强拆导致调用复杂、性能下降、组织能力跟不上	适度拆分，按业务边界而非技术冲动拆分
缺乏自动化的微服务	服务数量上升后，测试、发布、运维工作量暴涨	建设 CI/CD、自动化测试、自动发布、监控告警、自动回滚

高频易错句：**微服务不是目的**，提升业务迭代效率、**弹性**、稳定性和治理能力才是目的；**没有自动化和治理能力的微服务**会把复杂度从单体内部转移到分布式环境中。

七、云原生相关技术

7.1 容器技术

1. 容器的价值

容器是标准化软件单元，把应用及其依赖一起打包，使应用在不同计算环境中保持一致运行。

对比项	传统部署	虚拟化部署	容器部署
隔离粒度	进程/应用弱隔离	虚拟机级隔离	操作系统级隔离
启动速度	慢	较慢	秒级启动
资源开销	依赖环境	较高	较低
迁移能力	差	中等	强
典型价值	简单直接	硬件利用率提升	敏捷、 弹性 、可移植

Docker 的关键贡献是提出镜像打包规范，解耦应用与运行环境；**Kubernetes** 进一步成为**容器编排**和分布式**资源调度**的事实标准。

2. Kubernetes 核心能力

能力	说明
资源调度	根据 CPU、内存、 GPU 等资源需求选择节点
应用部署与管理	支持发布、回滚、配置管理、存储卷编排
自动修复	宿主机或应用异常时迁移、自愈
服务发现与负载均衡	Service、 DNS 、 负载均衡 机制
弹性伸缩	根据 CPU 利用率、响应时间等指标自动扩缩容

3. Kubernetes 控制平面与关键机制

组件/机制	作用
APIServer	统一 API 入口
Controller	维持期望状态
Scheduler	负责调度
etcd	保存集群状态
声明式 API	描述期望状态，而非手写执行步骤
CRD/Operator	扩展 Kubernetes 能力
CNI/CSI	屏蔽网络与存储差异
LoadBalance Service	屏蔽底层 负载均衡 差异

真题对应方向：题干出现“多云/混合云、环境一致、自动扩缩容、自动修复、跨基础设施迁移”，优先联想到 **Kubernetes + 容器镜像 + 声明式 API + CNI/CSI + 自动化运维**。

7.2 云原生微服务

1. 微服务发展逻辑

单体应用早期开发简单，但随着业务复杂度和团队规模增长，会出现集中研发、集中测试、集中发布、扩展困难、稳定性不足等问题。**微服务**通过把单体拆成多个松耦合子应用，提高研发、测试、部署和扩展效率。

2. 微服务设计约束

约束层次	关键要求	考点
微服务个体约束	按业务域划分，单一职责，团队小型化	SRP、业务边界、两披萨团队
服务间横向关系	可发现、可交互、面向失败设计	注册中心、REST/gRPC、IDL、熔断、限流、隔仓
服务与数据层纵向关系	数据私有、通过 API 访问	DSS、CQRS、计算存储分离
全局分布式约束	自动化、可观测、发布治理	CI/CD、蓝绿、金丝雀、链路追踪

3. 微服务常见技术

技术	重点记忆
Apache Dubbo	高性能 RPC、服务注册发现、负载均衡、服务治理
Spring Cloud	配置管理、服务发现、断路器、路由、总线等
Eclipse MicroProfile	Java 微服务规范、指标、健康检查、容错、跟踪
Tars	腾讯开源微服务框架，多语言、高性能、服务治理
SOFAStack/MOSN	金融级分布式架构，MOSN 类似 Envoy，可与 Istio 集成
DAPR	分布式应用运行时，支持云和边缘上的有状态/无状态微服务

7.3 无服务器技术 Serverless

1. Serverless 特征

特征	解释
全托管计算服务	用户只写代码或提交镜像，不关注服务器运维
通用性	可结合 BaaS API 支撑多类云应用
自动弹性伸缩	不必提前做固定容量规划
按量计费	避免为空闲资源付费

FaaS 是 Serverless 的典型形态，函数由事件触发执行，如**对象存储**上传事件、消息队列事件等。

2. Serverless 技术难点

难点	说明
编程模式变化	事件驱动对架构、开发习惯和交付流程有影响
生态成熟度	企业研发流程需要适配
冷启动	细粒度函数运行可能导致首次响应延迟
数据库连接成本	按需建立连接可能影响性能
调度复杂	需要精准、实时、智能的弹性调度
多租户隔离	需要流量隔离、安全隔离、资源隔离

易错点：Serverless 与容器并不冲突，教材中特别强调了 Serverless 与容器技术融合，例如基于容器镜像部署到 Serverless 环境，获得免运维、弹性伸缩、按量计费能力。

7.4 服务网格 Service Mesh

1. 服务网格核心特点

服务网格是在微服务架构之上发展出来的技术，把连接、安全、流量控制、可观测性等通用能力下沉到平台基础设施，实现应用与基础设施解耦。

架构组成	说明
数据平面	由代理拦截服务流量，如 Envoy
控制平面	统一配置流量治理、安全、观测策略
Sidecar 代理	与业务服务共同部署，代理服务间通信
典型能力	服务发现、熔断、限流、路由、mTLS、可观测

2. Istio 相关考点

组件	作用
Pilot	服务发现、流量管理
Mixer	访问控制、可观测性
Citadel	身份认证、流量加密
Envoy	数据平面代理

3. 服务网格技术趋势

方向	说明
xDS	数据平面与控制平面通信的重要协议
SMI	Service Mesh Interface，抽象服务网格能力
UDPA	基于 xDS 的通用数据平面 API
Wasm 插件	提升数据平面扩展性与隔离性
零信任结合	mTLS、Identity、RBAC、微隔离

真题对应方向：题干中出现“业务代码不改造，但要统一限流、熔断、灰度、链路追踪、服务间加密”，优先答 服务网格 /Sidecar/Envoy/Istio。

八、案例分析提炼

1. 旅行公司云原生改造

问题	方案	效果
技术体系不统一、资源利用率低、流量波动大	私有云平台、IaaS/PaaS 封装、容器、服务网格、弹性计算、跨云灾备、DevOps	资源使用减少、峰值流量支撑能力提升、稳定性提升

答题角度：资源池化 + 弹性调度 + 自动化恢复 + 跨地域/跨云灾备。

2. 汽车公司数字化转型

问题	方案	效果
互联网业务增多、敏态与稳态业务并存、交付周期长	容器云平台、混合云、微服务治理、业务中台、DevOps	应用交付周期缩短，支撑高可用、高扩展、高安全互联网业务

答题角度：敏稳分离 + 混合云 + 微服务拆分 + DevOps。

3. 快递公司核心系统云原生改造

问题	方案	效果
IDC 架构、IOE 依赖、交付周期长、大促保障难	Kubernetes、云原生数据库、应用容器化、微服务改造、ACK + 神龙 + 云数据库	成本降低、稳定性提升、交付效率提升、业务创新能力增强

答题角度：数据库拆分 OLTP/OLAP + 容器化解决环境一致性 + 微服务降低维护复杂度。

4. 电商业务云原生改造

问题	方案	效果
大促频繁、问题定位慢、容量评估不足、扩缩容频繁	ACK、Spring Cloud Alibaba、PTS、AHAS、链路追踪、限流降级	高可用、大促保障常态化、容量评估可量化、资源成本降低

答题角度：压测 + 容量评估 + 链路追踪 + 限流降级 + 弹性扩容。

5. 体育用品公司业务中台

问题	方案	效果
商业套件僵化、烟囱系统多、数据割裂、高库存高缺货	全渠道业务中台、共享服务中心、Kubernetes、EDAS、MQ、ARMS、PTS	数据融通、库存共享、业务复用、技术驱动业务创新

答题角度：DDD + 共享服务中心 + 业务中台 + 全渠道数据融合。

九、公式与指标熟练

本章计算不是主线，但案例题和论文中常用以下指标表达。

指标/公式	用法
可用性 = 正常服务时间 ÷ 总服务时间 × 100%	用于说明 SLA、系统稳定性
可用性 ≈ MTBF ÷ (MTBF + MTTR)	韧性设计中，提高 MTBF、降低 MTTR
资源节省率 = (改造前资源量 - 改造后资源量) ÷ 改造前资源量 × 100%	案例中论证降本效果
实例数 ≈ 峰值请求量 ÷ 单实例承载量 × 安全系数	弹性扩容和容量评估
错误率 = 失败请求数 ÷ 总请求数 × 100%	SLO/SLA 监控指标
平均响应时间/TP99	性能压测与大促保障指标

考试提醒：本章如果出现“公式”，多数不是纯数学计算，而是让你用指标证明架构设计有效，例如资源利用率、扩容能力、响应时间、错误率、可用性、恢复时间。

附：本章常用公式

可用性

$$A = \frac{T_{\text{normal}}}{T_{\text{total}}} \times 100\%$$

基于 MTBF 与 MTTR 的可用性近似

$$A \approx \frac{MTBF}{MTBF + MTTR}$$

资源节省率

$$R = \frac{Resource_{\text{before}} - Resource_{\text{after}}}{Resource_{\text{before}}} \times 100\%$$

实例数估算

$$N \approx \frac{Q_{\text{peak}}}{C_{\text{single}}} \times S_{\text{factor}}$$

错误率

$$E = \frac{Request_{\text{failed}}}{Request_{\text{total}}} \times 100\%$$

第 15 章 面向服务架构设计理论与实践

1. 本章考试定位

第 15 章属于系统架构设计师中企业应用集成、分布式架构、服务化架构、微服务架构的核心章节。考试中常见考法不是单纯背定义，而是让考生判断：一个企业存在多个异构系统、遗留系统、信息孤岛、业务流程变化快、跨部门协同困难时，如何用 SOA、ESB、服务注册、BPEL、Web Service、QoS 或微服务方式解决。

一句话总纲：SOA 是用标准化服务封装业务能力，通过接口、契约、注册、发现、编排和治理，实现异构系统松耦合集成与业务流程灵活重组的架构思想。

2. SOA 基础概念

考点	应掌握内容	易错提醒
服务	服务是由 服务提供者 提供、用于满足 服务 使用者请求的业务单元；在 SOA 中泛指系统对外提供的功能集。	服务 不是“某个类”或“某个 接口 方法”，而是可被业务调用的能力。
SOA 应用角度	SOA 是一种 应用框架 ，将业务应用划分为独立 业务功能 和流程，使 服务 可构建、部署、整合，提高 业务灵活性 。	不要只写“ Web Service 技术”， SOA 首先是架构思想。
SOA 软件原理角度	SOA 是 组件模型 ，把应用程序不同功能单元通过明确定义的 接口 和 契约 联系起来； 接口 应独立于硬件、操作系统和语言。	架构师答题应优先使用“ 组件模型 、 接口 、 契约 、 平台无关 、 松耦合 ”等关键词。
BPEL	面向 Web 服务 的 业务流程执行语言 ，用于 组合 、 编排 、 协调 多个 Web 服务 ，形成 复合服务 或 业务流程 。	BPEL 不是通信协议，而是 业务流程编排 语言。

教材明确从应用角度和软件原理角度分别定义 **SOA**，并强调架构师更应掌握软件原理角度：**服务**之间依赖**接口**和**契约**，而不是依赖具体实现平台。

3. SOA 发展脉络

阶段	关键技术/规范	考试抓手
萌芽阶段	XML 、 XSD 、 XSLT	XML 是 SOA 的数据交换基石； XSD 保证消息结构有效， XSLT 支持不同数据表达 转换 。
标准化阶段	SOAP 、 WSDL 、 UDDI	Web Service “三剑客”： SOAP 负责连接/消息， WSDL 负责描述， UDDI 负责注册发现。
成熟应用阶段	SCA 、 SDO 、 WS-Policy	SCA/SDO 支撑 SOA 编程模型， WS-Policy 支撑 策略 与安全交互。

国内外 **SOA** 建设差异也可考：美国更偏向“提取、包装**已有系统**功能形成**标准服务**”；国内许多行业则既要构建**服务**型系统，也要整合已有生产型系统。教材还指出 **SOA** 向**微服务**化发展，**微服务**更细粒度、更独立、更去中心化。

4. SOA 与微服务对比

维度	SOA	微服务
服务粒度	相对较粗，强调企业级 服务 整合。	更细，围绕单一业务能力拆分。
核心连接方式	常以 ESB 为核心连接子系统。	去中心化，弱化或去除重型 ESB 。
部署方式	服务 间可能依赖较重，部署复杂度较高。	服务 独立开发、测试、部署、扩展。
接口方式	常见 Web Service 、 SOAP/WSDL 等。	常见 RESTful API 、 消息队列 等。
数据管理	更强调企业级集成。	倾向 服务 独立、数据源独立。
适用场景	企业 遗留系统 整合、跨系统 业务流程编排 。	互联网 高并发 、 快速迭代 、 团队自治 、 弹性扩展 。

易错点：**微服务**不是 **SOA** 的完全替代，而是 **SOA** 思想在更细粒度、更去中心化场景下的发展。教材图 15-1 通过 **SOA** 架构与**微服务**架构图对比，突出 **SOA** 中 **ESB** 的集中集成作用，以及**微服务**架构中**服务**个体独立、拆分粒度更小的特点。

5. SOA 参考架构：六类服务

IBM WebSphere 业务集成参考架构是教材中的重点图示。可按“业务逻辑、控制、连接、创新优化、开发、IT 管理”六类记忆。

架构层/服务	核心作用	典型内容
业务逻辑服务	实现业务逻辑和业务能力。	业务应用 服务 、伙伴 服务 、应用与信息资产。
控制服务	实现人、流程、信息的集成。	信息 服务 、流程 服务 、交互 服务 。
连接服务	提供 服务 之间连接性。	ESB ，负责连接、 路由 、 转换 、 协议适配 。
业务创新和优化 服务	监控业务性能并优化流程。	事件框架、采集 服务 、监控 服务 、KPI。
开发服务	覆盖软件开发生命周期。	建模、设计、实现、测试 服务 。
IT 服务管理	支撑安全、高效、稳定运行。	安全目录、SSO、系统管理、虚拟化。

5.1 ESB 企业服务总线

ESB 是本章**高频**考点。它是架构模式，不是某个具体产品。其作用是通过总线模式简化应用集成拓扑，使**服务**请求者不需要关心**服务提供者**的位置、协议和实现技术。

ESB 核心能力：

- 1. 服务元数据描述与**服务注册**管理。
- 2. 消息传递、数据**转换**、**协议转换**。
- 3. **服务发现**、**路由**、匹配、选择。
- 4. 支持同步、异步等消息模式。
- 5. 安全、**QoS**、**可管理性**、**负载均衡**。
- 6. 提供**位置透明**和**协议透明**。

案例答题句：可引入 **ESB** 作为企业级**服务**连接层，将**遗留系统**、新建系统、数据**资源**和外部伙伴系统封装为**标准服务**，通过注册、**路由**、**转换**、**协议适配**和 **QoS** 管理，实现**异构系统松耦合**集成。

教材明确指出，**ESB** 通过总线结构简化应用之间的集成拓扑，并以基于标准的**连接服务**支撑**松耦合**、动态交互；在 **SOA** 参考架构中，**ESB** 的消息**路由**、注册命名、多种消息范型、多协议、多格式**转换**、**日志监控**等功能均属于重点。

6. Web Service 主要协议与 REST

协议/规范	记忆关键词	作用
UDDI	注册、发现、集成。	让 服务提供者 发布 服务 ，让 服务 请求者发现 服务 。
WSDL	描述 服务 。	描述 服务 做什么、如何访问、 服务 在哪里。
SOAP	XML 消息协议。	在 分布式 环境中交换结构化信息。
REST	资源 、 表述 、 状态转移 、 超链接 。	以 资源 为中心，通过 URI 和 HTTP 方法操作 资源 状态。

6.1 WSDL 必背结构

WSDL 可描述 **Web 服务**三个基本属性：

- 1. **服务**做什么：**服务**提供的操作或方法。
- 2. 如何访问**服务**：交互数据格式和必要协议。
- 3. **服务**位于何处：协议相关地址，如 URL。

WSDL 元素记忆顺序：types → message → operation → portType → binding → port → service

元素	作用
types	定义 Web 服务 使用的数据类型集合。
message	定义通信消息的数据结构。
operation	描述 服务 支持的操作。
portType	操作集合，抽象 服务 访问点类型。
binding	将抽象 接口 绑定到具体协议和数据格式。
port	协议/数据格式绑定 + 具体访问地址。
service	端口集合，即相关 服务 访问点集合。

6.2 SOAP 四部分

部分	作用
Envelope	描述消息内容、发送方、接收方和处理框架。
Encoding Rules	表示应用程序使用的数据类型实例。
RPC Representation	远程过程调用和应答约定。
Binding	使用底层协议交换信息。

6.3 REST 四要素

要素	含义
Resource	一切暴露给客户端的对象都可视为 资源 。

Representation	资源在某一时刻的状态表现，如 JSON、XML、HTML。
State Transfer	借助 GET、POST、DELETE 等 HTTP 方法转移资源状态。
Hyperlink	通过相关 URI 暴露后续可访问资源。

易错点：REST 是架构风格，不是具体协议；RESTful 是遵循 REST 约束的一类应用或服务架构。教材在主要协议和规范部分将 UDDI、WSDL、SOAP 列为基本 Web 服务协议，并单独说明 REST 的资源、表述、状态转移和超链接思想。

7. SOA 设计标准要求与 QoS

标准要求	内容	考试关键词
文档标准化	服务应具有平台独立的自描述 XML 文档。	WSDL。
通信协议标准	服务之间通过消息通信，消息通常用 XML Schema 定义。	XML、XSD。
统一登记与集成	服务通过 Registry 维护，应用在注册处查找并调用服务。	UDDI、Registry。
服务质量 QoS	安全、可靠通信、调用策略、事务、管理。	可靠性、安全性、策略、控制、管理。

7.1 QoS 重点标准

QoS 类型	典型规范/机制	说明
可靠性	WS-Reliability、WS-Reliable Messaging	支持仅一次、最多一次、重复消息过滤、保证传送等。
安全性	Web 服务安全规范、SAML	认证交换、消息完整性、消息保密。
策略	WS-Policy	描述服务消费者与提供者之间的策略断言。
控制	BPEL4WS / WSBPEL	控制一组离散服务组成的业务流程。
管理	WSDM	支持分布式 Web 服务管理。
协调与事务	WS-Coordination、WS-Transaction	支持多服务协作与事务处理。

案例答题句：关键业务系统不能只依赖 SOAP、WSDL、UDDI 等基本规范，还应设计 QoS，包括可靠消息、安全认证授权、策略声明、事务协调、日志监控和服务管理。

8. SOA 的作用：打破信息孤岛

企业中常见问题是不同系统由不同时间、工具、平台建设，导致业务请求难以统一调用，形成“信息孤岛”。SOA 的解法是：

1. 把应用和资源转换成服务。
2. 把服务标准化，形成资源共享。

传统 EAI 点对点集成需要大量“翻译器”，系统数量越多，连接复杂度越高。可用公式记忆：

$$\text{EAI 点对点连接数} = \frac{n(n-1)}{2}$$

例如 5 个应用互通需要 10 个连接。SOA 通过服务化和标准化减少点对点耦合，保护已有 IT 投资，并提升业务变化响应速度。教材在 SOA 作用部分用 EAI Server “翻译”数量增长说明传统方式难以长期解决信息孤岛问题。

9. SOA 八大设计原则

原则	应用解释	易错提醒
无状态	服务不依赖请求者或提供者先前状态。	不要把会话状态放在服务端作为调用前提。
单一实例	避免功能冗余。	同一能力不要重复建设多个服务。
明确定义接口	用 WSDL 定义公共接口，用 Schema 定义消息格式，用 WS-Policy 描述规约。	接口一旦发布不能随意变化。
自包含和模块化	服务封装稳定、重复出现的业务活动。	服务应能独立部署、版本控制和恢复。

粗粒度	外部 服务接口 宜 粗粒度 ，内部可细粒度。	不是越细越好，细粒度会增加交互复杂性。
松耦合	使用者只依赖 接口 ，不依赖位置、技术和状态。	位置透明 、 实现透明 、 状态透明 。
可重用	服务 应能被多个流程或应用 复用 。	只 服务 一个页面按钮的 接口 通常不是好 服务 。
互操作、兼容和 策略 声明	通过 策略 声明安全、费用、 服务 级别等要求。	WS-Policy 常与 策略 、互操作语义对应。

高分表达：**SOA 服务**应稳定、自治、**粗粒度**、可**复用**、**无状态**，并通过明确**接口**与**策略**声明实现**松耦合**和跨平台互操作。

教材将**无状态**、单一实例、**接口**明确、自包含、**粗粒度**、**松耦合**、**可重用**、互操作和**策略**声明列为 **SOA 服务**设计的重要原则。

10. SOA 设计模式

10.1 服务注册表模式

服务注册表用于**服务**合同、**策略**、元数据的发布和管理，是 **SOA** 治理的主控制点或**策略**执行点。

功能	说明
服务注册	服务提供者 发布 服务 合同，包括身份、位置、方法、绑定、配置、 策略 等。
服务位置	服务消费者 查询和发现符合要求的 服务 。
服务绑定	消费者根据 服务 合同绑定并调用 服务 。
配置文件管理	管理 服务生命周期 阶段和阶段 策略 ，如开发、测试、质量保证、生产。

易错点：**UDDI** 是一种注册环境，但**服务注册表**不只等同于 **UDDI**。

10.2 企业服务总线模式

ESB 解决点对点集成方式复杂度高、**可管理性**差、**复用度**低和系统脆弱的问题。它用中间件平台支持异构**服务**以消息和事件驱动方式交互，并提供 **QoS** 与**可管理性**。

10.3 微服务模式

模式	含义	注意点
聚合器 微服务	调用多个 微服务 并聚合结果，可直接展示或封装成更高层 服务 。	聚合器也有自己的缓存和数据库。
代理 微服务	根据业务需要委派调用不同 微服务 ，主要做请求转发和数据 转换 。	是聚合器的变形。
分支 微服务	同时调用两个不同 微服务 链。	调用链互不影响。
链式 微服务	A 调 B，B 调 C，结果逐级合并返回。	同步调用链不宜过长。
数据共享 微服务	过渡期多个 微服务 共享缓存或数据库。	会削弱 服务 自治，可能引发耦合。
异步消息传递	用 消息队列 替代部分同步 REST 调用。	响应快，但会增加复杂性和可用性风险。

教材指出，**微服务**架构具有复杂应用解耦、独立开发测试部署、技术选型灵活、容错、**松耦合**易扩展等优势，但也带来**分布式**通信、**服务发现**、**调用链跟踪**、分区数据库、**最终一致性**、测试和运维监控复杂度等挑战。

11. 构建 SOA 架构注意事项

11.1 先分析原有系统集成需求

架构师不能一上来推倒重建。基于 **SOA** 的企业架构通常应在已有投资基础上演进，优先**复用**已有**资源**，包括开发人员、语言、硬件平台、数据库和应用程序。

分析范围包括：

1. 应用程序集成需求。

2. 终端用户界面集成需求。
3. 流程集成需求。
4. 已有系统信息集成需求。

案例答题句：应先盘点遗留系统能力和集成需求，将可复用功能封装为服务；对外通过标准接口暴露，对内通过ESB、适配器、消息转换和流程编排实现集成，避免整体替换带来的高成本和高风险。

11.2 控制服务粒度

外部暴露服务推荐使用粗粒度接口，内部实现可以使用较细粒度服务。原因是：细粒度接口虽然灵活，但会导致交互模式多变，增加外部调用者适配难度；粗粒度接口能保证服务请求者以一致方式使用服务。

11.3 优先设计无状态服务

无状态服务要求请求消息包含完成操作所需的全部信息，不依赖前序调用状态。这样更利于重用、并发、负载均衡、集群扩展和可靠性提升。教材还指出，可通过BPEL把依赖关系定义成业务流程，并用粗粒度服务对外暴露。

12. SOA 实施过程

12.1 选择 SOA 解决方案

选择维度	备考要点
全局规划	评估已有系统、需改造系统、新建系统、未来需求、资金投入和实施路径。
企业需求	SOA 项目价值不只在软件替换，而在业务流程优化和业务创新。
平台与供应商	看平台开放性、标准支持、实施方法、成功案例、专业服务能力。

12.2 建立服务模型

方法	思路	产出
自顶向下分解法	从端到端业务流程逐层分解到业务活动。	服务候选者、服务目录。
业务目标分析法	从业务目标和 KPI 出发，分解为子目标并映射服务。	目标服务树。
自底向上分析法	分析已有系统、套装应用、行业规范和业务模型。	可复用服务列表。

12.3 建立业务流程

步骤	内容
建立业务对象	实体业务对象、过程业务对象、事件业务对象。
建立服务接口	接口应语义相关、操作数量适中，优先无状态。
建立业务流程	将 BP 分解为子流程，识别组件、服务、输入输出、策略和度量。

接口粒度易错点：一个服务接口只有单个操作或少量操作，可能粒度过细；一个服务包含大量无关操作，也可能粒度过粗。合理接口应围绕语义相关的一组操作组织。

第 16 章 嵌入式系统架构设计理论与实践

一、本章考试定位与学习主线

第 16 章的核心不是“背硬件清单”，而是理解嵌入式系统如何受硬件约束影响软件架构设计。考试常从三类角度命题：一是综合知识中的概念辨析，二是案例题中的架构选型、实时性与可靠性分析，三是论文题中嵌入式、物联网、安全攸关系统、分布式终端操作系统等场景的架构论证。

本章学习主线可以概括为：

硬件约束 → 嵌入式基础软件 → 实时性与可靠性 → 架构设计方法 → 典型案例落地

也就是说，嵌入式架构设计必须同时回答四个问题：设备资源是否受限、任务是否有实时截止时间、系统是否需要长期无人值守、软件是否需要跨平台移植和裁剪。

二、嵌入式系统概述

1. 嵌入式系统定义

嵌入式系统是为**特定应用**专门构建的计算机系统，通常嵌入到设备、装置或更大的系统中完成控制、检测、通信、处理等任务。它与通用计算机系统最大的不同在于：目标明确、**资源受限**、**软硬件强耦合**、**可靠性**和**实时性**要求高。

考试抓手：看到“特定应用、专用设备、软硬件紧密结合、**资源受限**、实时控制、长期稳定运行”，优先联想到**嵌入式系统**。

2. 发展历程

阶段	核心形态	典型特征	考试记忆点
单片微型计算机阶段	SCM / 单片机	无操作系统，汇编开发，功能单一	“早期、无 OS、汇编”
微控制器阶段	MCU	外设和接口增强，简单 OS，控制能力提升	“智能控制、系统开销小”
片上系统阶段	SoC	多种处理器兼容，内核小，效率高	“高度集成”
Internet 嵌入式阶段	网络化嵌入式	处理器集成网络接口，设备进入网络环境	“网络接口+联网应用”
智能化、云技术阶段	端-云协同、IoT	低功耗 、高集成、高可信、智能服务	“端侧传感+智能服务”

易错提醒：

SCM、MCU、SoC 不是简单的性能排序，而是**嵌入式系统**从单机控制到高度集成、网络化、智能化的发展路径。

三、嵌入式系统硬件体系结构

教材图 16-1 展示了典型**嵌入式系统**硬件组成：处理器、存储器、总线、定时器/计数器、**看门狗**、**I/O** 接口、外部设备等。考试通常不要求画完整图，但要求能判断每个部件的作用。

1. 处理器类型

类别	适用场景	关键特征
MPU	复杂控制、需要外扩资源	处理能力较强，但外围器件需外接
MCU	控制类设备、 低功耗 设备	单片化，体积小，功耗低， 可靠性 高
DSP	信号处理、通信、音视频	适合数字信号处理算法，常采用哈佛结构
GPU	图形、图像、并行计算	强图形渲染和并行处理能力
SoC	高集成 嵌入式系统	将处理器、外设、接口等集成到芯片级系统

常见考法：

- “某系统要求**低功耗**、体积小、外围接口集成度高”——优先选 MCU 或 SoC；
- “实时音频/通信信号处理”——优先联想到 DSP；
- “3D 图形渲染、图像处理”——优先联想到 GPU。

2. 芯片环境等级

等级	工作温度范围	常见适用
民用级	0~70℃	普通消费电子
工业级	-40~85℃	工业控制、户外设备
军用级	-55~150℃	军工、航空、严酷环境

易错提醒：

处理器选择不仅看主频，还要看**温度**、**湿度**、**振动**、**加速度**、**可靠性**等级、功耗和生命周期。

3. 存储器考点

类别	特征	考试重点
RAM	可读写、断电丢失	运行时数据、程序临时存放
ROM	只读或少量可改、断电不丢失	固化程序、固件、启动代码
SRAM	无需刷新，速度快，成本高	常用于高速缓存
DRAM	需刷新，容量大，成本低	常用于主存
Flash	可擦写，断电不丢失	固件升级、嵌入式存储常见

教材列举了大量 RAM/ROM 细分类，备考时不建议逐项死背，重点掌握**易失/非易失**、**是否可擦写**、**速度**、**成本**、**典型用途**。

4. 总线逻辑

分类角度	类别	作用
------	----	----

分类角度	类别	作用
按信息类型	数据总线	传输数据
按信息类型	地址总线	指定存储地址或设备地址
按信息类型	控制总线	传输控制信号
按连接部件	片内总线	CPU 内部部件连接
按连接部件	系统总线	CPU、主存、I/O 接口连接
按连接部件	局部总线	少数组件间高速交换
按连接部件	通信总线	外部设备或系统间通信

常见总线：PCI、cPCI、PCMCIA、I²C、SCI、CAN、IEEE 1394、MIL-STD-1553B 等。

考试重点不是记全名，而是能区分：片内、板级、外设、通信用途。

5. 看门狗电路

看门狗是**嵌入式系统**的重要恢复机制。其基本思想是：系统正常运行时，程序定期“喂狗”复位计数器；若程序跑飞或卡死，计数器溢出，触发中断或复位，使**系统重启**并恢复运行。

应用场景：无人值守、远程设备、工业控制、物联网终端。

易错提醒：看门狗不是性能优化部件，而是**故障恢复和可靠性保障部件**。

四、嵌入式软件架构概述

早期嵌入式软件规模小，多为监控程序+应用程序；后来引入嵌入式操作系统、BSP、驱动、文件系统、网络协议栈、数据库、中间件等，逐渐形成**分层**架构。教材图 16-3 给出了“硬件层—BSP—RTOS—应用软件”的简单嵌入式软件架构。

1. 典型层次

层次	主要职责
硬件层	CPU、存储器、外设、接口
BSP / 驱动层	屏蔽硬件差异，提供硬件访问接口
RTOS / EOS 层	任务调度、资源管理 、通信、文件、网络
中间件/基础软件层	数据库、通信、消息、分布式服务
应用层	业务逻辑、控制逻辑、交互功能

2. GOA 开放式架构

SAE AS4893 中的 GOA 架构体现了**嵌入式系统开放式架构**思想，核心目标是提升系统的**可移植性**、**可互操作性**、**可剪裁性**和**易获得性**。其思想是通过明确软件、硬件和接口结构，支持系统移植、升级和功能扩展。

考试关键词：开放性、接口规范、抽象、**分层**、移植、升级、互操作。

五、两种典型嵌入式系统架构模式

1. 层次化模式架构

层次化模式通过抽象层次组织系统，高层依赖低层，低层为高层提供服务。**嵌入式系统**中常见的“硬件—BSP—RTOS—中间件—应用”就是典型**分层**思想。

类别	特点	优点	缺点
封闭型 分层	只能调用同层或下一层	封装性好，移植性好	性能可能较低
开放型 分层	可调用低于本层的任意层	性能较好	破坏封装，移植性下降

易错提醒：

开放型不是“更先进”，封闭型也不是“更低级”。选择取决于系统更重视性能还是**可移植性**、可维护性。

2. 递归模式架构

递归模式用于复杂系统的逐步分解，可以将复杂用例逐步细化为子系统、组件、任务和接口。其核心价值是：在每一步细化时验证**可靠性**、**实时性**和**可扩展性**。

工作流	做法	适用
自顶向下	从系统级开始分解子系统和结构对象	需求明确、强约束系统
自底向上	从关键类、关系、已有构件开始构造	有成熟领域经验或可复用资产

真题对应：案例题常给一个复杂嵌入式场景，要求说明为何采用**分层**、递归、组件化或服务化设计。答题时要围绕“降低复杂度、接口清晰、便于验证、便于复用”展开。

六、嵌入式操作系统 EOS / RTOS

1. 定义与特点

嵌入式操作系统负责**嵌入式系统**中的软硬件资源分配、**任务调度**、并行活动控制与协调，通常包括底层驱动、系统内核、设备驱动接口、通信协议、图形界面等。

特点	备考解释
可剪裁性	按目标系统需要裁剪功能
可移植性	支持不同处理器和开发板
强实时性	满足控制类设备时限要求
紧凑性	代码精炼，适应 资源受限
高质量代码	安全攸关场景要求可靠
强定制性	面向具体设备和行业定制
标准接口	提供统一驱动和 API
强稳定性、弱交互性	长期运行，用户干预少
强确定性	在规定时间、资源范围内完成任务
可固化性	OS 和应用可固化到 ROM/Flash

2. 操作系统分类

类别	典型系统	适用场景
实时嵌入式操作系统	VxWorks 、 Nucleus 、 QNX 等	工业控制、通信、医疗、军工、轨道交通
非实时/消费类嵌入式操作系统	Android 、 iOS 、 WinCE 等	手机、机顶盒、电子书、消费电子

易错提醒：

Android、**iOS** 也属于嵌入式操作系统，但通常不是考试中“强实时系统”的典型代表。

3. EOS 一般架构

嵌入式操作系统通常由**硬件驱动程序**、**调试代理**、**操作系统内核**、**文件系统**、**网络系统**、**可配置组件**组成，并向应用提供**API**。教材图 16-7 体现了硬件相关软件与硬件无关软件**分层**的思想。

组成	作用
ASP	与处理器硬件相关的结构支持包
BSP	与处理器外围芯片、开发板相关的板级支持包
OS 内核	任务、内存、通信、时钟、 中断管理
可配置库	提供运行时库、设备管理、人机接口、图形图像等扩展
文件系统	提供程序和数据存储能力
I/O 系统	管理输入输出设备与主机交换
网络系统	支持 TCP/IP 、 UDP 等网络协议

4. 宏内核与微内核

对比项	宏内核	微内核
服务位置	用户服务和内核服务多在同一空间	用户服务和内核服务在不同地址空间
性能	调用开销小，性能较好	消息通信开销较大
耦合度	高	低
可扩展性	相对较弱	较强
可移植性	相对较弱	较好
典型例子	VxWorks 5.5、 VRTX	L4、 WinCE 、部分嵌入式 Linux 场景

易错提醒：

微内核不是一定比**宏内核**“快”，它的优势主要在**结构清晰**、**可扩展**、**可移植**、**安全隔离**；**宏内核**常见优势是**效率高**。

七、任务管理与实时调度

1. 任务状态

嵌入式操作系统中，任务是调度的基本单位，常见状态包括：

状态	含义
就绪态	已具备运行条件，等待 CPU
执行态	正在占用 CPU 运行

状态	含义
阻塞态	等待事件、资源或 I/O 完成

常见转换：就绪→执行、执行→就绪、执行→阻塞、阻塞→就绪。

2. 调度分类

分类	类别	重点
按调度信息时机	离线 / 在线	离线确定性强，在线灵活
按能否打断	抢占 / 非抢占	抢占利于关键任务及时执行
按优先级变化	静态 / 动态	静态简单，动态灵活但开销大

3. 常见实时调度算法

算法	全称	优先级依据	典型特征
EDF	Earliest Deadline First	截止时间越早，优先级越高	动态优先级，可抢占或非抢占
LLF	Least Laxity First	松弛度越小，优先级越高	适合抢占式调度
RMS	Rate Monotonic Scheduling	周期越短，优先级越高	静态优先级，周期任务经典算法

4. 公式熟练

CPU 利用率：

$$U = \sum_{i=1}^n \frac{C_i}{T_i}$$

其中，C_i为任务执行时间，T_i为任务周期。该公式常用于判断一组周期任务对 CPU 的负载压力。

LLF 松弛度：

$$L_i = D_i - t - C_i'$$

其中，D_i为截止时间，t 为当前时刻，C_i' 为剩余执行时间。松弛度越小，任务越紧急。

RMS 优先级规则：

$$T_i \text{ 越小} \Rightarrow \text{优先级越高}$$

EDF 优先级规则：

$$D_i \text{ 越早} \Rightarrow \text{优先级越高}$$

常见考法：给出两个或多个周期任务，要求判断在 EDF、LLF 或 RMS 下的执行顺序。答题时先列“到达时间、执行时间、截止时间/周期、剩余时间”，再按算法规则逐时刻判断是否抢占。

八、存储管理与任务间通信

1. 五种存储管理

方法	核心理想	优点	缺点
分区存储	将内存划分为若干区域	管理简单	容易产生碎片，空间利用率低
分页存储	逻辑页映射到物理页框	无外部碎片，便于共享	页表开销大，需硬件支持
分段存储	按逻辑模块划分段	符合程序逻辑，便于保护和共享	有外部碎片
段页存储	先分段，再分页	兼顾逻辑性和空间利用率	管理复杂，开销较大
虚拟存储	内存+外存形成虚拟地址空间	提高内存利用率，扩展程序空间	需硬件支持，有 I/O 瓶颈

易错提醒：

分页解决的是物理内存离散分配问题；分段体现的是程序逻辑结构；段页式是二者结合。

2. 任务间关系

关系	含义	处理机制
相互独立	仅竞争 CPU	调度
竞争	竞争共享资源	互斥
同步	协调执行步调	信号量、事件
通信	传递数据和信息	消息队列、共享内存、Socket

3. 常见通信方式

方式	适用	注意点
----	----	-----

方式	适用	注意点
共享内存	高效共享数据	需同步保护
信号量	互斥、同步	二进制、互斥、计数 信号量
消息队列	同 CPU 多任务消息传递	支持 FIFO 、优先级、时限消息
Socket	网络通信	抽象 TCP/IP 复杂操作
Signals	异步事件通知	类似软件中断

真题对应：

若题干强调“多个任务访问共享资源导致数据不一致”，答互斥、**信号量**、锁机制；

若强调“任务之间传递数据并解耦”，答**消息队列**；

若强调“跨网络透明通信”，答 **Socket** 或远程调用。

九、嵌入式数据库

1. 定义与特点

嵌入式数据库通常嵌入应用程序进程中运行，减少客户端/服务器结构带来的部署和通信开销。它通常轻量、占用内存少、零配置、便于随应用一起发布。嵌入式实时数据库还要求事务处理具备**可预测性**和定时约束。

特点	说明
嵌入式	可嵌入软件或硬件设备
实时性	能在时限内响应数据请求
移动性	适合移动终端和现场设备
伸缩性	适应不同硬件和软件平台
可靠性	长期无人值守运行

2. 分类

类别	核心思路	典型适用
基于内存	数据主拷贝或工作版本常驻内存	强实时、电信、工业控制、军工
基于文件	数据按文件格式存储	轻量、单机、小型嵌入式应用
基于网络	远程数据库映射到本地设备	移动计算、远程同步、云端数据

3. 嵌入式数据库与数据库服务器区别

对比项	数据库服务器	嵌入式数据库
运行方式	独立进程/服务	与应用同进程或同部署包
访问方式	JDBC/ODBC、 TCP/IP 等	API 直接调用
部署	需独立安装与管理	随应用发布
控制	数据与程序分离	访问控制多交给应用
优势	集中管理、多用户能力强	轻量、低开销、易部署

4. 主要功能

嵌入式数据库应具备高效存储、安全控制、实时事务管理和恢复机制。具体可拆分为：

功能模块	作用
数据库运行处理	实时事务、并发控制、日志、安全完整性
数据库存取	查询、更新、添加、删除
数据管理	缓冲区、索引、存储空间管理
数据库维护	数据恢复、转储、装入
数据库定义	安全性 和完整性定义

易错提醒：

嵌入式数据库不是“小型数据库”的简单别称，它强调**部署方式**、**资源占用**、**实时约束**和**与应用/设备的紧耦合**。

十、嵌入式中间件

1. 定义

嵌入式中间件位于嵌入式应用和操作系统之间，用于屏蔽底层操作系统、网络、数据库、硬件平台的异构性，为应用提供通信、存储、数据处理和集成能力。

2. 共性特点

特点	含义
通用性	满足大量应用需求
异构性	支持不同硬件和操作系统
分布性	支持跨网络、跨平台交互
协议规范性	支持标准协议
接口标准化	提供统一 API 或接口

嵌入式中间件还要特别关注网络化、流媒体、QoS、适应性等场景。

3. 常见类型

类别	作用
消息中间件	异步通信、系统解耦、消息可靠传递
分布式对象中间件	支持跨平台对象调用和分布式服务
数据访问中间件	屏蔽数据库差异
事务处理中间件	保证事务一致性
ESB	企业级服务集成
DDS	数据分发、实时发布/订阅

4. 消息中间件

消息中间件在发送者和接收者之间保存、路由、转发消息。核心优点是异步处理和松耦合。

模型	特点
点对点 PTP	一个消息通常由一个消费者接收
发布/订阅 Pub/Sub	多个订阅者可接收主题消息

场景判断：

订单系统、支付系统、通知系统之间需要解耦，常使用消息中间件。

5. CORBA 与 DDS

技术	核心定位	考试关键词
CORBA	分布式对象请求代理结构	ORB、对象、跨语言、跨平台、软总线
DDS	数据分发服务	发布/订阅、以数据为中心、QoS、强实时、跨平台

易错提醒：

CORBA 偏“对象调用和互操作”；DDS 偏“实时数据分发和 QoS 策略”。

十一、嵌入式软件开发环境

1. 交叉开发

交叉开发是嵌入式开发的基本方式：在通用计算机上开发、编译和调试软件，再下载到目标嵌入式设备运行。

角色	含义
宿主机	运行开发工具的通用计算机
目标机	实际运行嵌入式软件的设备
交叉编译器	在宿主机上生成目标机可执行代码

2. 开发环境特点

特点	说明
IDE 集成	编辑、编译、调试、图形界面
开放体系结构	支持标准、插件和第三方工具
可扩展性	可增加工具能力
可配置性	可裁剪库和工具
代码实时性	生成高效目标代码
可维护性	工具松耦合，便于升级
用户界面友好	提升开发效率

3. 调试方式

方法	特点
模拟器	在 宿主机 模拟目标处理器运行
在线仿真器 ICE	硬件级仿真，目标系统透明可控
监控器 Monitor	目标板运行监控程序，配合宿主调试
JTAG 仿真器	通过 JTAG 边界扫描接口调试

4. 常见工具链

工具/环境	特点
GCC	开源、跨平台、支持多语言和多体系结构
Workbench	WindRiver 配套，常用于 VxWorks 开发
MULTI	Green Hills 环境，强调安全、调试、覆盖率和 静态 分析

真题对应：

若题干出现“**宿主机**开发、**目标机**运行”，答案通常围绕**交叉开发**、交叉编译、下载、远程调试展开。

十二、嵌入式系统软件架构设计方法

1. 架构设计目标

嵌入式软件架构设计的目标包括：代码逻辑清晰、避免重复设计、提高**可移植性**、促进软件复用、实现**高内聚低耦合**。架构不是可运行程序，而是用于分析需求满足程度、比较设计方案、降低构造风险的表达。

2. ABSD 方法

ABSD，即基于架构的软件设计，强调由业务需求、质量需求和功能需求共同驱动架构设计。它是自顶向下、递归细化的方法，适合**嵌入式系统**这类目标明确、用途单一、质量和**可靠性**要求高的系统。

答题模板：

“先识别系统功能和**质量属性**，再选择合适架构风格，通过**分层**、**构件化**、接口定义和迭代细化，使架构满足**实时性**、**可靠性**、**安全性**、可裁剪性和可维护性要求。”

3. ADD 方法

ADD，即属性驱动设计，适合**质量属性**突出的**嵌入式系统**，尤其是**安全攸关系统**。它以**质量属性场景**为输入，通过选择架构模式和质量战术来分解模块、分配职责和定义接口。

质量场景六要素：

要素	含义
刺激源	谁触发事件
刺激	发生了什么
环境	事件发生条件
制品	受影响的系统部分
响应	系统如何反应
响应度量	如何量化反应结果

ADD 七步：

- 1. 评审输入。
- 2. 选择驱动因子，建立迭代目标。
- 3. 选择系统元素进行细化。
- 4. 选择设计概念。
- 5. 实例化架构元素、分配职责、定义接口。
- 6. 草拟视图并记录设计决策。
- 7. 分析当前设计并评审迭代目标。

易错提醒：

ADD 不是从功能模块直接拆代码，而是从**质量属性场景**驱动架构决策。

4. DARTS 实时系统设计方法

DARTS 用于实时系统设计，核心思想是将实时系统分解为多个并发任务，并定义任务之间的接口。

DARTS 五步：

步骤	内容
1	用 RTSA 开发系统规范
2	将系统划分为多个并发任务
3	定义任务间接口
4	设计每个任务
5	形成设计成果，包括 RTSA 规范、任务结构规范、任务分解

任务间接口形式：

接口	用途
消息通信	任务间传递数据
事件同步	无数据传递但需协调执行
信息隐藏模块 IHM	封装共享数据和状态

真题对应：

若题干强调“实时系统、并发任务、任务接口、时间约束、控制流/数据流”，优先联想到 **DARTS**。

十三、案例架构：鸿蒙、GENESYS、物联网操作系统

1. HarmonyOS 架构

HarmonyOS 采用**分层**设计，从下到上包括内核层、系统服务层、框架层、应用层。其重点是分布式架构、多终端协同、统一开发和安全可信。

层次	主要内容
内核层	内核子系统、驱动子系统、多内核设计、 KAL 、 HDF
系统服务层	分布式软总线 、分布式数据管理、 任务调度 、安全、AI 等
框架层	多语言框架 API 、 Ability 框架
应用层	系统应用、第三方应用， FA/PA 能力

四个技术特性：

1. 分布式架构实现**跨终端协同**。
2. 确定时延引擎和高性能 IPC 提升流畅性。
3. **微内核**架构提升可信安全。
4. 统一 IDE 支持一次开发、多端部署。

答题亮点：

分布式软总线、分布式设备虚拟化、分布式数据管理、分布式**任务调度**，是鸿蒙架构案例最容易出现在案例题和论文中的关键词。

2. GENESYS 架构

GENESYS 是面向**安全攸关系统**的跨领域通用嵌入式架构平台，解决复杂性管理、系统健壮性和能量有效使用三大挑战。

挑战	架构应对
复杂性管理	提升抽象级别，采用构件、接口和消息交换
系统健壮性	故障隔离 、构件重启、构件复制、信息安全贯穿各层
能量有效使用	软件构件迁移到硬件构件，功率门控，时间触发通信

GENESYS 重要思想是**计算与通信分离**。构件之间通过消息传输通信，构件内部实现可替换，只要接口行为和时序性质保持不变，就能降低系统集成风险。

四类接口：

接口	作用
LIF	构件间基于消息的连接接口
LI	构件与局部外部环境连接
III	配置、 资源管理 等技术无关接口
TDI	诊断、观察内部变量等技术相关接口

易错提醒：

GENESYS 不是某个具体操作系统，而是跨领域嵌入式架构平台和架构思想。

3. 物联网操作系统架构

物联网系统可理解为泛化的**嵌入式系统**，关键包括感知层、网络传输层和应用层。**物联网操作系统**通常涉及芯片层、终端层、边缘层、云端层等多层能力。

以 **FreeRTOS** 为例，其主要由 **BSP** 驱动、内核和组件组成：

层次	内容
BSP 驱动	管理硬件资源，为内核提供硬件操作服务
内核	多 任务调度 、内存管理、 任务间通信
公共组件	TCP 、 TLS 、 Wi-Fi 、 蓝牙 、 POSIX 等
定制组件	OTA 、 PKCS#11 、安全套件、 MQTT 、 HTTPS 等

物联网操作系统核心特征：

- 1. 内核尺寸可伸缩，整体架构可扩展。
- 2. 内核具备**实时性**。
- 3. **高可靠性**，适应海量节点和难维护场景。
- 4. **低功耗**，需考虑休眠、节能、降频等技术。

真题对应：

若题干涉及“海量终端、**低功耗**、**远程升级**、**安全通信**、实时响应、**边缘设备**”，答题应围绕 **IoT OS** 的伸缩性、**实时性**、**可靠性**、**安全性**和**低功耗**展开。

第 17 章 通信系统架构设计理论与实践

一、本章考试定位

第 17 章属于**通信系统架构设计**专题，综合知识与案例分析均可能涉及。综合知识常考概念、架构优缺点、协议作用、**IPv4/IPv6** 过渡技术、**SDN**、**NAS/SAN**、5G 边缘计算；案例分析常考**园区网/广域网/高可用网络/IPv6 双栈改造/网络安全与绿色设计**。

备考抓手可以归纳为：

一条主线：业务需求 → 网络架构选型 → **高可用**与安全设计 → **IPv6/SDN/5G/存储**等技术落地。

两个核心能力：会比较架构优缺点；会把技术放入具体场景中选型。

三个常考关键词：**高可用**、层次化、可演进。

二、通信系统概述

通信系统又称**通信网络**，是利用通信线路把地理分散、功能独立的计算机系统和通信设备连接起来，通过网络软件和通信协议实现**资源共享**与**信息传递**的系统。教材第 17 章主要讲 5 类常用网络架构及网络构建技术：**局域网**、**广域网**、**移动通信网**、**存储网络**、**软件定义网络**。

考点提炼

通信网络的发展趋势：

从低速拨号到光纤高速接入；从 GE/10GE 到 40GE/100GE/400GE；从简单总线式网络到复杂异构多层网络；从传统物理网元到 5G **虚拟化**、服务化、可灵活部署的网络功能。

易错提醒

不要把“**通信系统**”理解成单纯通信链路。考试中更关注其**架构设计能力**：如何按业务、**可靠性**、安全、性能、成本等因素设计网络。

三、通信系统网络架构

3.1 局域网网络架构

局域网是单一机构拥有的专用计算机网络，覆盖范围小，通常在建筑或建筑群内；传输速率高，误码率低，**可靠性**较高，支持有线和无线介质。教材给出的典型特征包括：覆盖范围通常较小、速率一般在 10Mb/s 以上，典型 1Gb/s，甚至 10Gb/s；误码率通常较低。

1. 局域网组成

局域网通常由**计算机**、**交换机**、**路由器**、**服务器**、**接入设备**等组成。现代**园区网**通常已经不是简单二层交换网络，而是二层交换与三层路由并存。

2. 局域网典型架构对比

架构	核心结构	优点	缺点	适用场景
单核心架构	一台核心二层/三层交换设备 + 多台接入交换机	结构简单、投资低、接入方便	核心 单点故障 、扩展能力有限、端口密度压力大	小规模、部门集中、成本敏感
双核心架构	两台核心交换设备互联， 接入层 双归或多路径	支持网关保护/负载均衡， 可靠性 高，可热切换	投资高，核心端口密度要求高	中大型 园区网 、核心业务较多
环型架构	多台核心交换设备形成双 RPR 环	节省光纤资源， RPR 具备自愈保护，支持 QoS	多环互通复杂，易形成环路，实施难度较高	地理分布呈环状、需要链路自愈
层次局域网	核心层 、 汇聚层 、 接入层	易扩展、易排障、便于分层控制	设计复杂度高，需要合理规划层级	主流 园区网 、大中型企业网

3. 局域网协议应用

主备保护：**VRRP**、**HSRP**、**GLBP**。
二层链路保护/扩展：**STP**、**LACP**。
三层动态路由：**OSPF**、**RIP**、**BGP**。

真题对应

常见问法：

给出**园区网**拓扑，问应采用单核心、双核心、环型还是**三层模型**；给出故障场景，问应部署 **VRRP**、**STP**、**LACP** 还是 **OSPF**；给出 **VLAN** 场景，问 **VLAN** 内与 **VLAN** 间转发分别属于哪一层。

易错提醒

VLAN 内通常是二层转发，**VLAN** 间需要三层路由；**接入层**设备通常只做二层转发，核心或**汇聚层**承担三层路由与策略控制。

3.2 广域网网络架构

广域网用于连接比**局域网**范围更广的计算机设备。其由**通信子网**和**资源子网**组成，通常可分为**骨干网**、**分布网**、**接入网**；小规模网络可只设置**骨干网**和**接入网**。

广域网架构对比

架构	特点	优点	风险/不足
单核心 广域网	一台核心路由设备连接各 局域网	简单、省投资、互访效率高	核心 单点故障 ，扩展差，端口要求高
双核心 广域网	两台核心路由设备，支持网关保护或负载均衡	可靠性 高，可热切换	成本高，路由冗余设计复杂，易形成 路由环路
环型 广域网	三台以上核心路由设备构成环	多路径、 高可靠 、可无缝热切换	占用端口多，环路控制难度高
半冗余/全冗余 广域网	多核心路由设备网状互连	灵活扩展，多路径，适合 OSPF 等链路状态协议	管理和排障较难
对等子域广域网	划分两个独立子域，域间互连	可做路由汇总，适合区域相对独立场景	域间路由冗余难，边界设备性能要求高
层次子域广域网	多个子域存在上下层关系	扩展性好，适合大型 广域网	低层子域互访需经高层子域，域间设计复杂

真题对应

案例题常给出“总部—省分—地市—网点”结构，要求判断**骨干网**、**分布网**、**接入网**职责，或者判断采用**层次子域**、**对等子域**还是半冗余结构。

易错提醒

“链路多”不等于“设计一定好”。**广域网**路由冗余如果没有合理的路由控制，可能引入**路由环路**、**非法路由发布**、**排障困难**。

3.3 移动通信网网络架构：5G 重点

5GS 为 **UE** 提供服务时，通常需要与 **DN** 互连。**DN** 可以是 Internet、**IMS**、专用网络等。**5GS** 与 **DN** 通过 **N6** 接口连接，**UPF** 是 **5GS** 面向 **DN** 的接入点。

1. 5GS 与 DN 互连

核心理解：

概念	含义
UE	用户设备
5GS	5G System，5G 系统
DN	Data Network，数据网络
UPF	用户面功能，负责用户面业务转发
N6	UPF 连接数据网络的接口
UL	上行， UE → DN
DL	下行， DN → UE

从 DN 看，UPF 可以视为普通路由器；从 5GS 看，N6 对端通常也是路由器。因此，5GS 与 DN 的本质是路由互通关系。

2. 透明模式与非透明模式

模式	地址来源	认证特征	适用理解
透明模式	UE 使用运营商规划地址	通常不触发外部 DN-AAA 认证	5GS 提供基本 ISP 隧道服务，对 Intranet 配置透明
非透明模式	UE 使用 Intranet/ISP 地址空间	通常由 SMF 向外部 DN-AAA 发起认证	适合企业专网、行业专网、需要专用隧道和安全保护的场景

3. 5G 边缘计算 MEC

5G MEC 的关键是把 UPF 与边缘计算平台 MEP 部署在靠近用户的位置，实现业务就近分流，降低时延，减轻中心网络压力。运营商或第三方应用可通过 NEF、PCF、SMF 等机制触发本地分流策略。

4. SSC 模式

模式	业务连续性特点
SSC Mode 1	用户移动过程中 IP 接入点保持不变
SSC Mode 2	先释放旧会话，再立即建立新会话
SSC Mode 3	释放旧会话前先建立新会话，连续性更好

易错提醒

MEC 不是单纯“把服务器放近一点”，而是结合 5G 的 UPF、SMF、PCF、NEF 等功能，实现用户位置感知、本地分流、动态策略配置。

3.4 存储网络架构

计算机访问磁盘存储有三种方式：DAS、NAS、SAN。

类型	访问方式	典型协议/技术	特点
DAS	主机通过 I/O 端口直接访问存储	IDE、ATA、SCSI 等	简单、直连、扩展性较弱
NAS	通过分布式文件系统访问存储	CIFS/SMB、NFS	易管理、易共享、基于以太网，性能低于本地存储
SAN	通过独立高速存储网络访问块存储	FCP、iSCSI、FCoE、FC-NVMe	高性能、低延迟、高可用，适合数据库、银行、ISP、5G 网元存储

NAS 与 SAN 对比

NAS 关注易用、易管理、可扩展、低 TCO；SAN 关注高性能、低延迟、高可用。NAS 多基于以太网，SAN 可基于以太网或光纤通道。

真题对应

选择题常考：

CIFS/SMB 对应 Windows；NFS 最早面向 UNIX，常见于 Linux；iSCSI 是把 SCSI 命令封装后通过 IP 网络传输；FCP 是常见 SAN 块协议。

易错提醒

NAS 是“文件级共享”，SAN 是“块级存储网络”。数据库、核心交易、5G 网元数据持久化等更偏向 SAN；部门文件共享更偏向 NAS。

3.5 软件定义网络 SDN

SDN 的核心思想是控制面与数据面分离，控制面集中管控，数据面通用化、轻量化转发，并通过开放接口支持可编程网络。

SDN 三层架构

平面	作用
应用平面	网络应用，调用网络能力，不关心底层细节
控制平面	SDN 控制器，掌握全局网络状态，下发转发规则
数据平面	转发设备，根据流表执行转发

SDN 接口

接口	连接对象	作用
南向接口 SBI	控制器 ↔ 数据平面	下发流表/转发规则，典型协议 OpenFlow
北向接口 NBI	控制器 ↔ 应用平面	为应用开放网络能力
东西向接口	控制器 ↔ 控制器	多控制器协同、扩展与负载均衡

易错提醒

OpenFlow 是典型南向接口协议，不是北向接口；流表由控制器建立、维护和下发，交换机按流表执行转发。

四、网络构建关键技术

4.1 网络高可用设计

1. 可用性公式

可用性：
$$A = \frac{MTBF}{MTBF + MTTR}$$

其中，MTBF 是平均无故障时间，MTTR 是平均故障修复时间。提高可用性的两条路径：提高 MTBF，或降低 MTTR。教材强调，工程上更常通过快速发现故障、快速恢复业务来降低 MTTR。

2. 拓扑可用性公式

设 NE1 在线率为 R_1 ，NE2 在线率为 R_2 。

拓扑	公式	记忆方法
一对一	$A_1 = R_1R_2$	串联相乘
一对二	$A_2 = R_2[1 - (1 - R_1)^2]$	NE1 双点并联后再串 NE2
二对一	$A_3 = R_1[1 - (1 - R_2)^2]$	NE2 双点并联后再串 NE1
双一对一	$A_4 = 1 - (1 - R_1R_2)^2$	两组串联系统并联
双一对二	$A_5 = [1 - (1 - R_1)^2][1 - (1 - R_2)^2]$	两类设备均并联后再组合

3. 高可用技术

网络部件层面：主控冗余、业务板热插拔、电源/风扇冗余、软件热补丁、异常保护、数据备份。

协议层面：OSPF/BGP、BFD/NQA、802.3ad、VRRP、多下一跳。

易错提醒

可靠性和可用性不是同一个概念。可靠性强调“少出故障”，可用性强调“长期可服务时间比例”。一个系统即使故障少，但恢复慢，可用性仍可能不高。

4.2 IPv4 与 IPv6 融合组网

IPv4 与 IPv6 会长期共存，因此过渡技术非常重要。教材归纳为三类：双协议栈、隧道技术、网络地址翻译技术。

1. 双协议栈

节点同时具有 IPv4 协议栈和 IPv6 协议栈，可同时与 IPv4 节点和 IPv6 节点通信。适合逐步演进，兼容性强，但需要设备、应用和运维均支持双栈。

2. 隧道技术

技术	核心机制	适用场景
ISATAP	IPv6 报文封装在 IPv4 中，IPv4 地址嵌入 IPv6 地址	IPv4 网络内支持 IPv6 主机互通
6to4	地址前缀 2002，将 IPv4 地址嵌入 IPv6 前缀	IPv6 站点经 IPv4 网络互联
4over6	IPv4 over IPv6	IPv4 业务承载在 IPv6 网络上
6over4	IPv6 over IPv4 组播网络	双栈节点在组播 IPv4 网络上传送 IPv6 分组

3. 网络地址翻译

NAT 技术把 IPv4 与 IPv6 地址分别视为内外部地址进行映射，实现 IPv4 主机与 IPv6 主机互通。相关协议包括 NAT-PT、SIIT。

真题对应

常见题型：

- “已有 IPv4 网络投资不能废弃，但要接入 IPv6，如何设计？”优先考虑双栈或隧道；
- “IPv4 主机和 IPv6 主机互通”常对应地址翻译；
- “纯 IPv6 用户经既有 IPv4 网络访问”常联系隧道。

易错提醒

6to4 不解决 IPv4 地址短缺问题，因为需要全局合法 IPv4 地址。双栈不是隧道，NAT 也不是隧道。

4.3 SDN 关键技术

SDN 关键技术包括：**控制平面技术**、**数据平面技术**、**转发规则一致性更新技术**。

控制平面：单控制器可能成为性能瓶颈，可通过单控制器扩展或多控制器扩展；多控制器可采用扁平控制模型或层次控制模型。

数据平面：硬件方式速度快但灵活性弱；软件方式灵活性强但性能弱；NP 处理网络任务通常优于 CPU。

规则一致性：规则更新可能导致不同交换机状态不一致，可采用“两阶段提交”思想，或使用增量式一致性更新减少规则空间占用。

易错提醒

SDN 不等于“没有**路由协议**”，它强调的是控制逻辑集中化、转发规则可编程化。

五、网络构建和设计方法

5.1 网络需求分析

需求分析是网络构建起点，最终形成**网络需求规格说明书**。教材将**需求分析**分为五类：业务需求、用户需求、应用需求、计算机平台需求、网络需求。

需求类型	重点
业务需求	干系人、关键时间点、投资规模、业务增长、 可靠性 、安全、远程访问
用户需求	用户需要的服务、响应时间、 可用性 、可伸缩性、安全性、成本
应用需求	应用类型、部署地点、使用方法、增长性、响应时间、 可靠性
平台需求	PC、工作站、小型机、中型机、大型机等接入设备类型
网络需求	拓扑、性能、管理、安全、 局域网/广域网 连接方式

易错提醒

网络设计不是先买设备，而是先做**需求分析**。案例题中如果问“设计步骤”，应先答**需求分析**，再答架构、地址、路由、安全、**高可用**等。

5.2 网络技术遴选

1. 局域网技术

STP：解决二层冗余引发的环路问题。

VLAN：把物理网络划分为多个逻辑广播域，实现部门隔离。

无线**局域网**：关注 AP 覆盖、冗余、SSID 一致、虚拟网规划。

线路冗余：可采用备份或负荷分担。

交换设备功能：链路聚合、冗余网关、PoE、多业务模块。

服务器冗余：负载均衡器、**NAT**、**DNS** 轮询、双机热备；双机热备包括单活和双活。

2. 广域网技术

远程接入技术：PSTN、**ISDN**、Cable Modem、DSL、PON、无线宽带、小区宽带、PLC 等。

广域网互连技术：**DDN**、SDH、**MSTP**、**VPN**。

性能优化：预留带宽、拨号线路备份、数据压缩、链路聚合、按优先级调度、基于协议的带宽预留。

3. 地址规划

原则：结构化、层次化；中心授权管理；授权下发；终端动态地址；合理使用私有地址并规划 **NAT** 地址池。

4. 路由协议选择

考点维度：距离矢量/链路状态；度量值；协议优先级；层次化/非层次化；内部/外部；分类/无类；静态路由。

5.3 层次化网络模型设计

经典**三层模型**：**核心层**、**汇聚层**、**接入层**。

层次	功能	设计要点
核心层	高速转发、连接不同区域	冗余、 高性能 、 低时延 ，避免复杂策略处理
汇聚层	策略控制、路由汇总、安全控制	屏蔽 接入层 细节，向 核心层 做路由汇总
接入层	用户接入、认证、计费、日志	控制用户接入方式，收集 IP/MAC/访问日志

层次化设计原则

控制层数，避免过多层次增加时延；从**接入层**开始分析流量，再做上层容量规划；模块化设计，边界清晰；严格控制**接入层**，避免非法外联；避免跨层连接造成安全和运维问题。

易错提醒

核心层不要承担过多复杂策略，例如大量过滤、策略路由等，否则会增加处理时延，影响核心转发性能。

5.4 网络安全设计

网络安全技术主要包括：

技术	作用
防火墙	网络边界安全屏障，可允许、拒绝、重定向、审计流量
VPN	利用公共网络建立私有专用网络，常见 IPSec、GRE、MPLS VPN、VPDN
访问控制	主体按策略访问客体，涉及认证、授权、审计
安全隔离	分子网隔离、VLAN 隔离、逻辑隔离、物理隔离
安全协议	SSL、SET、HTTPS
安全审计	脆弱性测试、事件记录、分析、响应

易错提醒

HTTPS 是基于 SSL/TLS 的 HTTP 安全通信；SET 主要用于电子支付场景；SSL 提供加密、身份认证、完整性保护等能力。

5.5 绿色网络设计

绿色网络不仅是买低功耗设备，更是从整体架构优化能耗、降低 TCO。教材强调四大原则：标准化、集成化、虚拟化、智能化。

原则	核心
标准化	降低异构复杂度，减少转换设备和维护成本
集成化	减少设备、机架、线缆、电源、人力资源
虚拟化	端到端资源池化、按需调配
智能化	统一管理、自愈、自动分析、减少人工与重复数据

易错提醒

绿色设计不能只答“节能设备”。案例题应从架构标准化、设备集成、资源虚拟化、智能运维、安全可靠一起答。

六、通信网络构建案例分析

6.1 高可用网络构建

1. 接入层高可用

接入层高可用设计包括：冗余引擎、冗余电源、双归属接入、链路聚合、802.1x、动态 ARP 检查、IP 源地址保护等。教材对多种接入—汇聚连接模型后，认为三角形组网具有更高可用性和更灵活的设备扩展能力；但因存在二层环路，需要启用 MSTP，并在汇聚层部署 VRRP。

2. 汇聚层高可用

汇聚层到核心层可采用 OSPF 等动态路由协议。三角形连接具备全冗余链路和转发路径，故障收敛快，但端口占用多、成本高。汇聚层还可部署 FW、LB，实现安全边界、流量控制、负载均衡、SSL 卸载等。

3. 核心层高可用

核心层是网络枢纽，应优先采用双核心或多核心，以主备或负荷分担方式运行。核心层宜采用冗余的点到点三层互连，结合 ECMP 提高带宽利用率和收敛能力。

高可用指标记忆

指标	要求
核心层设备故障恢复时间	< 500ms
汇聚层设备故障恢复时间	< 1s
核心/汇聚双主控切换时间	< 200ms
核心—汇聚、接入—汇聚链路故障恢复时间	< 500ms
链路聚合故障恢复时间	< 1s

6.2 园区网双栈构建

园区网从 IPv4 升级到 IPv4/IPv6 双栈，应制定升级流程：

- 1. 制订网络设备升级计划。
 - 2. 评估现网设备对 IPv6 的支持情况。
 - 3. 评估需升级至双栈的网络服务。
 - 4. 制定 IPv6 地址分配方案。
 - 5. 制定详细 IPv6 网络升级方案。
- 设计要点：

核心层和汇聚层使用双栈交换机；接入层可保留现有二层交换机，保护已有投资；IPv4 和 IPv6 网关布设在汇聚三层交换机；汇聚和核心之间、接入和汇聚之间采用双归链路；汇聚层运行 VRRP；核心层双活。

对于不能全面双栈的区域，可使用 ISATAP 等隧道技术接入 IPv6 网络，但大量用户接入时要注意隧道端节点性能瓶颈。

6.3 5G 网络应用：智能电网

5G 可支撑高带宽、大连接、低时延、高可靠场景。教材以智能电网为例：通过 5G 将电源侧、电网侧、用户侧、供应链中的智能感知设备接入 IoT 平台，由平台进行数据采集、AI 训练/推理、电网监控、边缘计算，为输电、配电、综合能源等业务决策提供支撑。

真题对应

如果案例问“为什么 5G 适合智能电网/工业控制/远程医疗/无人驾驶”，答题关键词应包括：高带宽、大连接、低时延、高可靠、边缘计算、数据采集、实时控制、业务连续性。

第 18 章 安全架构设计理论与实践

一、本章总纲：安全架构到底考什么

一句话总纲：
安全架构 = 风险识别 + 安全策略 + 安全模型 + 技术防护 + 组织管理 + 审计响应 + 持续改进。

考试不是单纯问“什么是安全”，而是让你在一个信息系统、云平台、工业系统、电子商务系统或数据库系统场景中回答：

考查方向	必须会答
综合知识	安全威胁分类、CIA、BLP/Biba/CWM/Chinese Wall、OSI 安全服务、WPDRRC
案例分析	如何设计安全体系、如何做认证授权审计、如何部署防火墙/IDS/备份/容灾
论文写作	如何围绕“安全性”进行架构设计、权衡性能与安全、设计分层防御和安全管理机制

二、18.1 安全架构概述

1. 信息系统面临的安全威胁

教材将威胁理解为覆盖物理环境、通信链路、网络系统、操作系统、应用系统、管理系统等多个层面的系统性风险。常见威胁包括信息泄露、完整性破坏、拒绝服务、非法使用、窃听、业务流分析、假冒、旁路控制、授权侵犯、木马、陷阱门、抵赖、重放、病毒、人员渎职、媒体废弃、物理侵入、窃取和业务欺骗等。

考试压缩记忆：

威胁类型	关键词	常见考法
被动攻击	窃听、信息截取、业务流分析	不改变数据，只偷看、分析
主动攻击	篡改、假冒、重放、拒绝服务、病毒	改数据、冒身份、阻服务
人员风险	渎职、误操作、无意泄密、内部攻击	管理制度和权限控制
物理风险	火灾、水灾、盗窃、电源故障	机房、备份、容灾
系统配置风险	默认配置、弱口令、开放端口	安全加固、漏洞扫描

易错提醒：
窃听、业务流分析属于被动攻击；篡改、重放、拒绝服务属于主动攻击。题干中出现“不破坏通信内容，只分析通信频率、流向、总量”，优先选业务流分析。

2. 安全架构的定义和范围

教材将安全架构看成系统架构面向安全性方向的细分，可从三道防线理解：

防线	作用	答题关键词
产品安全架构	从产品源头构建安全质量属性	安全内建、默认安全、安全设计
安全技术体系架构	构建通用安全基础设施	身份鉴别、访问控制、密码、审计、防病毒

审计架构	发现风险并形成监督闭环	日志、 审计 、 风险发现 、 追责
------	-------------	---

核心目标是 **CIA**：
C = Confidentiality **机密性**：防止未**授权**披露。
I = Integrity **完整性**：防止未**授权**修改。
A = Availability **可用性**：防止资源不可用、服务中断。
扩展到考试场景，还要加上：**可控性**、**不可抵赖性**、**可审计性**、**合规性**。

三、18.2 安全模型

安全模型用于把抽象**安全策略**变成可判断、可执行、可验证的规则。教材重点介绍**状态机模型**、**BLP**、**Biba**、**Clark-Wilson**、**Chinese Wall** 等典型模型。

1. 五大模型对比表

模型	解决重点	核心规则	典型场景
状态机模型	系统始终保持安全状态	初始安全 + 状态迁移仍安全	形式化安全验证
Bell-LaPadula, BLP	机密性	低不能读高；高不能写低	军事、涉密、多级安全
Biba	完整性	低不能写高；高不能读低	防止低可信数据污染高 完整性 数据
Clark-Wilson, CWM	商业 完整性	CDI、UDI、IVP、TP、 职责分离	银行、 财务 、 交易系统
Chinese Wall	利益冲突隔离	同一 利益冲突组 最多访问一个公司数据集	投行、咨询、 审计 、 竞业隔离

2. BLP 模型：机密性模型

记忆公式：

BLP = 保密优先 = No Read Up + No Write Down

规则	含义	口诀
简单安全规则	低级主体不能读高级客体	不能向上读
星属性规则	高级主体不能向低级客体写	不能向下写
强星属性	不允许跨级读写	同级才安全
自主安全规则	用 访问控制矩阵 补充 DAC	矩阵控权限

易错提醒：
BLP 保护的是**机密性**，不是**完整性**。题目只要强调“秘密、机密、绝密、数据不能泄露”，优先联想 **BLP**。

3. Biba 模型：完整性模型

记忆公式：

Biba = 完整性优先 = No Write Up + No Read Down

规则	含义	口诀
星 完整性 规则	低 完整性 主体不能写高 完整性 客体	低不能污染高
简单 完整性 规则	高 完整性 主体不能读低 完整性 客体	高不能受低影响
调用属性规则	低 完整性 主体不能调用高 完整性 服务	低不能调高

易错提醒：
BLP 和 **Biba** 方向相反。
BLP 防泄密，**Biba** 防污染。

4. Clark-Wilson 模型：商业完整性模型

核心术语：

术语	含义
CDI	受 完整性 保护的数据项

UDI	未受完整性保护的数据项
IVP	完整性验证过程，检查 CDI 是否合法
TP	转换过程，把数据从一个有效状态变成另一个有效状态
职责分离	防止一个人独自完成关键舞弊操作

答题模板：
答题模板：对交易、财务、订单、账务类系统，应采用 Clark-Wilson 完整性思想，通过受控数据项 CDI、完整性验证 IVP、转换过程 TP 和职责分离机制，确保数据只能通过合法业务过程被修改，并通过审计记录形成追踪能力。

5. Chinese Wall 模型：利益冲突隔离

核心判断：

概念	含义
C	单个企业或客户的信息
CD	某企业全部数据集合
COI	存在利益冲突的一组公司数据集合

规则：
一个主体访问过某利益冲突组中的某公司数据后，只能继续访问该公司数据，或访问其他无冲突组的数据；不能再访问同一利益冲突组中竞争公司的数据。
易错提醒：
Chinese Wall 不是普通防火墙，而是利益冲突隔离模型。题干出现“投行、咨询师、审计机构、竞争客户、商业秘密”，优先选它。

四、18.3 系统安全体系架构规划框架

教材将信息技术系统安全架构建立在组织策略、风险评估、现状与需求基础上，并将风险威胁分析对象抽象为应用、存储、主机、网络、物理五个层次。

1. 信息系统安全体系三部分

体系	内容	考试关键词
技术体系	物理安全技术、系统安全技术	防火墙、IDS、加密、备份
组织体系	机构、岗位、人事	领导层、管理层、执行层
管理体系	法律、制度、培训	安全制度、责任分离、安全意识

2. 安全规划三条主线

第一，依托企业信息化战略规划。安全目标必须服务业务目标，不能脱离企业信息化建设。
第二，围绕技术安全、管理安全、组织安全进行规划。
第三，以信息系统和信息资源保护为核心，形成蓝图、现状、需求、措施四步闭环。
案例答题模板：
答题模板：信息系统安全规划应以企业信息化战略为牵引，围绕技术、管理、组织三个维度，先识别资产和风险，再划分安全区域，制定访问控制、认证授权、审计、备份、容灾、应急响应和人员管理措施，最终形成覆盖物理、网络、系统、应用、数据和运营的安全保障体系。

五、18.4 信息安全整体架构设计：WPDRRC 模型

WPDRRC 模型是本章高频考点。教材将其归纳为 6 个环节和 3 大要素：6 个环节是预警、保护、检测、响应、恢复、反击；3 大要素是人员、策略、技术。

1. WPDRRC 六环节

环节	英文	作用	常用技术
W 预警	Warning	发现风险趋势和薄弱点	风险评估、漏洞扫描、威胁情报
P 保护	Protect	事前防护	加密、签名、认证、访问控制、防火墙

D 检测	Detect	发现入侵和异常	IDS、 漏洞 检测、 完整性 检测
R 响应	React	事件发生后处置	告警、隔离、封堵、报告
R 恢复	Restore	恢复业务	容错、冗余、 备份 、修复、 容灾
C 反击	Counterattack	取证和依法打击	取证、溯源、证据保全

记忆公式：

WPDRRC = 预警在前，保护为基，检测发现，响应处置，恢复业务，反击取证。

易错提醒：

PDR 只有保护、检测、响应；PDRR 增加恢复；MPDRR 增加管理；**WPDRRC** 最完整，包含**预警和反击**。

2. 信息安全体系架构设计五方面

方面	设计重点
物理安全	环境、设备、介质、防灾、防盗、防破坏
系统安全	网络结构、 操作系统 、应用服务器、安全加固
网络安全	访问控制 、 防火墙 、 入侵检测 、扫描、防 病毒
应用安全	资源共享控制、身份 认证 、数据 备份 、应用权限
安全管理	制度、流程、安全平台、人员培训、应急机制

真题型考法：

给出某单位“部署了**防火墙**和杀毒软件，但缺少安全制度、应急预案、人员培训、**日志审计**”，问安全体系缺陷。答案不能只补技术设备，必须补**组织、制度、审计、应急和人员意识**。

六、18.5 网络安全体系架构设计

教材基于 OSI 安全体系结构，定义 5 类**安全服务**、8 类**安全机制**，并说明**认证**、**访问控制**、**机密性**、**完整性**、**抗抵赖**等框架。

1. OSI 五类安全服务

服务	解决问题	常见技术
鉴别 服务	你是谁	口令、证书、令牌、生物特征
访问控制 服务	你能做什么	ACL、RBAC、ABAC、策略判定
数据 机密性 服务	防泄露	加密 、路由控制、流量填充
数据 完整性 服务	防篡改	摘要、MAC、 数字签名 、序列号
抗抵赖 服务	防否认	数字签名 、时间戳、第三方公证

2. OSI 八类安全机制

常考机制包括：**加密**机制、**数字签名**机制、**访问控制**机制、数据**完整性**机制、**鉴别**交换机制、业务流填充机制、**路由控制**机制、公证机制。

3. 分层多点防御

教材强调**网络安全**不能只依赖单点技术，而应形成**多点技术防御**、**分层技术防御**、**支撑性基础设施**。这与 2026 案例题趋势高度一致：**云平台**、工业互联网、电子政务、金融系统常要求“**纵深防御**”。

答题模板：

答题模板： **网络安全**架构应采用**纵深防御**思想，在网络和基础设施、边界、计算环境三个区域进行多点防护；在外部边界、内部边界、主机、应用和数据层部署多层机制；通过 **PKI**、密钥管理、**日志审计**、检测响应平台支撑持续运行。

4. 认证框架

鉴别方式 5 类：

1. 用户知道的：**口令**、PIN。
2. 用户拥有的：IC 卡、**令牌**、**证书**。
3. 用户固有的：指纹、人脸、虹膜等**生物特征**。

- 4. 可信第三方：CA、KDC、身份提供方。
- 5. 环境特征：主机地址、位置、设备指纹等。

易错提醒：

认证解决“身份是谁”；**授权**解决“能做什么”；**审计**解决“做了什么”。

5. 访问控制框架

核心公式：

$$\text{访问控制} = \text{发起者} + \text{AEF} + \text{ADF} + \text{目标} + \text{ADI} + \text{策略规则}$$

缩写	含义
ACI	访问控制信息
ADI	访问控制判决信息
ADF	访问控制判决功能
AEF	访问控制实施功能

考试抓手：

ADF 负责“判断允许还是拒绝”，AEF 负责“执行允许或阻止”。

6. 机密性、完整性、抗抵赖框架

框架	目标	手段
机密性	信息只对 授权 者可用	禁止访问、物理保护、 路由控制 、 加密
完整性	防止未经 授权 改变或损毁	访问控制 、 数字签名 、密封、序列号、摘要
抗抵赖	防止发送方或接收方否认行为	证据生成 、证据存储、 证据验证 、纠纷解决

抗抵赖四阶段：

证据生成 → 证据传输/存储/恢复 → 证据验证 → 纠纷解决

七、18.6 数据库系统的安全设计

数据库安全目标是保证数据的**完整性**、**机密性**、**可用性**，常用**安全策略**包括用户管理、存取控制、**数据加密**、**审计**跟踪和攻击检测。

1. 数据库安全评估标准

标准/概念	备考抓手
TCSEC	又称橘皮书，分 D、C、B、A 等级
TDI	TCSEC 在数据库管理系统方面的扩充解释
D 类	基本无安全保护
C 类	有一定安全保护，但通常不称真正安全产品
B 类及以上	强制访问控制 ，真正意义上的安全产品

易错提醒：

考试若问“真正意义上的安全产品”，按教材口径选 **B1 及以上**。

2. 数据库完整性设计原则

数据库**完整性**是数据的正确性和相容性，**完整性**约束可以由 DBMS 实现，也可以由应用程序实现。

原则	答题关键词
静态约束优先放数据库模式	主键 、 外键 、 唯一性 、 检查约束
动态约束多由应用程序实现	状态变化、业务流程规则
实体完整性 和 引用完整性 优先保证	主键 、 外键
慎用 触发器	性能开销、多级触发难控
命名规范前置	易识别、易维护
充分测试 完整性 约束	排除冲突和性能问题
专职数据库设计小组	统一负责分析、设计、测试、实施、维护

3. 数据库完整性设计阶段

需求分析 → 概念结构设计 → 逻辑结构设计 → 物理/实施与测试

案例表达：
答题模板：数据库安全设计应先识别用户、角色、数据密级和业务规则，再通过用户管理、访问控制、加密、审计、备份和攻击检测保障安全；完整性方面，应在需求阶段识别业务规则，在概念模型和逻辑模型中落实实体完整性、引用完整性和用户自定义完整性，并通过测试排除约束冲突和性能影响。

八、18.7 系统架构的脆弱性分析

安全架构不仅要考虑“如何防”，还要考虑“哪里弱”。教材用“木桶理论”说明：系统安全性往往由最薄弱环节决定。脆弱性可能存在于物理设备、软件、协议、数据库、应用、人员、制度和**安全策略**中。

1. 漏洞来源

来源	说明
软件设计瑕疵	协议、架构、组件组合不当
软件实现弱点	编码错误、接口处理不当
软件本身缺陷	未检查输入、未处理资源耗尽、系统调用不当
系统和网络错误配置	默认配置、弱口令、开放无用服务

2. 软件脆弱性特点

- 1. 脆弱性是隐藏弱点，本身未必立即造成危害。
- 2. 逻辑错误是大量脆弱性的根源。
- 3. 与具体系统环境密切相关。
- 4. 修补旧漏洞时可能引入新漏洞。

3. 脆弱性生命周期

引入 → 产生破坏效果 → 修补 → 消失

引入原因常见为：输入验证错误、权限检查错误、操作序列化错误、边界检查错误、设计缺陷等。破坏效果常见为：非法执行代码、非法修改对象、非法访问数据、拒绝服务。

4. 典型架构脆弱性

架构	脆弱性重点	备考判断
分层架构	层间依赖、底层错误扩散、通信开销	层次越多，调用链越长
C/S	客户端被分析、数据截取、二层直连数据库	客户端含敏感信息很危险
B/S	Web 暴露面大、HTTP 风险、易受 Web 攻击	需认证、HTTPS、WAF、输入校验
事件驱动	组件顺序不确定、数据传递复杂、死循环、高并发一致性	异步解耦但流程难控
MVC	结构复杂、视图控制器耦合、数据访问效率低	分离职责但增加复杂性
微内核/插件	插件通信开销、整体优化困难、插件可信问题	扩展性强但需插件安全
微服务	分布式复杂、服务通信失败、服务治理复杂	要治理、熔断、认证、链路追踪

九、18.8 安全架构设计案例

1. 电子商务/RADIUS/AAA 案例

AAA 分别是：

AAA	含义	作用
Authentication	认证	判断用户是否能接入

Authorization	授权	判断用户能使用哪些服务
Accounting	审计/计费	记录资源使用情况

RADIUS 架构分三层：协议逻辑层、业务逻辑层、数据逻辑层。协议逻辑层处理网络协议，业务逻辑层处理认证、授权、计费，数据逻辑层通过数据库代理池降低数据库压力。其高性能设计重点是分层、进程/线程处理、共享内存、数据库代理池、负载均衡、自我监控。

案例答题模板：
答题模板： 对高并发接入认证系统，应采用分层架构，将协议处理、业务处理和数据访问解耦；通过多进程/多线程、共享内存、数据库连接池或代理池提升吞吐量；通过负载均衡避免单点瓶颈；通过加密认证、日志审计和进程监控保证安全性和可用性。

2. 混合云工业安全架构案例

教材案例将混合云工业安全生产管理系统分为设备层、控制层、设计/管理层和应用层，并要求重点考虑设备安全、网络安全、控制安全、应用安全、数据安全。

安全方面	设计重点
设备安全	维护、检测、保养、设备状态监控
网络安全	防火墙、入侵检测、漏洞扫描、防病毒
控制安全	冗余、容错、降级、备份、容灾
应用安全	密码策略、访问控制、服务器告警、用户策略
数据安全	加密、完整性、强认证、备份、异地容灾

第 19 章 大数据架构设计理论与实践

一、本章考试定位

第 19 章不是单纯介绍“大数据技术名词”，而是考查架构师在海量数据、实时数据、历史数据、异构数据、复杂查询、低延迟与高可靠之间做架构取舍的能力。综合知识题容易考概念、特征、层次、优缺点；案例题容易给出“日志分析、广告平台、智能决策、物联网实时监控、视频播放分析”等场景，要求选择 Lambda 或 Kappa 架构并说明理由；论文题可作为“大数据平台架构设计”“实时计算平台设计”“数据中台/数据湖架构设计”“高并发数据分析系统设计”的素材。

本章主线可以压缩为一句话：
传统数据库架构难以支撑海量、实时、异构数据处理 → 大数据系统要求鲁棒、容错、低延迟、可扩展、可调试 → Lambda 用批处理层+加速层+服务层兼顾历史全量与实时增量 → Kappa 用统一流处理链路简化代码与口径 → 具体项目按历史分析、实时性、维护成本、技术栈能力进行取舍。

二、传统数据处理系统的问题

传统系统最初通常是“应用直接访问数据库”。当访问量和数据量提升后，会出现数据库响应慢、请求超时、写入阻塞、硬件错误频繁、运维复杂度上升等问题。常见补救方式包括异步队列、读写分离、分库分表、分区、复制、重分区脚本等。教材图 19-1 展示了在 Web 服务器和数据库之间增加异步队列以缓冲写请求；图 19-2 展示读写分离；图 19-3 展示基于 Hadoop 的 M/R 管道。

传统优化手段	作用	根本局限	考试判断
异步队列	削峰、缓冲写请求	不消除数据库最终写入压力，工作处理层跟不上时仍堆积	只能缓解，不能根治
读写分离	主库写、从库读，提升读性能	一致性、同步延迟、故障切换复杂	适合读多写少
分库分表/水平分区	分散单库压力	应用需感知分区位置，resharding 成本高	扩展性提升但复杂度上升
复制/副本	提升可靠性和读能力	一致性、同步、存储成本增加	高可用常用但非免费
重分区脚本	迁移数据、扩容分区	容易写错分区，测试和运维要求高	典型运维风险点

易错提醒：
“加队列”不等于解决过载；“分库分表”不等于架构简单；“备份”不等于人为错误可恢复。大数据系统真正要解决的是数据持续增长、低延迟查询、分布式容错、错误可恢复、架构复杂度可控。

三、大数据处理系统面临的挑战

教材将大数据处理系统的挑战归纳为三类。第一类是处理非结构化和半结构化数据。大数据中结构化数据占比低，图像、文本、日志、社交网络数据、用户行为数据等往往是半结构化或非结构化数据，需要转换、建模、抽取特征和挖掘规律。第二类是刻画复杂性与不确定性。大数据具有高维、多变、随机性强等特点，需要从复杂数据中发现可用模式。第三类是处理数据异构性与决策异构性的关系，即把一次挖掘得到的“粗糙知识”与经验、偏好、场景知识等主观知识结合，形成可支持管理决策的“智能知识”。

高频考点压缩

挑战	含义	架构应对
非结构化/半结构化	数据格式不统一，不能直接用关系表处理	数据采集、清洗、建模、特征抽取、数据湖/湖仓
复杂性/不确定性	数据维度高、变化快、随机性强	分布式计算、模型训练、指标体系、可追溯计算
数据异构/决策异构	数据来源不同，决策目标不同	统一数据模型、指标口径、元数据、二次挖掘
一次挖掘	从数据中提取潜在模式	得到“粗糙知识”
二次挖掘	结合经验、常识、偏好、业务场景	形成可决策的“智能知识”

四、大数据处理系统的 8 个架构特征

Nathan Marz 提出的大数据系统属性是本章综合知识高频点，也可直接写入案例题“架构设计目标”。

特征	备考解释	案例题写法
鲁棒性和容错性	机器会宕机，人会误操作，系统要能恢复	采用不可变数据、重算视图、副本、故障迁移
低延迟读写	读写响应需达到毫秒到秒级	实时层/缓存/索引/预计算视图
横向扩容	通过增加机器维持性能	scale out 优先于 scale up
通用性	支持金融、社交、电商、物联网等场景	不与单一业务强耦合
延展性	新需求、新视图、新算法可加入	插件化、模块化、统一数据模型
即席查询	用户可按临时需求查询	数据仓库、OLAP、索引、搜索引擎
最少维护	组件和算法简单，降低故障率	减少复杂链路和双系统维护
可调试性	每个值能追踪来源和计算过程	血缘、日志、审计、可观测性

记忆口诀：

容错低延扩，通展即维调。
即：容错、低延迟、可扩展、通用、延展、即席查询、少维护、可调试。

五、Lambda 架构

1. Lambda 架构核心思想

Lambda 架构由 Nathan Marz 提出，目标是构建同时满足高容错、低延迟、可扩展、可持续更新的大数据系统。它把离线计算和实时计算结合起来，以不可变主数据集、读写分离、复杂性隔离为核心思想，可集成 Hadoop、Kafka、Spark、Storm、HBase 等组件。教材图 19-4 展示了 Lambda 的三层结构：批处理层、加速层、服务层。

结构公式：

Lambda 架构 = Batch Layer + Speed Layer + Serving Layer

最终查询结果 = Batch View + Real-time View

2. 三层结构

层次	处理对象	主要职责	输出	特点
批处理层 Batch Layer	全量历史数据	存储主数据集，预计算查询函数	Batch View	精准、全量、延迟高
加速层 Speed Layer	最近增量数据流	实时处理新数据，更新实时视图	Real-time View	低延迟、可能近似、不全量
服务层 Serving Layer	查询请求	合并批处理视图和实时视图	最终查询结果	面向用户低延迟查询

3. 批处理层的 3 个数据属性

批处理层管理主数据集，主数据集必须满足：
原始 + 不可变 + 永远真实

这三个词是选择题和案例题高频点。**原始**表示尽量保留源数据；**不可变**表示不覆盖历史事实；**永远真实**表示**主数据集**是**重算**和**纠错**的依据。

4. Monoid 特性

教材强调 **Monoid** 的**结合律**特性在分布式计算中很重要。满足**结合律**的函数可以拆分到多台机器并行计算，再合并局部结果。例如求和天然满足；平均值不直接满足，但可拆成“总和/数量”，总和和数量都可**并行**累加。

公式化记忆：

可并行聚合 = 局部计算 + 结果合并
Avg = Sum / Count

考试用法：

如果题目问“为什么某些查询适合**预计算**和**分布式合并**”，答：因为查询函数满足或可转化为满足 **Monoid** 特性的函数，可将计算拆分、并行执行、保存局部结果并合并最终结果。

5. Lambda 架构优点与缺点

方面	内容
优点	容错性 好；查询灵活；易伸缩；易扩展；适合 历史全量+实时增量 并存
缺点	需要维护批处理和流处理两套代码；编码开销大；重新部署和迁移成本高； 服务层 合并逻辑可能复杂

6. Lambda 与 Event Sourcing、CQRS

模式	核心思想	与 Lambda 的关系
Event Sourcing	系统以事件为核心，事件持久化，业务数据是事件产生的视图	Lambda 的 主数据集 思想与事件溯源相似，错误可通过 事件重放/重算 修复
CQRS	命令写与查询读分离	Lambda 通过批处理/流处理写出 View ，查询直接读 View ，本质上体现 读写分离

易错提醒：

Lambda 不是简单的“两个系统拼接”，而是用**不可变**数据和 **View** 模型把批处理结果与实时结果统一到**服务层**。**服务层**若设计不好，会成为查询瓶颈。

六、Kappa 架构

1. Kappa 架构核心思想

Kappa 架构由 Jay Kreps 提出，它不是 **Lambda** 的完全替代，而是其简化版本。**Kappa** 删除 **Batch Layer**，以统一的流处理链路处理**实时数据**，并通过**日志重放**处理**历史数据**。其本质是强化 **Speed Layer**，使它既能实时处理，也能在业务逻辑变化时重新处理**历史数据**。教材图 19-10 展示了 **Kappa** 数据处理架构。

结构公式：

Kappa 架构 = 数据流/**日志** + 流处理引擎 + **服务层/结果视图**
历史重算 = **日志保留** + **Offset** 回放 + **新视图**替换旧视图

2. 数据系统本质

教材给出一个非常重要的公式：

数据系统 = 数据 + 查询

数据有两个关键性质：

性质	含义	架构影响
When	数据与时间相关，有全局先后顺序	日志、事件流、Offset、Watermark 都围绕时间顺序
What	数据本身 不可变 ，过去事实不能修改	Update/Delete 可理解为新增事实，而非覆盖事实

进一步可记：

传统 **CRUD** → 大数据**不可变**模型下更接近 **CR**
Update/Delete = 新增一条修正事实

3. Kappa 实现步骤

以 **Kafka** 为例，**Kappa** 的实现过程可以压缩为四步：

- 1. 部署 **Kafka**，并设置**日志保留期 Retention Period**。
- 2. 当业务逻辑或算法更新时，启动新的作业实例。

3. 将 Log Offset 设为 0，从历史日志重新计算并生成新视图。
4. 新视图追上旧视图后，应用切换到新视图，再停止旧实例并删除旧视图。

4. Kappa 优点与缺点

方面	内容
优点	实时与离线代码统一；维护简单；数据口径一致；避免 Lambda 中 Batch View 与 Real-time View 合并问题
缺点	消息中间件长期缓存历史数据压力大；大量回溯重算消耗实时计算资源；多实时流关联容易受乱序影响；放弃离线计算的稳定性

5. Kappa 架构变形

变形	解决的问题	核心做法
Kappa+	解决长期日志保留和 backfill 成本高	流计算直接读取 HDFS/数据仓库数据，结合 Hudi 等支持增量和更新
混合分析系统	弥补 Kappa 展示层和热点查询能力不足	Kafka+Flink 处理流数据，再对接 ElasticSearch 做热点索引分析

易错提醒：

Kappa 不是“不存数据”，而是依赖长期日志、数据湖或历史存储来支持重放。Kappa 简化了计算链路，但对流处理引擎、消息系统、乱序处理、状态管理能力要求更高。

七、Lambda 与 Kappa 对比和选择

对比维度	Lambda	Kappa	选型提示
架构复杂度	批处理+流处理两套链路	一套流处理链路	想降低维护成本选 Kappa
开发维护	两套代码，测试和运维复杂	一套代码，口径统一	算法频繁改动选 Kappa
计算开销	批处理和实时处理都要运行	必要时回放重算	常态开销 Kappa 较低
实时性	速度层提供实时视图	流处理天然实时	二者都可实时
历史数据处理	批处理层适合大规模历史分析	依赖日志回放，压力较大	历史分析重选 Lambda
即席查询	批处理/数仓更适合	原生能力较弱，需 ES/OLAP 补充	探索性分析选 Lambda
结果准确性	Batch View 最终修正实时近似	依赖流处理结果和重放正确性	强一致历史结果偏 Lambda
技术栈	Hadoop/Spark/Storm/HBase/Hive	Kafka/Flink/Samza/ES/Hudi	按团队技术能力选

选择口诀

历史多、批量重、即席查、全量准 → Lambda

实时强、链路简、代码一套、流式主 → Kappa

既要实时又要热点分析 → Kappa + ES/OLAP/湖仓

离线训练 + 实时推断难统一 → Lambda

八、案例分析归纳

1. 某网奥运大数据平台：Lambda 场景

该案例用于说明 Lambda 适合“大型活动实时概览+赛事历史回顾”。当日概览需要秒级刷新在线人数、浏览量、停留时间、播放次数等实时指标；赛事回顾需要查询自定义时间段内的历史最高在线人数、逐日走势、点播排行等历史统计。平台采用数据集成层、数据存储层、数据计算层、数据应用层，并通过离线计算、实时计算和合并计算协同工作。

案例题答法：

该场景同时存在实时增量查询和历史全量统计，适合 Lambda 架构。

实时层处理在线人数、地域频道分布等秒级指标；

批处理层对历史播放、浏览、停留等数据做离线预计算；

服务层合并 Batch View 与 Real-time View，向前端提供低延迟查询。

2. 某网广告平台：Lambda 演进场景

该案例重点考“标准 Lambda 不是万能模板”。第一版严格按 Lambda 搭建，但服务层需要同时查 MySQL、Redis、业务库并做聚合，导致服务层复杂且性能不足。第二版把第三方曝光数据同步提前到实时处理层。第三版进一步将实时转化数据与曝光数据合并到一张 MySQL 统计表中，把原本服务层的关联逻辑提前到数据采集和计算阶段，最终降低查询复杂度。

案例启示：

服务层不要承担过重的关联、聚合、补偿逻辑。

能在采集层、计算层预处理的，应尽量前移；
最终对外暴露简单稳定的查询模型。

3. 某证券实时日志分析平台：Kappa 场景

该案例用于说明 **Kappa** 适合“**日志**实时采集、实时清洗、实时指标、**实时监控**”。平台通过 **Filebeat** 采集**日志**，**Kafka** **缓存**，**Flink** 清洗解析与指标加工，**ElasticSearch** 存储**日志**，**OpenTSDB** 存储时序指标，再提供搜索、可视化和**全息监控**。

案例题答法：

日志数据天然是时间序列流，适合 **Kappa 架构**。

采用 **Filebeat + Kafka + Flink + ElasticSearch/OpenTSDB**；

Kafka 保证数据流缓冲和顺序管理；

Flink 实时清洗、解析、聚合；

ES 支持**全文检索**，**OpenTSDB** 支持时序指标查询；

监控告警通过独立**消息队列**保持有序和实时推送。

4. 某电商智能决策平台：Kappa 场景

该案例说明 **Kappa** 适合“实时点击、下单、曝光、出价数据驱动算法参数更新”。平台通过 **Kafka** 接收 B 端**实时数据**，**Flink** 过滤和聚合指定**时间窗口**指标，结果写入 **Hive** 和 **Tair**，决策服务从 **Tair** 读取**数据计算**新参数，通过 **Zookeeper** 通知客户端拉取并**本地缓存**，业务系统可**低延迟**使用最新参数。

案例启示：

实时决策类系统应优先保证数据新鲜度、参数更新及时性和业务调用**低延迟**。

实时计算结果进入**缓存**，**服务端计算参数**，客户端**本地缓存**参数，可减少网络调用延迟。

九、公式熟练与答题模板

1. 核心公式

数据系统 = 数据 + 查询

数据 = **When** + **What**

Lambda = **Batch Layer** + **Speed Layer** + **Serving Layer**

最终结果 = **Batch View** + **Real-time View**

批处理层主数据集 = **原始** + **不可变** + **永远真实**

Kappa = **日志流** + 流处理 + 结果视图

历史**重算** = **Offset 归零** + **日志重放** + **新旧视图切换**

Scale out = 增加机器数量

Scale up = 增强单机性能

Avg = Sum / Count

2. 案例题通用架构描述模板

该系统面对海量、多源、异构、**实时数据**，传统数据库直接支撑会出现写入瓶颈、查询延迟和扩展困难。

架构设计上应采用大数据处理架构，将**数据采集**、存储、计算、服务和应用展示分层解耦。

数据采集层通过**日志采集/消息队列**接入多源数据；

数据存储层保存**不可变原始**数据和计算结果；

计算层根据场景选择 **Lambda** 或 **Kappa**；

服务层通过**索引**、**缓存**、**预计算视图**提供**低延迟**查询；

同时配套**容错**、扩容、监控、**审计**、**脱敏**和**权限控制**，保证系统可靠、安全、可演进。

3. Lambda 选择模板

本场景既需要实时查询，又需要对大规模**历史数据**进行全量统计和**探索性分析**，适合 **Lambda 架构**。

批处理层保存**不可变主数据集**并生成 **Batch View**；

速度层处理最近增量数据并生成 **Real-time View**；

服务层合并两类视图对外提供查询。

该方案**容错性**好、历史分析能力强，但需要维护两套计算逻辑，应通过统一**指标口径**和**自动化测试**降低维护成本。

4. Kappa 选择模板

本场景数据持续以流形式产生，业务以实时处理和近实时决策为主，历史分析需求可通过**日志**回放或**数据湖**补偿完成，适合 **Kappa 架构**。

系统以 **Kafka** 等**消息队列**承载数据流，以 **Flink** 等流处理引擎完成清洗、聚合和指标计算，结果写入**搜索引擎**、**时序库**、**缓存**或数仓。

该方案链路简单、代码统一、口径一致，但要重点解决**日志保留**、**乱序处理**、**状态恢复**、**回溯重算**和**存储成本**问题。

第 20 章 系统架构设计师论文写作要点

一、本章定位：论文不是背知识点，而是考“架构师能力呈现”

系统架构设计师论文的本质不是单纯考教材概念，也不是泛泛考写作能力，而是考考生能否把**架构理论**、**项目实践**、**问题分析**、**方案设计**、**实施效果**组织成一篇可信、完整、有深度的工程论文。教材明确强调，论文考查重点包括：是否具备系统架构设计**实践经验**、是否能分析和解决问题、是否具备清晰表达能力。

1. 论文核心能力模型

能力维度	考试要求	论文中应体现的内容
实践经验	证明自己参与过真实或可信项目	项目背景、规模、业务场景、技术环境、 本人角色
架构思维	能从全局而非代码细节看问题	架构目标 、 约束条件 、 质量属性 、 技术选型 、 权衡取舍
问题分析	能发现关键矛盾	性能瓶颈、扩展性不足、可靠性风险、安全风险、耦合过高
方案设计	能提出架构级解决方案	分层 、 微服务 、 SOA 、 事件驱动 、 缓存 、 异步 、 容灾 、 安全架构 等
实施效果	能证明方案落地有效	性能提升、故障减少、扩展性增强、维护成本降低、用户满意
表达能力	条理清楚、重点突出	摘要 完整， 正文 分段合理，论点与题目 三问 对应

备考结论
论文不是“我知道什么”，而是“我在某个项目中，面对什么问题，基于什么架构思想，采取了什么措施，取得了什么效果”。

二、2026 考试口径修正：教材笔试表述要转为机考训练

教材中关于“方格纸、用黑色中性笔、每格一个字”的内容，是旧笔试场景下的实务建议。**2026** 年官方通知明确，**系统架构设计师**高级资格采用计算机化考试；高级资格综合知识和案例分析连考，论文单独考试 **120 分钟**，**不得提前交卷**。官方通知还列出了**机考**可用输入法，包括微软拼音、极点五笔、搜狗拼音、搜狗五笔等。

机考口径下的备考调整

教材原口径	2026 备考修正
练习方格纸、控制手写速度	改为练习 机考 输入速度，重点训练 120 分钟 完成 摘要 和 正文
使用黑色中性笔、保持卷面整洁	改为保持段落清晰、标点规范、结构醒目
按格子估算字数	改为按输入字数控制： 摘要 约 300~400 字 ， 正文 约 2000~3000 字
文字要写在格子内	改为避免大段堆叠，控制自然段长度
字迹工整	改为术语准确、逻辑清楚、无明显错别字和语病

易错提醒
不能因为**机考**就忽略字数要求。**摘要**过短、**正文**不足、**正文没有完整收尾**，仍然是论文**高风险扣分点**。

三、论文准备：先准备项目，再准备技术，最后准备表达

教材强调，不同考生应根据**实践经验**差异采取不同准备策略：经验丰富者要整理项目经验；经验欠缺者要学习案例、文档和范文；学生或项目经验较少者要强化阅读、模仿和练习。

1. 项目素材准备公式

本章没有计算类公式，但论文备考有固定结构公式：

可写项目 = 真实背景 + 明确规模 + 本人角色 + 架构问题 + 技术措施 + 实施效果 + 经验反思
一个可用于论文的项目，至少应准备以下信息：

准备项	写作要求	示例方向
项目名称	具体， 不要 笼统	某省级政务服务平台、某企业供应链协同系统
项目背景	说明为什么建设	原系统并发低、数据孤岛严重、业务扩展困难
项目规模	体现复杂度	用户量、并发量、模块数、数据量、团队规模

准备项	写作要求	示例方向
本人角色	必须 明确	架构设计负责人、核心架构师、技术负责人
架构问题	与论文题目挂钩	可用性、性能、安全、扩展性、集成、建模
方案措施	2~3 个重点展开	分层架构 、 微服务 拆分、 缓存 、消息队列、网关、安全控制
实施效果	尽量量化	响应时间降低、部署效率提升、故障恢复时间缩短
不足反思	体现真实项目经验	监控初期不足、服务粒度需优化、 数据治理 仍需完善

2. 近三年项目优先原则

教材建议主题项目一般应选择**近 3 年**内完成的项目。项目过于久远，容易让阅卷人怀疑技术背景、业务环境和架构方案是否仍有现实意义。

备考建议
 至少准备 2 个项目模板。一个偏业务信息系统，如政务、金融、教育、医疗、供应链；一个偏技术平台，如数据中台、**微服务**平台、**统一认证**平台、**云原生**改造平台。这样可以覆盖不同论文题目。

四、论文时间分配：120 分钟必须写完

教材给出的论文解答步骤包括：选题、构思、**摘要**、**正文**、检查。时间参考为：选题 3 分钟、构思 12 分钟、**摘要** 15 分钟、**正文** 80 分钟、检查 10 分钟。

推荐机考时间表

阶段	建议时间	任务重点
选题	3 分钟	选最熟悉、最能匹配项目的题目
构思	10~12 分钟	写出 三问 对应提纲， 不要 直接开写
摘要	12~15 分钟	写清项目、角色、主题、措施、效果
正文	80 分钟左右	按题目 三问 展开，重点写实践
检查	8~10 分钟	查错字、逻辑矛盾、结尾缺失、 题目偏离

易错提醒
不要在技术背景部分写太久。论文得分重点不是“背了多少理论”，而是“能否把理论用于项目”。

五、摘要写作：300~400 字，必须有实质内容

教材强调，**摘要**应控制在 **300~400 字**；没有**摘要**、**摘要**过于简略或缺少实质内容都会明显影响得分；如果**摘要少于 120 字**，风险极高。

1. 摘要结构公式

摘要 = 项目背景 + 本人角色 + 论文主题 + 关键措施 + 实施效果 + 不足或改进

摘要要素	写法要求	不建议写法
项目背景	简洁说明业务场景和建设原因	大篇幅讲行业意义
本人角色	明确“我/本人/作为架构设计负责人”	只写“我们团队”而无个人职责
论文主题	必须 扣题	写成万能项目介绍
关键措施	概括 2~3 项架构措施	只写“采用先进技术”
实施效果	写结果、收益、评价	只写“效果良好”
不足改进	简短即可	写太多缺陷导致否定项目

2. 通用摘要模板

本文以我参与的某某系统建设项目为背景，讨论某某架构设计问题。该项目面向某类用户，主要实现某某业务功能，建设过程中面临某某性能、可靠性、扩展性或安全性问题。我在项目中担任系统架构设计负责人，主要负责总体架构设计、关键**技术选型**和核心方案落地。针对上述问题，我从某某架构划分、某某关键机制、某某**质量属性**保障三个方面开展设计工作，并在实施过程中结合项目实际进行了优化。系统上线后，在响应时间、并发处理能力、可维护性、稳定性等方面取得了较好效果。本文结合该**项目实践**，对相关设计方法、实施过程和经验教训进行总结。

摘要高分关键
少写空泛意义，多写“项目 + 角色 + 措施 + 效果”。

六、正文写作：正文不是摘要延长，而是摘要扩展

教材指出，**正文**要求约 **2000~3000 字**，建议最佳字数约 **2500 字**左右。**正文**不是**摘要**的延伸，而是**摘要**的扩展；**摘要**是**正文**的抽象，两者**不能**前后矛盾。

1. 正文推荐结构

正文部分	建议字数	写作重点
第一部分：项目概述	300~500 字	背景、目标、规模、 本人角色
第二部分：理论与方法	500~700 字	回答题目中的专业知识问题
第三部分： 项目实践	1000~1400 字	结合项目写 2~3 项架构措施
第四部分：效果与总结	300~500 字	效果、问题、改进、经验

2. 正文展开公式

实践段 = 问题背景 + 决策依据 + 方案设计 + 实施过程 + 实际效果 + 反思改进

示例：

论题方向	实践展开点
架构风格	为什么选 分层、微服务、事件驱动、C/S、B/S、SOA
建模方法	用例图、类图、时序图、DFD、ER 图、领域模型 如何支持设计
质量属性	性能、可用性、安全性、可修改性如何转化为架构策略
微服务架构	服务拆分、 服务治理、接口管理、配置中心、熔断限流
安全架构	身份认证、权限控制、数据加密、 日志审计 、安全隔离
云原生架构	容器化、弹性伸缩、 灰度发布 、服务编排、 监控告警
数据架构	数据 分层、主数据 、数据同步、 数据质量、数据治理

七、教材实例一：软件系统建模方法及其应用

教材给出的实例一是“论软件系统**建模方法**及其应用”。该题要求先叙述项目和本人的工作，再说明常用**建模方法**，最后结合项目说明采用了哪些**建模方法**及效果。

1. 常用建模方法整理

建模方法	核心思想	适用场景	论文写法
功能分解法	按功能逐层分解系统	业务边界较清晰、模块功能明确的系统	写清模块划分、接口关系、功能层次
数据流法/结构化分析	跟踪数据流动和加工过程	数据处理流程明显的管理信息系统	写 DFD 、数据字典、加工说明
信息工程建模法	识别实体、属性和关系	数据关系复杂、重视 数据库设计 的系统	写 ER 模型、实体关系、 主数据
面向对象建模法	用对象映射问题域	业务对象复杂、交互行为多的系统	写 类图、时序图 、对象关系、封装继承

2. 真题对应写法

遇到“**建模方法**”类论文，**不能**只罗列 **UML** 图形。应说明：

- 为什么需要建模：降低复杂性、**统一沟通语言**、指导设计和开发。

- 用了哪些模型：业务用例、领域模型、数据模型、接口模型、部署模型。
- 模型如何发挥作用：帮助需求确认、服务划分、数据库设计、接口设计、测试验证。
- 实施效果：减少需求歧义、降低返工、提升沟通效率、支撑后续维护。

易错提醒
建模论文最容易写成“画了很多图”。高分写法不是“画了什么”，而是“这些模型如何帮助架构决策”。

八、教材实例二：软件架构风格

教材给出的实例二是“论软件架构风格”。该题要求说明常用架构风格，并结合项目说明采用了哪种架构风格及原因。教材列举了管道/过滤器、数据抽象和面向对象、事件隐式调用、分层系统、仓库/黑板、C2、C/S、三层 C/S、B/S 等风格。

1. 常用架构风格备考表

架构风格	核心特征	优点	局限或注意点
管道/过滤器	数据流经多个处理构件	适合批处理、数据转换、流水线处理	交互性弱，不适合复杂状态共享
面向对象	数据与操作封装为对象	易复用、易维护、贴近业务领域	对象划分不当会导致复杂依赖
事件驱动	通过事件触发组件响应	解耦强、扩展灵活	调试复杂，事件顺序和一致性需控制
分层架构	上层调用下层服务	结构清晰、职责明确	层间调用过多可能影响性能
仓库/黑板	以中央数据结构为核心	适合共享数据和知识推理	中央数据可能成为瓶颈
C/S	客户端与服务器分工	交互能力强，适合局域网业务	客户端维护成本高
三层 C/S	表示层、功能层、数据层分离	扩展性、维护性优于二层 C/S	层间接口设计要求较高
B/S	浏览器、Web 服务器、数据库服务器	部署维护方便，适合互联网应用	前后端安全、性能和兼容性需关注
SOA/微服务	服务化、自治、接口契约	适合复杂业务系统演进	服务治理、分布式事务、监控要求高

2. 架构风格论文写作重点

架构风格题不要停留在定义层面，要回答三个问题：

问题	应写内容
为什么选这种风格	业务复杂度、团队规模、扩展要求、部署环境、质量属性
如何落地这种风格	层次划分、模块划分、服务划分、接口设计、数据访问方式
产生什么效果	降低耦合、提升扩展性、提高部署效率、增强可维护性

示例表达

在该项目中，我没有简单采用传统单体架构，而是基于业务边界和系统扩展要求，采用分层架构与服务化设计相结合的方式。表现层负责用户交互，业务服务层承载核心业务规则，数据访问层屏蔽数据库差异；对订单、用户、支付、消息等变化频繁且边界清晰的能力，则进一步抽象为独立服务。这样既保持了系统结构清晰，又为后续业务扩展和弹性部署提供了基础。

九、关键技术储备：论文必须能迁移前面章节知识

教材第 20 章强调，论文题目要求在掌握架构师专业知识的基础上，把专业知识应用到工程项目中。可能涉及的关键技术包括经典架构风格、SOA、微服务、质量属性设计、架构评估、架构建模和描述等。

高频论文方向与前置知识对应

论文方向	需要调用的教材知识	实践落点
架构风格	分层、C/S、B/S、SOA、微服务、事件驱动	为什么选型、如何组织系统
质量属性	性能、可用性、安全性、可修改性、可测试性	质量属性场景、策略、效果
架构评估	ATAM、SAAM、风险点、敏感点、权衡点	如何发现架构风险
系统建模	UML、DFD、ER、领域模型	如何支撑需求和设计
安全架构	认证、授权、加密、审计、访问控制	安全风险与防护体系
云原生架构	容器、微服务、弹性、监控、DevOps	部署效率和弹性能力

论文方向	需要调用的教材知识	实践落点
数据架构	数据治理、数据集成、主数据、数据质量	数据一致性和共享能力

十、常见问题与解决办法

教材总结了论文写作中的典型问题，包括走题、字数不够、字数过多、摘要归纳欠妥、文章深度不够、缺少特色、口语化、表达能力弱、缺乏主题项目、项目年代久远、结构死板、段落太长等。

论文失分点速查表

常见问题	表现	解决办法
走题	只看标题，不看三个问题	正文标题直接对应题目三问
字数不足	摘要过短，正文空泛	训练固定结构，正文写到约 2500 字
字数过多	理论铺垫太长	只深入写 2~3 个核心措施
摘要空洞	全是背景意义	摘要必须出现项目、角色、措施、效果
深度不够	技术点很多但都很浅	选 2~3 项措施深入展开
缺少特色	像教材概念堆砌	写项目中的具体问题和取舍
口语化	“我觉得”“差不多”“挺好”	使用正式工程语言
缺主题项目	泛称“银行系统”“政务系统”	必须写具体项目
项目太旧	技术背景过时	优先选择近 3 年项目
结构死板	全文大一二三、小 123	分段自然，标题清晰即可
段落太长	一段十几行	每段围绕一个观点，控制长度

十一、高分论文的“项目实践段”写法

项目实践段是论文得分核心，建议每个技术措施都按固定闭环展开：
问题 → 原因 → 方案 → 实施 → 效果 → 反思

示例：性能优化方向

环节	写作内容
问题	高峰期接口响应时间长，数据库压力大
原因	热点查询集中、同步调用链长、缓存缺失
方案	引入缓存、异步消息、读写分离、接口限流
实施	对高频基础数据设置缓存，对非核心流程异步化
效果	响应时间下降，并发能力提升
反思	后续补充缓存一致性和监控告警机制

示例：可用性方向

环节	写作内容
问题	单点故障影响核心业务
原因	应用部署集中，缺少故障隔离
方案	集群部署、负载均衡、熔断降级、容灾备份
实施	核心服务多实例部署，关键接口设置降级策略
效果	故障影响范围缩小，恢复速度提高
反思	需要持续完善演练机制和监控指标

示例：安全架构方向

环节	写作内容
问题	多角色、多系统访问存在越权风险

环节	写作内容
原因	权限模型不统一，接口鉴权不完整
方案	统一认证、RBAC 权限控制、数据加密、日志审计
实施	建立用户、角色、权限、资源的统一模型
效果	降低越权访问风险，提高审计追溯能力
反思	后续可加强细粒度权限和异常行为分析

十二、论文选题策略：选最能匹配项目的题

教材建议，临场应选择自己最熟悉、把握最大的题目；如果没有完全对应经历，也可选择自己比较熟悉的技术相关题目。

选题优先级

优先级	选择标准
第一优先	项目经历与题目完全匹配
第二优先	项目经历可通过架构措施迁移匹配
第三优先	技术熟悉但项目匹配度一般
不建议	题目看起来简单但没有项目支撑

临场判断口诀

口诀

能写项目，优先项目；能写措施，优先措施；能写效果，才算稳妥。

十三、论文结尾写法：不要突然结束

结尾不需要太长，但**必须**完成**闭环**。建议写三层意思：

- 本项目通过某某架构设计解决了某某问题。
- 实施后在性能、可用性、扩展性、安全性或维护性方面取得效果。
- 后续仍需在监控、治理、自动化、**数据质量**或安全体系方面继续优化。

通用结尾模板

通过上述架构设计与实施，本项目较好地解决了原系统在某某方面存在的问题，系统上线后运行稳定，能够满足当前业务发展需要。实践表明，系统架构设计**不能**停留在**技术选型**层面，而应结合业务目标、**质量属性**和实施约束进行综合权衡。当然，项目仍存在某某方面的改进空间，后续将继续从监控治理、自动化运维和架构演进等方面进行优化。